

## **АННОТАЦИЯ**

Отчёт 118 с., 8 ч., 51 рис., 10 табл., 47 источников, 1 прил.

**РАСПОЗНАВАНИЕ И ЛОКАЛИЗАЦИЯ ИЗОГНУТОГО ТЕКСТА, ТРАНСФЕРНОЕ ОБУЧЕНИЕ, РАСПОЗНАВАНИЕ ОБЪЕКТОВ НА СЛОЖНЫХ ГРАФИЧЕСКИХ СЦЕНАХ, ИСКУССТВЕННЫЕ НЕЙРОННЫЕ СЕТИ, КОМПЬЮТЕРНОЕ ЗРЕНИЕ, МАШИННОЕ ОБУЧЕНИЕ**

Разработана легковесная модификация модели нейронной сети EAST, позволяющая с высокой скоростью выполнять задачу локализации текстовых областей на изображениях, содержащих сложные графические сцены.

Представлены теоретические сведения, связанные с наиболее актуальными методами локализации и распознавания текста. Реализованы и протестированы модель нейронной сети CRNN+CTC-loss и сквозная модель распознавания текста на изображениях, основанная на моделях EAST и CRNN+CTC-loss.

Предложена легковесная модификация сквозной модели FOTS, основанная на разработанной легковесной модификации модели EAST и модели CRNN+CTC-loss.

# СОДЕРЖАНИЕ

|                                                                                   |           |
|-----------------------------------------------------------------------------------|-----------|
| <b>ВВЕДЕНИЕ.....</b>                                                              | <b>6</b>  |
| <b>1 ОСНОВНЫЕ ПОНЯТИЯ КОМПЬЮТЕРНОГО ЗРЕНИЯ .....</b>                              | <b>14</b> |
| 1.1 Классификация .....                                                           | 14        |
| 1.2 Сегментация.....                                                              | 15        |
| 1.3 Классификация с локализацией.....                                             | 16        |
| 1.4 Детекция.....                                                                 | 16        |
| 1.5 Бинаризация.....                                                              | 16        |
| <b>2 ОСНОВНЫЕ ПОНЯТИЯ НЕЙРОННЫХ СЕТЕЙ.....</b>                                    | <b>18</b> |
| 2.1 Полносвязный слой.....                                                        | 18        |
| 2.2 Сверточный слой.....                                                          | 20        |
| 2.3 Слой субдискретизации.....                                                    | 22        |
| 2.4 Слой нормализации по мини-батчам .....                                        | 24        |
| 2.5 Рекуррентный слой .....                                                       | 25        |
| <b>3 ОБЗОР ПОДХОДОВ К РЕШЕНИЮ ЗАДАЧИ .....</b>                                    | <b>31</b> |
| 3.1 Подходы к локализации изогнутого текста.....                                  | 33        |
| 3.2 Подходы к распознаванию изогнутого текста .....                               | 35        |
| <b>4 АКТУАЛЬНЫЕ НЕЙРОСЕТЕВЫЕ МЕТОДЫ ДЕТЕКЦИИ И<br/>РАСПОЗНАВАНИЯ ТЕКСТА .....</b> | <b>45</b> |
| 4.1 ResNet .....                                                                  | 45        |
| 4.2 MobileNet .....                                                               | 47        |
| 4.3 Модели детекции текста.....                                                   | 49        |
| 4.3.1 EAST.....                                                                   | 49        |
| 4.3.2 CRAFT.....                                                                  | 56        |
| 4.4 Модели распознавания текста .....                                             | 59        |
| 4.4.1 CRNN.....                                                                   | 60        |
| 4.5 Сквозные модели.....                                                          | 68        |
| 4.5.1 FOTS .....                                                                  | 68        |
| <b>5 ИССЛЕДОВАНИЕ МОДЕЛИ CRNN .....</b>                                           | <b>73</b> |
| 5.1 Описание эксперимента .....                                                   | 73        |

|                                                             |            |
|-------------------------------------------------------------|------------|
| 5.2 Методика оценивания.....                                | 74         |
| 5.3 Описание данных .....                                   | 75         |
| 5.4 Результаты эксперимента.....                            | 76         |
| <b>6 ОБЪЕДИНЕНИЕ МОДЕЛЕЙ ДЕТЕКЦИИ И РАСПОЗНАВАНИЯ .....</b> | <b>77</b>  |
| 6.1 EAST .....                                              | 77         |
| 6.2 CRNN.....                                               | 78         |
| 6.3 Результаты .....                                        | 78         |
| <b>7 МОДИФИЦИРОВАНИЕ АРХИТЕКТУР .....</b>                   | <b>80</b>  |
| 7.1 Легковесная модель детекции .....                       | 80         |
| 7.2 Легковесная сквозная модель .....                       | 82         |
| <b>8 ИССЛЕДОВАНИЕ МОДИФИЦИРОВАННЫХ АРХИТЕКТУР .....</b>     | <b>84</b>  |
| 8.1 Описание эксперимента .....                             | 84         |
| 8.2 Методика оценивания.....                                | 84         |
| 8.3 Описание данных .....                                   | 86         |
| 8.4 Результаты .....                                        | 88         |
| 8.4.1 Объединение моделей .....                             | 89         |
| 8.4.2 Легковесный EAST .....                                | 89         |
| 8.4.3 Легковесный FOTS .....                                | 94         |
| <b>ЗАКЛЮЧЕНИЕ .....</b>                                     | <b>96</b>  |
| <b>БИБЛИОГРАФИЧЕСКИЙ СПИСОК .....</b>                       | <b>98</b>  |
| <b>ПРИЛОЖЕНИЕ. ПРОГРАММНЫЙ КОД .....</b>                    | <b>105</b> |

## ВВЕДЕНИЕ

Письменная коммуникация играет большую роль в современном мире. Текст – это не только книги, сообщения, электронная почта и документы. Текст пронизывает различные аспекты нашего окружения: от продуктов, которые мы покупаем, до улиц наших городов. Рекламные щиты, объявления на стенах домов, вывески магазинов, дорожные знаки – всё это даёт нам большое количество информации каждый день.

Автоматическое обнаружение и распознавание текста на любой естественной сцене представляет из себя интересную задачу и имеет широкий спектр применения. Решение этой задачи способно значительно упростить взаимодействие человека с окружающим миром.

Но стоит заметить, что преобразование изображений, содержащих текст в естественных условиях, в текстовую информацию – задача сложная с технической точки зрения. В отличие от традиционного метода оптического распознавания символов (Optical Character Recognition или OCR), который фокусируется на распознавании текста в отсканированных документах, распознавание текста на подобных графических сценах нацелено на идентификацию любых символов в любой среде.

Далее изображения, содержащие текст в естественных условиях, будем называть "сложными графическими сценами", поскольку в большинстве случаев такие изображения получаются в непростых для съемки местах и могут иметь неоднородный фон, содержать различные искажения и зашумления, а также текст переменного качества и различных характеристик. Примеры таких изображений могут включать фотографии уличных вывесок, кадры из кинофильмов с субтитрами и данные в системах навигации роботов. В англоязычной литературе распознавание текста на изображениях такого характера принято называть термином "Scene Text Recognition" (STR).

Как правило, STR можно разделить на два этапа: детекцию (обнаружение) и распознавание. Детекция текста включает в себя определение областей

изображения, которые, вероятно, содержат текст. Распознавание текста предполагает преобразование этих идентифицированных областей текста в последовательности символов. На рисунке 1 [1] показана работа STR-алгоритма, который выделяет области с текстом в прямоугольники, а затем распознает символы, обеспечивая транскрипцию для каждого из обнаруженных прямоугольников.

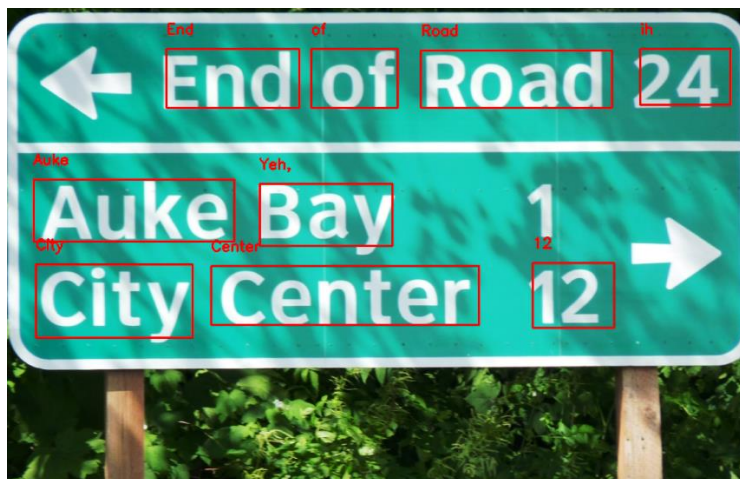


Рисунок 1 – Результат детекции и распознавания текста

В традиционном оптическом распознавании символов текст всегда находится в хорошо выровненных, черно-белых отсканированных документах, где есть четкий контраст между текстом и фоном. Текст занимает значительную часть изображения, а абзацы имеют предсказуемые и структурированные размеры и формы. Такой подход превосходно решает задачи автоматизации систем учёта и документооборота, перевода книг и рукописей в цифровой вид, анализа больших текстов с помощью поиска и сопоставления с некоторым словарем. Принцип его работы достаточно прост: изображение, содержащее текст, проходит предварительную обработку, затем страница сегментируется на подобласти с символами и, наконец, для каждого символа вычисляется некоторая мера схожести с символами из словаря. Схема этого процесса представлена на рисунке 2.

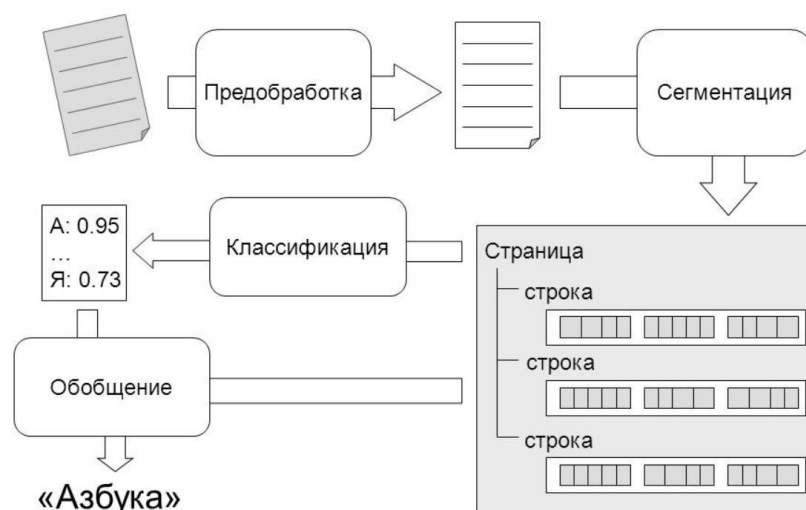


Рисунок 2 – Принцип работы традиционного OCR

Но в отличие от традиционного метода, STR имеет дело с текстом, представленным в различных размерах, формах, цветах, оттенках и контрастах, то есть не имеющего явной локализации. Это значительно усложняет задачу и требует отдельного этапа работы системы. Кроме того, на сложных графических сценах часто встречаются специфические слова, которые не содержатся в заранее заданных словарях, а сам текст во многих задачах может иметь произвольную длину алфавита и самих слов. В этом контексте необходимо рассмотреть основные аспекты сложной графической сцены, примеры которых показаны на рисунке 3.



Рисунок 3 – Проблемы фона, формы и шума, соответственно

Во-первых, сцена характеризуется фоном. Текст можно найти в различных условиях, таких как стены, улицы с вывесками, автомобили и т.д. В некоторых

случаях такой текст не имеет единого фона: помимо самого текста на изображении могут присутствовать другие объекты, такие как деревья, люди, транспорт и т.д. Неоднородный фон, в частности, значительно усложняет задачу бинаризации (приведения изображения к двухцветному), целью которой является уменьшение затрат памяти.

Во-вторых, текст может иметь необычную форму. Характеристики текстов, включая шрифт, размер, ориентацию и даже кривизну, могут существенно различаться. Во многих случаях текст намеренно оформляется орнаментально и изгибается в эстетических целях. Последнее значительно усложняет процесс детекции, даже если текст написан одним из наиболее распространенных шрифтов. Помимо изогнутости, текст также может быть излишне растянутым или, напротив, иметь слишком малое расстояние между символами.

В-третьих, для фотографий характерен шум. При получении изображений текста в естественных сценах могут возникать различные источники шума, включая неравномерное освещение, низкое разрешение, размытие и деформацию формы вследствие движения. Шум может иметь совершенно разную природу, поэтому даже качественная камера и профессиональная съемка не могут избавить от всех последствий зашумления изображений, что затрудняет как детекцию, так и распознавание.

Когда мы говорим о распознавании текста на сложной графической сцене, мы имеем дело с множеством практических применений, особенно в промышленной среде. Различные области, такие как логистика и конвейерный мониторинг, используют эту технологию для эффективной автоматизации своих процессов. Используя STR, эти отрасли могут оптимизировать операции и повысить общую производительность. Например, это избавляет от необходимости вручную вводить информацию о вагонах, проезжающих железнодорожное депо. Пример работы такой STR-системы представлен на рисунке 4 [2].



Рисунок 4 – Автоматическое считывание данных о железнодорожных грузах

В дополнение к многочисленным приложениям в промышленности, системы детекции и распознавания текста, по мере своего развития, смогут открыть широкий спектр возможностей в сочетании с другими технологиями. Приведем в качестве примера области, наиболее активно развивающиеся на данный момент:

- 1) навигация роботов: благодаря интеграции STR роботы (или любые системы обучения с подкреплением, которые используют компьютерное зрение в качестве основного источника данных) смогут более эффективно ориентироваться и взаимодействовать с окружением, поскольку научатся интерпретировать и понимать текстовые или знаковые инструкции;
- 2) визуальные ответы на вопросы (Visual Question Answering): STR может внести свой вклад в системы VQA, позволяя им читать и понимать текстовые вопросы, что, в свою очередь, может привести к более точным и информативным ответам;
- 3) создание подписей к изображениям (Image captioning): в сочетании с STR, системы создания подписей к изображениям могут генерировать описания, которые включают соответствующую им информацию, извлеченную из текста, присутствующего на изображении;



- 4) автоматический перевод текста: STR может облегчить автоматический перевод текста на изображениях, обеспечивая многоязычный перевод в реальном времени для различных мобильных приложений, включая онлайн-переводчики;
- 5) помощь людям с плохим зрением: используя технологию STR, можно разрабатывать приложения, помогающие людям с ослабленным зрением читать и понимать текстовую информацию в их окружении;
- 6) автоматизация систем безопасности: автоматическое считывание текста может стать основой для систем, осуществляющих мониторинг передвижения на различных контрольно-пропускных пунктах. Наиболее очевидным примером является распознавание номерных знаков автомобилей.

Учитывая трудности, присущие STR, в течение многих лет предлагались различные подходы и методы, решающие те или иные проблемы. Ранние исследования в основном опирались на ручную работу с признаками [3]. Однако появление передовых аппаратных возможностей в последние годы способствовало заметному прогрессу методов глубокого обучения, значительно расширив возможности алгоритмов STR. С тех пор как AlexNet [4] продемонстрировал прорывные результаты в конкурсе ImageNet Visual Recognition в 2012 году, область глубокого обучения начала переживать стремительный прогресс.

Конкурс ImageNet посвящен классификации изображений, где заданному набору изображений необходимо присвоить соответствующие метки из 20000 доступных категорий, включающих различные объекты: от животных и их пород до фруктов и прочих предметов, которые мы наблюдаем каждый день. Достижения в области классификации изображений, как следствие, также повлияли на развитие технологий STR, в которых активно начала применяться техника трансферного обучения (Transfer Learning), для повышения эффективности.

С того момента глубокое обучение быстро стало доминирующей технологией в области распознавания текста на изображениях. В частности, сверточные нейронные сети (CNN) получили широкое распространение в качестве архитектуры для обнаружения текста, а рекуррентные нейронные сети (RNN) дали начало алгоритмам распознавания связного текста, благодаря своему свойству запоминания информации.

Целью настоящей работы является исследование актуальных на данный момент моделей нейронных сетей, применяющихся в задачах STR, а также разработка на их основе собственных, более совершенных методов. Акцент был поставлен на создание легковесных алгоритмов, которые требуют меньшее количество вычислительных операций, аппаратных ресурсов и времени для выполнения, но которые, однако, сохраняют способность показывать приемлемый результат. Это очень важно, поскольку аппаратные и временные ограничения часто присутствуют в нейросетевых методах, балансирующих, в некотором смысле, между качеством и ресурсоемкостью.

Работу можно разделить на четыре основных блока:

- 1) в качестве первого шага к пониманию проблемы был проведен обзор и анализ существующих на данный момент методов детекции и распознавания текста на изображениях сложных графических сцен. Для начального эксперимента реализован подход с использованием архитектуры CRNN, которая является наиболее популярной моделью в задаче распознавания символов, и алгоритма CTC-loss. Описан и исследован подход с применением данной архитектуры в задаче распознавания заранее локализованного текста на изображениях, содержащих сложные графические сцены. Проведено исследование архитектуры CRNN на двух наборах данных с использованием дополнительного словаря и без его использования;
- 2) после исследования архитектуры CRNN, она была объединена с одной из наилучших на данный момент (state-of-the-art) моделей детекции – EAST

[5]. В результате было создано приложение, позволяющее последовательно выполнять детекцию и распознавание текста, и которое в дальнейшем послужило неким эталоном качества, дав представление о характеристиках проблемы. Модели были использованы в оригинальной форме, а также применялись предварительно обученные веса, предоставленные их авторами;

- 3) на третьем этапе описан процесс разработки алгоритма детекции текста на сложных графических сценах с нуля, в котором используется трансферное обучение на лучших вариациях архитектур MobileNet [6]. Эти быстрые и простые сверточные нейронные сети, изначально разработанные для классификации изображений на вышеупомянутом конкурсе ImageNet, благодаря своим различным особенностям хорошо подошли для создания специально предназначенной для обнаружения текста легковесной модели. Полученная модель была обучена и протестирована, и показала результаты, сравнимые с теми, что предлагают state-of-the-art архитектуры. Подход к моделированию проблемы детекции был вдохновлен моделью EAST;
- 4) на завершающем этапе разработанный ранее метод детекции текста был объединен со стратегией распознавания, предложенной в [7], и техникой RoiRotate, описанной в [8]. RoiRotate помогает объединить алгоритмы детекции и распознавания, позволяя создать легковесный сквозной (end-to-end) алгоритм распознавания текста. Этот алгоритм решает обе задачи в рамках одной нейронной сети, используя значительно меньшее количество параметров и вычислительной мощности, чем при последовательном использовании отдельных компонентов детекции и распознавания.

# 1 ОСНОВНЫЕ ПОНЯТИЯ КОМПЬЮТЕРНОГО ЗРЕНИЯ

Компьютерное зрение – это область искусственного интеллекта, которая занимается созданием компьютерных алгоритмов и программ для обработки, анализа и понимания изображений. Она находит применение во многих областях, включая медицину, промышленность, автомобилестроение и многие другие.

Большинство алгоритмов компьютерного зрения можно поделить на две части: детекцию (выделение области) и непосредственно распознавание в области детекции. Но выделяют также и некоторые задачи, различия между которыми изображено на рисунке 5 [9]. Рассмотрим каждую из них.



Рисунок 5 – Визуальное описание четырех типов задач нахождения объектов на изображении

## 1.1 Классификация

Под задачей классификации подразумевается отнесение некоторого объекта (в нашем случае изображения), обозначаемого как  $Y$ , к определенной категории  $t \in T$ , где  $T$  – набор классов (категорий). Стоит сказать, что рассматривать задачу классификации изображений мы будем с точки зрения использования

трансферного обучения (Transfer learning), то есть когда существующие алгоритмы адаптируются таким образом, чтобы решать другие задачи.

Сама задача классификации изображений стала набирать популярность сразу после того, как была опубликована работа, в которой описывается архитектура нейронной сети AlexNet [4]. Это далеко не первая сверточная нейронная сеть, однако именно она впервые показала поистине впечатляющие результаты на конкурсе ImageNet, задав, тем самым, вектор развития для последующих сверточных архитектур, количество которых начало стремительно расти вместе с количеством задач, связанных с распознаванием объектов в целом.

Из наиболее значимых архитектур, появившихся в результате прорыва AlexNet, можно назвать ResNet (Residual Network) [10] и MobileNet. Данные архитектуры будут рассмотрены далее в работе, поскольку потребуются для проведения исследований.

## 1.2 Сегментация

Задача, которую решает семантическая сегментация, заключается в том, чтобы определить принадлежность каждого пикселя на изображении к определенной категории. Например, если на изображении есть человек, который переходит дорогу, то модель должна для каждого пикселя определить, относится ли он к телу человека, профилю дороги, дорожному знаку, небу или к какому-либо иному классу.

Использование только семантической сегментации имеет серьезный недостаток в сравнении с задачами распознавания объектов. Он заключается в том, что подобных задачах пиксели маркируются по принадлежности к классу объекта, и при этом не создается различий между объектами. Например, если связный набор пикселей, характеризующий один класс, называется одним объектом, то на исходном изображении два объекта, перекрывающие друг друга, будут определены как один, что не является верным.

В попытках решить эту проблему появилась задача сегментации экземпляров (instance segmentation), в которой каждый объект на изображении определяется отдельно. Модели, решающие задачу сегментации экземпляров, используются, например, для подсчета людей в толпе и автоматического управления транспортом.

### 1.3 Классификация с локализацией

Задача классификации с локализацией предполагает определение местоположения экземпляра объекта на изображении с помощью рамки, ограничивающей его положение (bounding box), и предсказание метки категории класса. Рамка имеет форму прямоугольника, стороны которого параллельны осям исходного изображения, и минимальная площадь рамки обеспечивает полное нахождение экземпляра объекта внутри нее. Модель в данном случае одновременно обучается осуществлять правильную классификацию и наиболее точно определять границы рамки.

### 1.4 Детекция

Задача детекции объектов заключается в выделении нескольких объектов на изображении при определении координат ограничивающих рамок и классификации их из множества заранее заданных классов. В отличие от задачи классификации с локализацией, количество объектов, присутствующих на изображении, заранее неизвестно.

### 1.5 Бинаризация

Бинаризация – это процесс преобразования изображения в градациях серого или цветного изображения в двухцветное изображение, где каждый пиксель классифицируется как черный или белый. Значения каждого пикселя условно

кодируются как «0» и «1». Значение «0» условно называют задним планом или фоном, а «1» – передним планом. Благодаря наличию всего двух цветов размер изображения значительно снижается, сокращаются вычислительные затраты, а точность детекции признаков увеличивается благодаря фильтрации.

Однако, как уже было замечено, эффективность существующих универсальных алгоритмов бинаризации значительно снижается, когда мы имеем дело со сложными графическими сценами, которые характеризуются, в частности, разнородным фоном.

Для решения данной проблемы необходимо дополнительно проводить исследования и сравнение методов бинаризации, учитывать специфику задачи и внедрять этап бинаризации в модели распознавания, что само по себе является достаточно сложной задачей. В связи с этим, вопросы бинаризации вынесены за рамки настоящей работы, а представленные модели нейронных сетей обучаются на цветных изображениях и на изображениях в градациях серого.

## 2 ОСНОВНЫЕ ПОНЯТИЯ НЕЙРОННЫХ СЕТЕЙ

Многие проблемы алгоритмов распознавания текста на изображениях были решены с появлением глубоких нейронных сетей, способных, в частности, справляться с задачей в условиях деформации текста, окклюзий (пересечения текстов) и зашумленности. Последние модели выполняют как детекцию, так и распознавание текста, однако существуют также варианты, выполняющие только одну функцию. Нейронная сеть может быть использована в системе распознавания текста в качестве классификатора. При обучении она получает на вход изображения, анализирует все позиции черных пикселей и выравнивает коэффициенты, минимизируя ошибку. Таким образом, достигается лучший результат распознавания.

На данный момент известно большое количество архитектур нейронных сетей, состоящих из различных типов слоев. Рассмотрим те типы слоёв, которые используются в моделях, представленных в настоящей работе.

### 2.1 Полносвязный слой

Вычисление прямого распространения ошибки для полносвязного слоя нейронной сети – это процесс линейного преобразования входного вектора  $x = (x_1, x_2, \dots, x_m)$  с матрицей весов  $W$  размерности  $n \times (m + 1)$ , где каждый элемент преобразованного вектора затем проходит через нелинейную функцию активации:

$$f(Wx) = \begin{pmatrix} f(w_1^T x) \\ \vdots \\ f(w_n^T x) \end{pmatrix},$$

где  $x$  – вектор входных данных,  $n$  – количество нейронов в слое,  $f$  – функция активации слоя,  $W$  – матрица весов слоя.



Как правило, чаще используют добавление к матрице  $W$  единичного столбца такой же размерности  $n$  вместо прибавления к результату отдельного вектора смещения:

$$W = \begin{pmatrix} w_1 \\ \dots \\ w_n \end{pmatrix} = \begin{pmatrix} w_{11} & \dots & w_{1m} & 1 \\ \dots & \dots & \dots & \dots \\ w_{n1} & \dots & w_{nm} & 1 \end{pmatrix},$$

где  $n$  – количество нейронов в слое,  $m$  – размерность входных данных.

Оба подхода математически эквивалентны. Схема вычисления полносвязного слоя показана на рисунке 6.

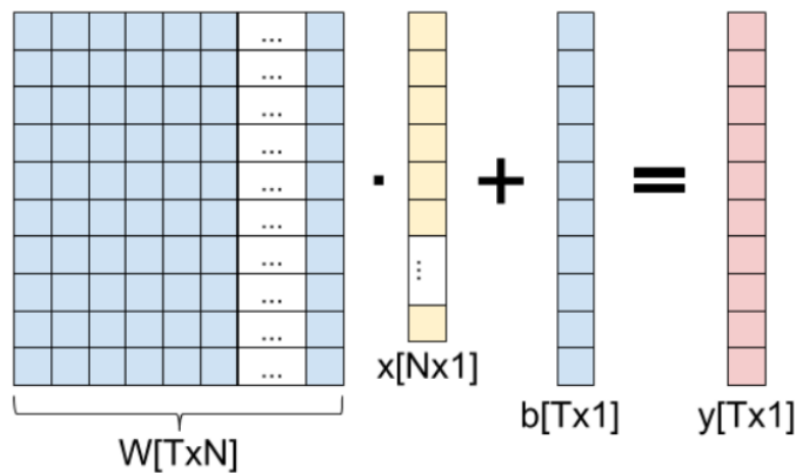


Рисунок 6 – Схема вычисления полносвязного слоя

Обратное распространение ошибки (backpropagation) по полносвязному слою представляет из себя градиент композиции функций:

$$\frac{dt}{dx} = \frac{df}{dx} \times \frac{dt}{df};$$

$$\frac{dt}{dW} = \frac{df}{dW} \times \frac{dt}{df},$$

где  $t$  – значения нейронов следующего слоя,  $f$  – функция активации слоя,  $x$  – значения нейронов слоя,  $W$  – матрица весов слоя.

Градиент по слою  $x$  – это ошибка, которая переходит на предыдущий слой нейронной сети, а градиент по матрице  $W$  используется для обновления значений весов в процессе оптимизации с помощью алгоритма градиентного спуска или его вариаций.

## 2.2 Сверточный слой

Сверточный слой – основной компонент в сверточной нейронной сети. Он состоит из набора обучаемых фильтров, каждый из которых охватывает небольшой участок изображения по ширине и высоте.

При обработке входных данных каждый фильтр покрывает все координаты ширины и высоты изображения, и в каждой такой точке он вычисляет сумму всех своих весов и параметров. В результате этого производится двухмерная матрица активаций фильтра на изображении. Фильтры обучаются нейронной сетью таким образом, чтобы активироваться при обнаружении конкретных визуальных признаков, таких как границы, определенный цвет или даже образы. Если проводить аналогию с человеческим мозгом, каждую точку на матрице активаций можно считать результатом действий нейрона, который обрабатывает маленький участок изображения и передает сигнал последующим нейронам.

Итак, сверточный слой в нейронной сети выполняет операцию, которая состоит в вычислении суммы взвешенных значений входных данных предыдущего слоя с использованием матрицы весов  $W$ . Для каждого элемента входной матрицы  $x$ , находящегося на  $i$ -й строке и  $j$ -м столбце, прямое распространение сверточного слоя с матрицей весов  $W$  размерности  $(2d + 1) \times (2d + 1)$  математически описывается следующим образом:

$$y_{i,j} = \sum_{-d \leq a, b \leq d} W_{a,b} x_{i+a, j+b} ,$$

где  $y_{i,j}$  – результат свертки для элемента с координатами  $(i,j)$ ,  $x_{i+a,j+b}$  – элемент входной матрицы с координатами  $(i+a, j+b)$ ,  $W_{a,b}$  – элемент матрицы весов с координатами  $(a,b)$ . Принцип работы сверточного слоя показан на рисунке 7.

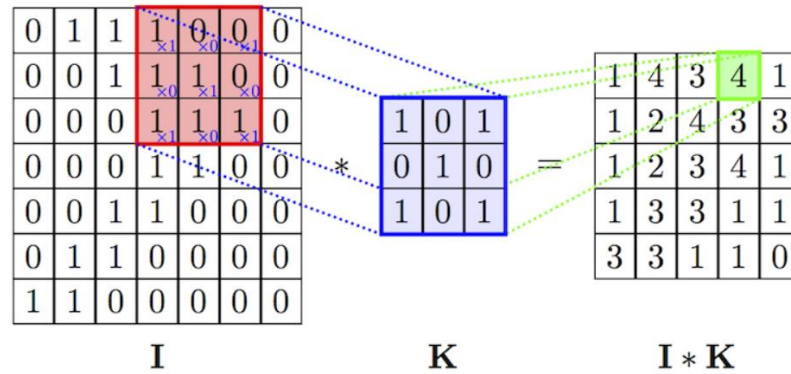


Рисунок 7 – Схематическое представление процесса работы свертки

Для последующего слоя размер матрицы вычисляется по формуле:

$$(w, h) = (mW - kW + 1, mH - kH + 1),$$

где  $w$  – ширина матрицы следующего слоя,  $h$  – высота матрицы следующего слоя,  $mW$  – ширина матрицы предыдущего слоя,  $mH$  – высота матрицы предыдущего слоя,  $kW$  – ширина ядра свертки,  $kH$  – высота ядра свертки.

Часто для сохранения размерности входных данных сверточного слоя используется метод, который называется "padding". Он заключается в добавлении нулевых значений вокруг краев исходной матрицы, для того, чтобы достичь требуемой размерности. Схема работы сверточного слоя с заполненными краями карты признаков изображена на рисунке 8. Как правило, каждый сверточный слой вычисляет несколько карт признаков, каждая из которых имеет свои собственные веса относительно каждой входной карты признаков.

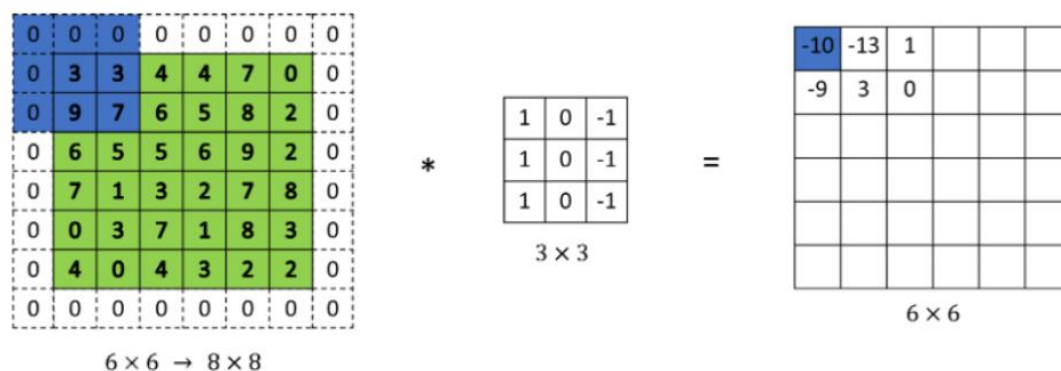


Рисунок 8 – Схематичное представление процесса работы свертки с матрицей, имеющей дополненные нулевыми значениями края

## 2.3 Слой субдискретизации

В архитектуре сверточных нейронных сетей часто применяют пулинг-слои между сверточными слоями, известные также как слои субдискретизации или подвыборочные слои. Главная задача этого слоя – уменьшить размерность данных, передаваемых дальше в нейронную сеть, чтобы, вместе с этим, уменьшить количество вычислений и контролировать переобучение.

Пулинг-слои действуют независимо на каждом слое входных данных и уменьшают разрешение данных, используя операцию макс-пулинга (max pooling). Обычно пулинг-слои имеют фильтры размером  $2 \times 2$ , использованные с шагом 2, чтобы уменьшить размер данных в 2 раза по ширине и высоте, удалив 75% активаций. Значение макс-пулинга в данном случае является максимальным из четырех значений, а глубина остается неизменной. На рисунке 9 показан принцип работы слоя субдискретизации с окном размером  $2 \times 2$  и шагом 2.

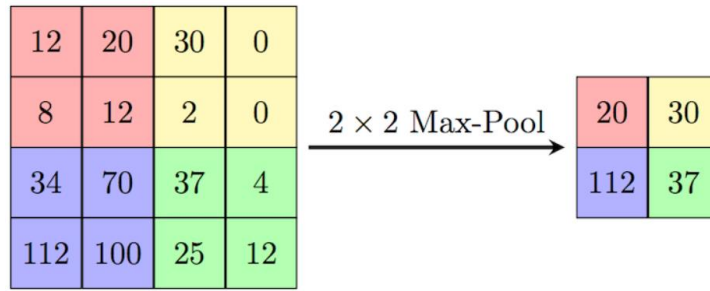


Рисунок 9 – Визуализация принципа работы слоя субдискретизации

Обычно используются пулинг-слои с параметрами  $F = 3$ ,  $S = 2$ , а также часто используются пулинг-слои с  $F = 2$ ,  $S = 2$ . Для данных более высокого разрешения пулинг-слои с большими параметрами могут быть неэффективными. Кроме макс-пулинга, фильтры могут также рассчитывать значения, используя другие функции, например, по среднему значению (average pooling) или по L2-норме. На практике макс-пулинг показал себя лучше любых других видов пулинга.

Обратное распространение ошибки для макс-пулинга заключается в передаче значения градиента к входному значению, имеющему максимальное значение. Иногда индекс максимального значения сохраняется в процессе пулинга, чтобы оптимизировать градиентный спуск во время обратного распространения ошибки.

Саму операцию макс-пулинга математически можно описать формулой:

$$y_{i,j} = \max_{-d \leq a \leq d, -d \leq b \leq d} (x_{i+a,j+b}),$$

где  $x$  – выход предыдущего слоя,  $d$  – размер окна субдискретизации,  $y_{i,j}$  – результат применения макс-пулинга для элемента с координатами  $(i, j)$ .

## 2.4 Слой нормализации по мини-батчам

Метод нормализации по мини-батчам, предложенный в статье [11], на данный момент широко применяется во многих архитектурах нейронных сетей. Использование нормализации по мини-батчам значительно повышает устойчивость нейронных сетей к инициализации весов и снижает вероятность переобучения. Для каждого нейрона вычисляются математическое ожидание и стандартное отклонение по мини-батчу  $x = \{x_1, x_2, \dots, x_m\}$ :

$$\mu_x = \frac{1}{m} \sum_{i=1}^m x_i;$$
$$\sigma_x^2 = \frac{1}{m} \sum_{i=1}^m (x_i - \mu_x)^2,$$

где  $m$  – размер мини-батча,  $x$  – множество значений нейрона по всему мини-батчу.

После этого данные нормализуются посредством вычитания математического ожидания и деления на стандартное отклонение, согласно формуле:

$$x'_i = \frac{x_i - \mu_x}{\sqrt{\sigma_x^2 + \varepsilon}},$$

где  $\varepsilon$  – дополнительно введенная константа, предотвращающая деление на ноль.

Наконец, с учетом дополнительного масштабирования и сдвига нормализации, вычитаем конечный результат по формуле:

$$y_i = \gamma x_i + \beta,$$

где  $\gamma$  – дополнительный оптимизируемый параметр масштабирования нормализации по каждому из элементов,  $\beta$  – дополнительный оптимизируемый параметр для сдвига нормализации по каждому из элементов.

Кроме того, нейронные сети, в которых применяется нормализация по мини-батчам, обычно сходятся быстрее на большинстве задач, чем нейронные сети без такой нормализации. В оригинальной архитектуре CRNN авторы использовали два слоя батч-нормализации.

## 2.5 Рекуррентный слой

Поскольку в распознавании текста имеет значение порядок символов, архитектура CRNN, которая использует сверточные слои при извлечении признаков, для учёта последовательного характера символов использует рекуррентные слои – наиболее распространенный способ работы с любыми последовательностями.

Задачи машинного обучения, связанные с последовательностями, условно разделяют на пять типов (см. рисунок 10):

- 1) "один к одному" (one-to-one): модель принимает один вход и генерирует один выход (например, в задаче классификации изображений);
- 2) "один ко многим" (one-to-many): модель принимает один вход и генерирует последовательность выходов (например, в задаче аннотирования изображений);
- 3) "многие к одному" (many-to-one): последовательность входов соответствует одному выходу (например, в задаче анализа тональности текстовых отзывов);
- 4) "многие ко многим" (many-to-many): последовательность входов соответствует последовательности выходов (например, в задаче машинного перевода);

- 5) "многие ко многим" с синхронизацией: синхронизированная последовательность входов и выходов (например, в задаче классификации видео с маркировками кадров).

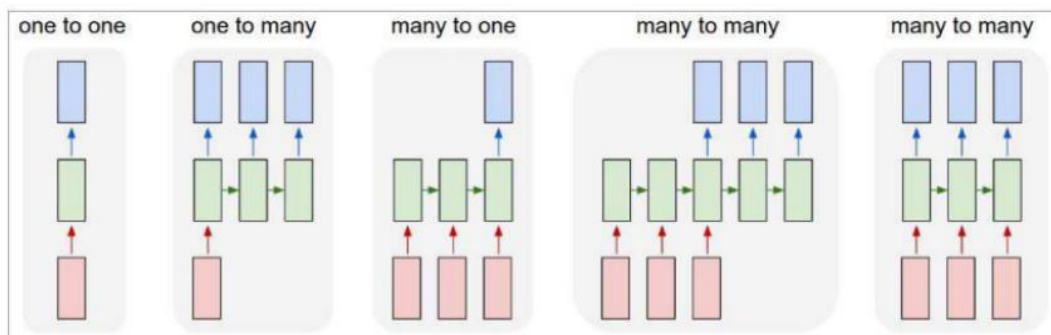


Рисунок 10 – Классификация задач машинного обучения по характеру последовательностей

Рассматриваемая нами задача относится к четвертому типу, где последовательность входов соответствует последовательности выходов без какой-либо синхронизации. Последовательность признаков, которая извлекается сверточными слоями, имеет переменную длину и соответствует последовательности выходных символов в распознаваемом слове.

Основной принцип работы рекуррентных слоев заключается в использовании значений нейронов, полученных на предыдущих итерациях обучения нейронной сети. Его схематичное представление показано на рисунке 11. В рекуррентном слое определяется скрытое состояние  $s$ , которое сохраняется и передается от предыдущего к следующему элементу последовательности.



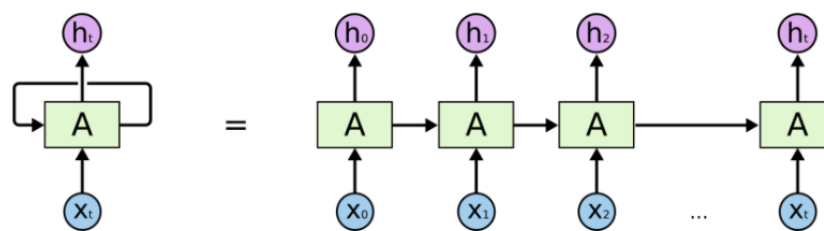


Рисунок 11 – Схематичное представление работы рекуррентного слоя с эквивалентным развернутым во времени представлением, демонстрирующим, как скрытые слои передают информацию из предыдущих временных интервалов

Традиционный рекуррентный слой изображен на рисунке 12 и описывается формулами:

$$s_t = f(Ws_{t-1} + Ux_t + b);$$

$$h_t = g(Vs_t + c),$$

где  $x_i$  – входные данные на текущем временном шаге,  $s_{t-1}$  – значения скрытого состояния на предыдущем временном шаге,  $W$  – матрица весов для скрытого состояния,  $U$  – матрица весов для входных данных,  $b$  – смещение данных перед нелинейностью,  $f$  – нелинейная функция активации рекуррентного слоя (как правило, это функция  $\tanh$ ),  $s_t$  – значение скрытого состояния на текущем временном шаге,  $V$  – матрица весов на выходе слоя,  $c$  – смещение данных на выходе слоя,  $g$  – функция нелинейности на выходе (например,  $\text{softmax}$ ),  $h_t$  – выходные значения слоя.

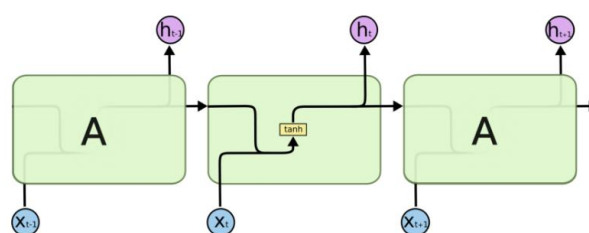


Рисунок 12 – Структура традиционного рекуррентного слоя

Одной из основных проблем классической архитектуры рекуррентных нейронных сетей является проблема длинных зависимостей. Это связано с тем, что при матричном перемножении информация о долгосрочных зависимостях на более ранних временных шагах может быть постоянно перезаписана, что ограничивает память нейронной сети и делает ее неспособной воспроизводить долгосрочные зависимости. Чтобы решить эту проблему, были разработаны более сложные компоненты нейронной сети, такие как LSTM (Long Short-Term Memory).

Ячейки долговременной памяти были предложены в статье [40]. В традиционном блоке RNN существует скрытая связь между входным и выходным слоями, поэтому каждый раз, когда поступает кадр  $x_t$  последовательности, внутреннее состояние  $h_t$  обновляется с помощью нелинейной функции  $g$ :  $h_t = g(x_t, h_{t-1})$ . Затем состояние  $h_t$  используется для предсказания, и таким образом прошлые кадры  $\{x_t'\}_{t' < t}$  рассматриваются для предсказания  $x_t$ .

Слой LSTM содержит три мультипликативных узла (или шлюза), которые условимся далее называть гейтами: забывающий гейт (forget gate), входной гейт (input gate) и выходной гейт (output gate), а также рекуррентную ячейку со скрытым состоянием (cell state). Структура рекуррентного LSTM-слоя показана на рисунке 13.

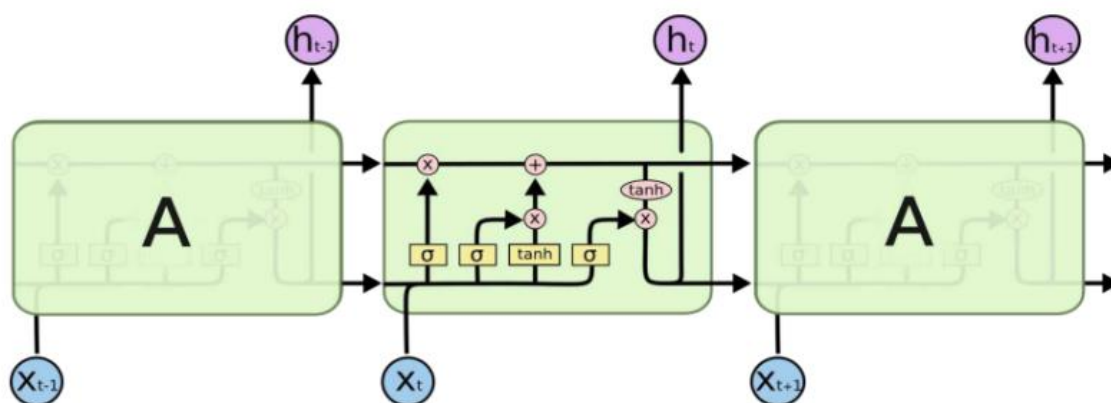


Рисунок 13 – Структура рекуррентного LSTM-слоя

Забывающий гейт  $f_t$  играет важную роль в контроле данных, поступающих с предыдущих временных шагов. Его задача – решить, какие данные из предыдущего состояния должны быть сохранены, а какие должны быть забыты. Для этого используется формула:

$$f_t = \sigma(W_f[h_{t-1}, x_t] + b_f),$$

где  $h_{t-1}$  – скрытое состояние на предыдущем временном шаге,  $x_t$  – входные данные на текущем временном шаге,  $W_f$  – матрица весов забывающего гейта,  $b_f$  – смещение забывающего гейта,  $\sigma$  – нелинейная функция активации (сигмоида).

Входной гейт  $i_t$  обрабатывает входные данные для текущего временного шага и описывается формулой:

$$i_t = \sigma(W_i[h_{t-1}, x_t] + b_i),$$

где  $W_i$  – матрица весов входного гейта,  $b_i$  – смещение входного гейта.

После того, как значения забывающего и входного гейтов были найдены, вычисляются кандидат на изменение состояния ячейки  $C'_t$  (candidate cell gate) и состояние ячейки  $C_t$  (cell state) LSTM по формулам и соответственно:

$$C'_t = \tanh(W_C[h_{t-1}, x_t] + b_C);$$

$$C_t = f_t C_{t-1} + i_t C'_t,$$

где  $W_C$  – матрица весов состояния ячейки,  $b_C$  – смещение состояния ячейки,  $\tanh$  – гиперболический тангенс в качестве нелинейной функции активации.

В выходном гейте вычисляются скрытое состояние в текущий момент времени  $h_t$  и значение выходного гейта  $o_t$ :

$$h_t = o_t \tanh(C_t),$$

$$o_t = \sigma(W_o[h_{t-1}, x_t] + b_o);$$

где  $W_o$  – матрица весов выходного гейта,  $b_o$  – смещение выходного гейта.

Для обработки изображений с использованием нейронных сетей, где известно начало и конец последовательности, применяются двунаправленные (bidirectional) LSTM слои (см. рисунок 14). Они работают по следующему принципу: создаются два независимых LSTM слоя, один из которых передает скрытое состояние от начала последовательности к ее концу, а другой – наоборот, от конца последовательности к ее началу. Затем эти два слоя объединяются. Таким образом, сеть может учитывать информацию о каждом символе не только от предыдущих символов, но и от последующих в процессе обучения.

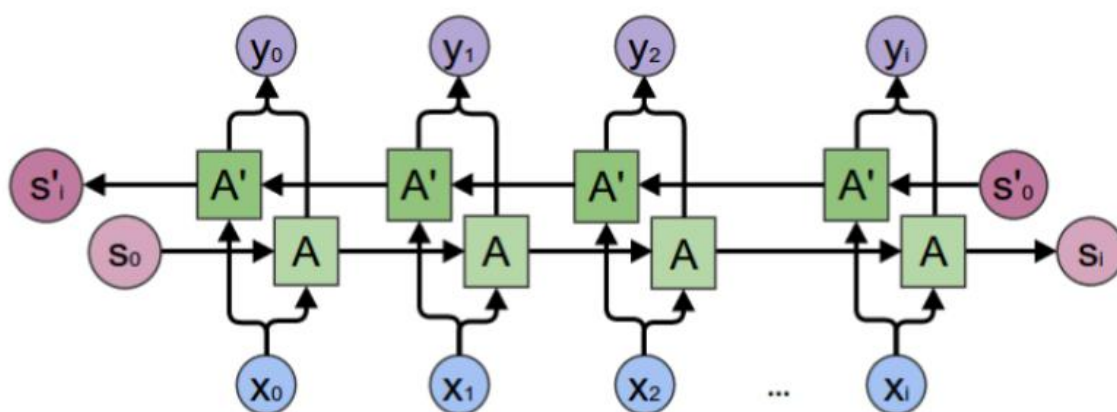


Рисунок 14 – Схема работы двунаправленного LSTM-слоя

### 3 ОБЗОР ПОДХОДОВ К РЕШЕНИЮ ЗАДАЧИ

Как уже было сказано, распознавание текста на сложных графических сценах является одной из наиболее сложных задач в области компьютерного зрения. Это связано с тем, что на таких сценах может содержаться большое количество разнообразных объектов и фоновых элементов, что затрудняет локализацию и сегментацию текста. Кроме того, распознавание текста может быть затруднено на изображениях низкого качества, с шумом и размытием.

Если не брать во внимание характеристики самой сцены, текст самостоятельно может иметь не вполне стандартные свойства. Как правило, всякий распознаваемый текст классифицируется на три типа: машинописный, рукописный и изогнутый.

Машинописный текст (typewritten text) – текст, написанный на компьютере или пишущей машинке. Он имеет регулярную структуру и явную геометрию символов. Ввиду этого, распознавание машинописного текста – проблема, давно решенная традиционными методами оптического распознавания символов.

Рукописный текст (handwritten text) – это любой текст, написанный человеком. Распознавание рукописного текста является более сложной задачей, поскольку символы могут отличаться от установленной пользователем формы, размера и стиля. Более того, разнообразие форм начертания символов является уникальным и зависит от стиля пишущего человека. Однако, и данная проблема на данный момент решается с отличной точностью, если речь не идет о малочисленных языках и, в особенности, языках, имеющих сложное иероглифическое письмо. В качестве примера можно привести тамильский язык.

Изогнутый текст (curved text или multi-oriented text) – основная и наиболее сложная область исследований современного оптического распознавания символов. Это текст, который является нетипичным по форме, может принимать необычные формы и углы, намеренно содержать различного рода искривления.

Более высокие показатели распознавания рукописных символов могут быть достигнуты с использованием контекстной и грамматической информации.

Например, в ходе распознавания искать целые слова в словаре легче, чем пытаться выявить отдельные знаки из текста. Знание грамматики языка может также помочь определить, является ли слово глаголом или существительным. Формы отдельных рукописных символов иногда могут не содержать достаточно информации, чтобы точно (более 98%) распознать весь рукописный текст.

Однако, в настоящей работе речь идет о распознавании изогнутого текста, поскольку именно этот вид текста характерен для сложных графических сцен. Хорошими примерами изогнутого текста могут послужить различные вывески магазинов, представленные на рисунке 15.

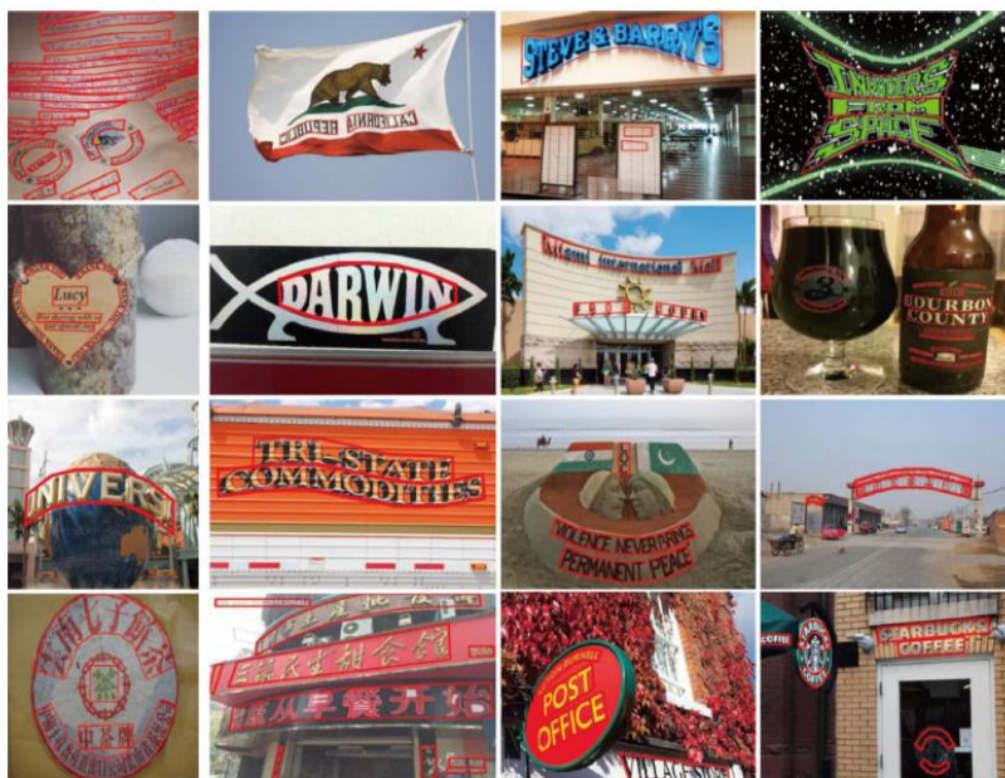


Рисунок 15 – Примеры изображений сложных графических сцен, содержащих изогнутый текст

Общим для большинства алгоритмов распознавания изогнутого текста является процесс выравнивания текста в области детекции по определенному правилу, фактически превращающего искомый текст в прямой машинописный или рукописный.

В настоящей работе особое внимание было уделено решению задачи STR при помощи трёх актуальных моделей нейронных сетей:

- 1) CRNN (Convolutional Recurrent Neural Network). Модель объединяет в себе сверточные и рекуррентные слои для распознавания текста. Процесс свертки извлекает признаки, а рекуррентные слои используют временные зависимости между этими признаками. В отличие от традиционных систем OCR, CRNN может распознавать и транскрибировать текст без сегментации на уровне символов;
- 2) EAST (An Efficient and Accurate Scene Text Detector). Представляет из себя модель исключительно детекции. EAST использует простую, но эффективную архитектуру с полностью сверточной нейронной сетью, и достигла самых высоких результатов на различных эталонных наборах данных, сохраняя при этом быстрое время работы (inference), что делает ее пригодной для использования в сценариях, когда текст распознается в режиме реального времени;
- 3) FOTS (Fast Oriented Text Spotting with a Unified Network). Данная модель является сквозной (end-to-end), то есть способна обнаружить и распознать текст одновременно. Состоит из четырех основных модулей: извлечения признаков, сегментации, распознавания и предсказания.

Представленные модели подробно описаны в следующей главе. В этой главе рассмотрены подходы, которые были изучены в рамках данной работы. Они дают важный исторический контекст для понимания актуальных моделей, которые были созданы в дальнейшем, в том числе на их основе.

### 3.1 Подходы к локализации изогнутого текста

Для локализации текста на изображении можно использовать различные методы, такие как детектирование границ с помощью алгоритмов Canny или Sobel, сегментация изображения с помощью метода k-means или алгоритма Watershed, использование сверточных нейронных сетей (CNN). Известно

большое количество различных алгоритмов сегментации и идентификации области, не использующих глубокое обучение, подробное описание которых, однако, выходит за рамки данной работы. Далее будут рассмотрены лишь некоторые, наиболее популярные методы.

Как уже было сказано, локализация может включать в себя ряд этапов. Помимо идентификации требуемой области, может производиться также сегментация символов или текста в случаях, когда на изображении представлено несколько областей с текстом.

При этом важно выбрать подходящий метод под конкретную задачу, поскольку от качества локализации сильно зависит качество распознавания. В данной работе для этих целей используется, в основном, модель EAST, которая является наиболее универсальным подходом, а также не требует большого количества вычислительных ресурсов. Это также крайне важно, поскольку при проведении исследований не использовалась специализированная аппаратная инфраструктура.

Предлагались методы, основанные на использовании контекста окружения (environmental context) [12]. Ключевой принцип заключался в анализе фона, то есть контекста изображения. Важную роль здесь играет простое эмпирическое предположение: крайне маловероятно, что текст на изображении будет находиться на фоне неба, травы или в другом, неестественном для него окружении.

Также стоит упомянуть и другой интересный подход, основанный на предположении, что в тексте буквы (и, соответственно, слова) имеют примерно одинаковую, постоянную толщину штриха. В одной из подобных работ, например, был предложен алгоритм SWT (Stroke Width Transform), который позволяет выделять признаки, исходя из схожести регионов, содержащих текст. [13]

В рамках данного подхода границы символов можно находить, например, при помощи алгоритма Canny Edge Detector, после чего в направлении градиента



будет вычисляться парная граничная точка, а если градиент является коллинеарным градиенту в первой точке, то все точки между ними заполняются значением найденной ширины. На следующей итерации по найденным отрезкам вычисленное значение ширины ограничивается медианным значением вдоль отрезка. [14]

Одна из первых работ, в которой предлагалось использовать сверточные нейронные сети для решения задачи локализации изогнутого текста, была представлена в 2016 году. [15]

### 3.2 Подходы к распознаванию изогнутого текста

Процесс распознавания текста на сложной графической сцене включает в себя декодирование изображения, локализованного на предыдущем этапе, в последовательность символов. На этом этапе может быть выполнена проверка найденного слова по словарю при помощи алгоритмов проверки орфографии, например, на основе редакционного расстояния (Levenshtein distance), либо с использованием статистических языковых моделей или нейросетевых методов.

Существует три основные категории методов для распознавания текста на изображениях [16]:

- методы на основе символов (character-based) [17] [18];
- методы на основе слов (word-based) [19];
- методы на основе последовательностей (sequence-based) [20].

Методы на основе символов сначала локализуют отдельные символы, а затем классифицируют их и объединяют в слова.

В статье [21] предлагается следующий метод распознавания последовательности цифр: сначала из входного изображения  $X$  размером  $128 \times 128 \times 3$  извлекается вектор признаков  $H \in R^{4096}$  с помощью глубокой сверточной сети. Затем этот вектор передается через 6 независимых без слоев без деления весов (weight sharing) с функцией активации softmax, чтобы вычислить распределение вероятностей  $P(S1|H), \dots, P(S5|H)$  для каждого

символа в последовательности и вероятность  $P(L|H)$  для определения длины последовательности. На рисунке 16 приведены примеры изображений с номерными знаками домов, которые распознаются этой системой.



Рисунок 16 – Примеры изображений с номерами распознаваемых домов

На рисунке 17 представлена визуализация вычислительного графа этой нейронной сети.

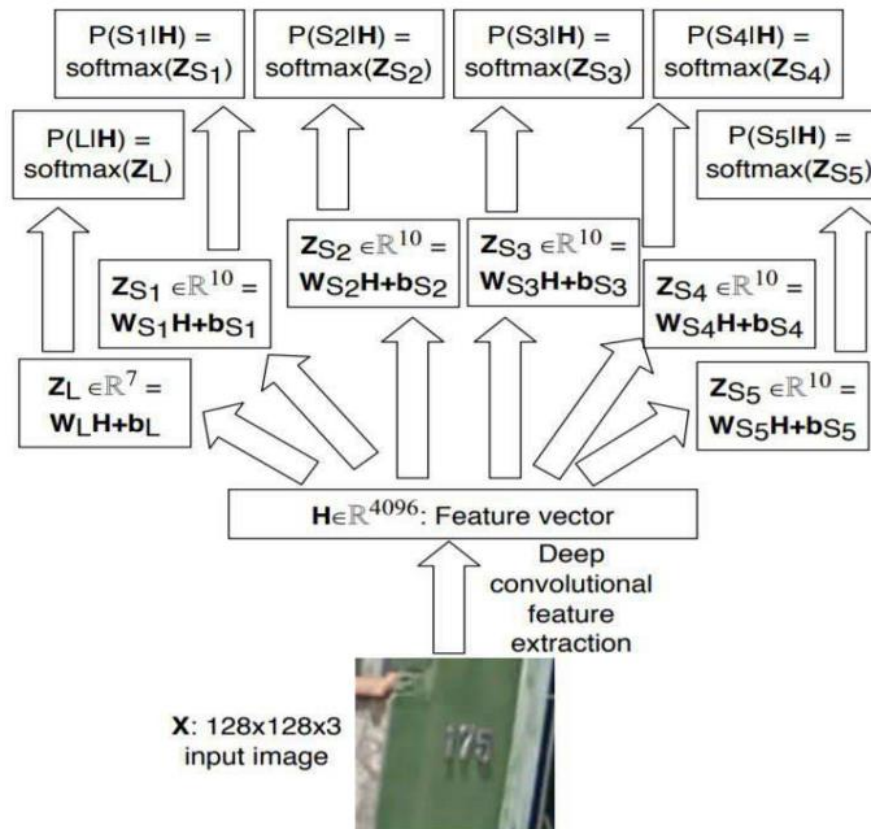


Рисунок 17 – Вычислительный граф нейронной сети, извлекающей вектор признаков  $H \in \mathbb{R}^{4096}$

Методы на основе слов (word-based) отличаются от методов на основе символов тем, что не локализуют и не распознают отдельные символы, а сразу распознают слова целиком [19]. В работе [20] предлагается использовать сверточные нейронные сети, которые работают с заранее определенным словарем слов (см. рисунок 18). Выходной слой такой сети представляет собой вектор распределения вероятностей размером  $1 \times 1 \times N$ , где  $N$  – это число слов в словаре. Этот подход прост в реализации, но не очень гибок в практическом применении при распознавании слов, которых нет в словаре. К таким словам, например, могут быть отнесены названия брендов.

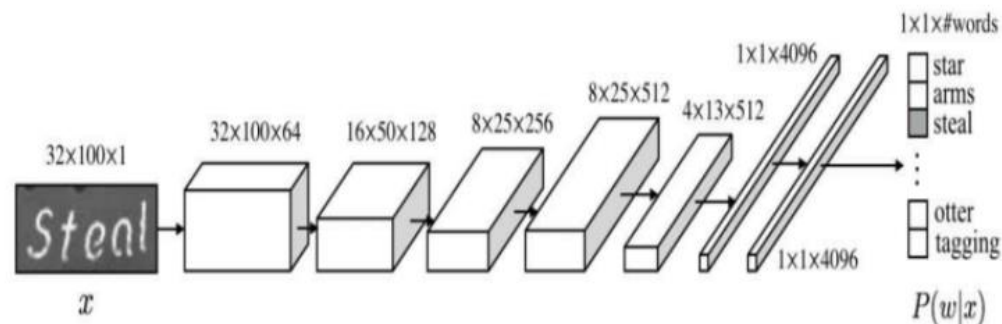


Рисунок 18 – Схема сверточной нейронной сети с заранее заданным словарём

Но наилучшим в нашем случае классом методов являются методы на основе последовательностей (sequence-based). Они заключаются в представлении входного изображения в виде последовательности, которая впоследствии декодируется. Одним из таких методов считается архитектура CRNN.

Важно добавить, что часто, в случаях, когда вышеописанный класс методов не показывает требуемую точность, после обучения нейронной сети может быть также использована дополнительная проверка каждого декодированного слова по заранее определенному ограниченному словарю с последующим посимвольным сопоставлением. Такой подход может быть использован во многих разобранных далее моделях, и зачастую он может привести к повышению точности классификации. Однако следует помнить, что, как уже было сказано, при таком подходе система всё равно не имеет возможности распознавать слова, которых нет в словаре.

Модель CRNN+CTC-loss, предложенная в статье [22] и реализованная в рамках эксперимента в данной работе, сочетает сверточные и рекуррентные нейронные сети. Функция потерь CTC-loss позволяет решать задачу распознавания текста без явного выравнивания между входными данными и выходными метками (ответами). Данный подход и его модификации до сих пор применяются в задаче распознавания текста на изображениях, несмотря на

появление более сложных моделей нейронных сетей. Визуализация архитектуры CRNN представлена на рисунке 19.

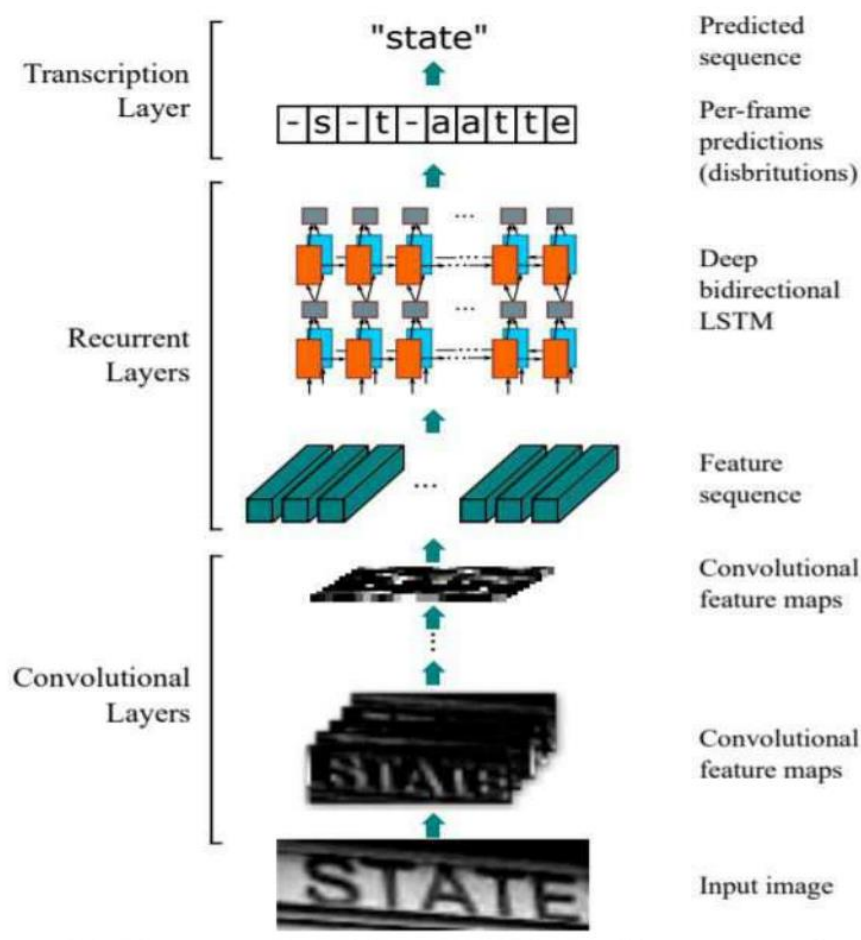


Рисунок 19 – Схема архитектуры CRNN

В работе [23] предлагается использовать нейронные сети с механизмом внимания (attention) перед рекуррентными слоями. Данный подход позволяет нейронной сети обучиться концентрировать своё внимание на наиболее важных участках изображения при распознавании текста. Визуализация архитектуры такой нейронной сети показана на рисунке 20.

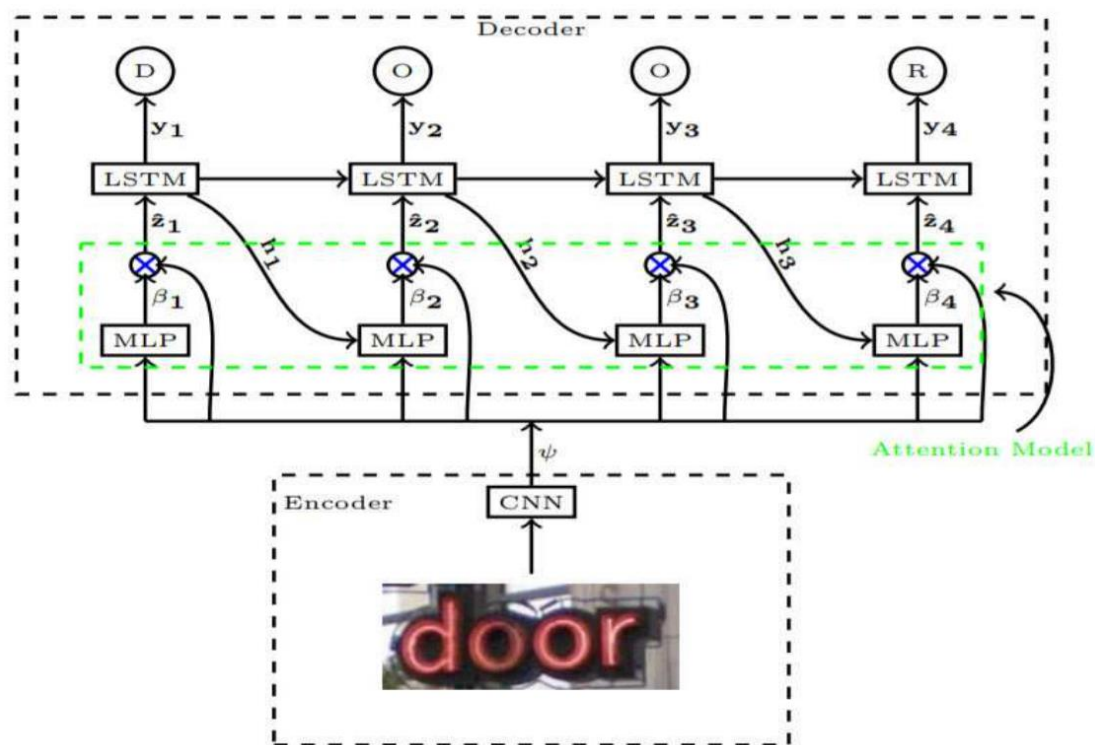


Рисунок 20 – Пример архитектуры нейронной сети, использующей механизм внимания

В работе [24] предлагается в первую очередь выделять на исходном изображении несколько участков при помощи скользящего окна с последующим выделением признаков сверточными слоями, классификацией и передачей алгоритму CTC-loss. Данный подход позволяет снизить размерность входных данных, ускорить обучение и повысить точность распознавания. Визуализация архитектуры такой нейронной сети показана на рисунке 21.

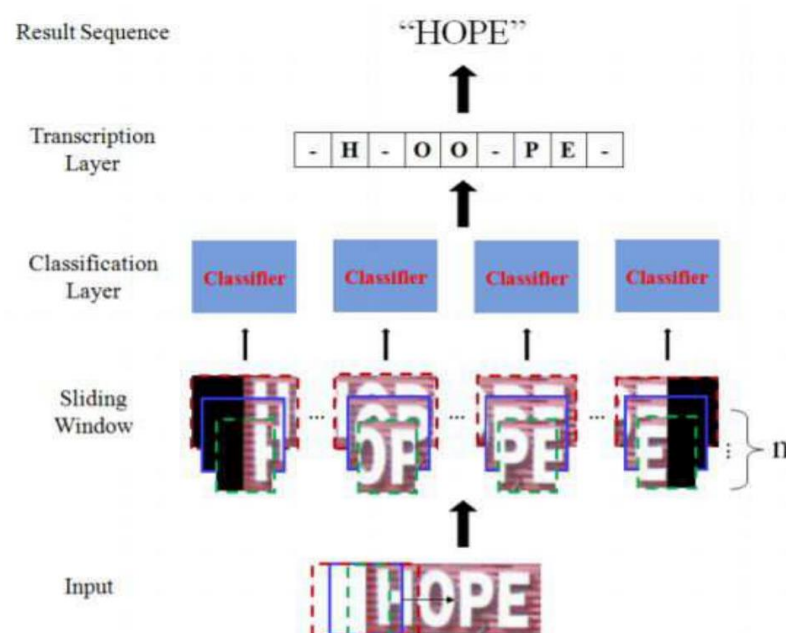


Рисунок 21 – Архитектура нейронной сети, использующей скользящее окно и CTC-loss

В работе [25] было предложено применять нейронные сети без рекуррентных слоев с целью улучшения параллельности вычислений. Для обработки входного изображения было применено скользящее окно, после чего с помощью сверточных слоев извлекались признаки. Затем полученные признаки были переданы на специальный сверточный слой с механизмом внимания (attention), который был использован для кодирования и декодирования последовательностей. Схема данной архитектуры представлена на рисунке 22.

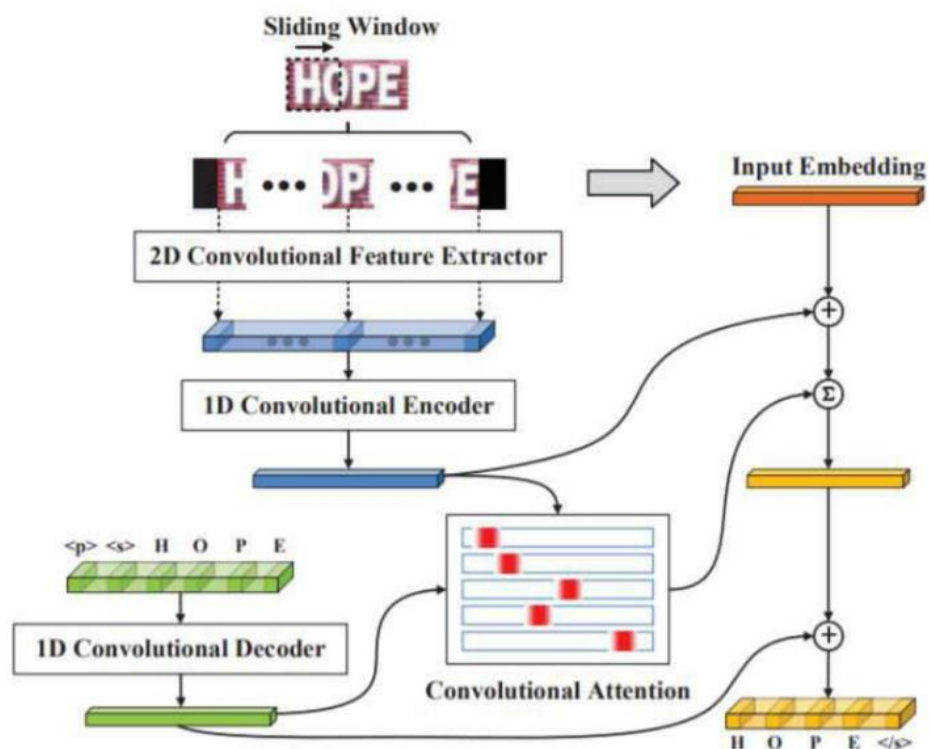


Рисунок 22 – Архитектура нейронной сети, использующей скользящее окно и механизм внимания

В работе [26] предлагается использовать сверточные нейронные сети без рекуррентных слоев, которые используют 2D-механизм внимания (2D-attention). На рисунке 23 представлена визуализация вычислительного графа данной нейронной сети.



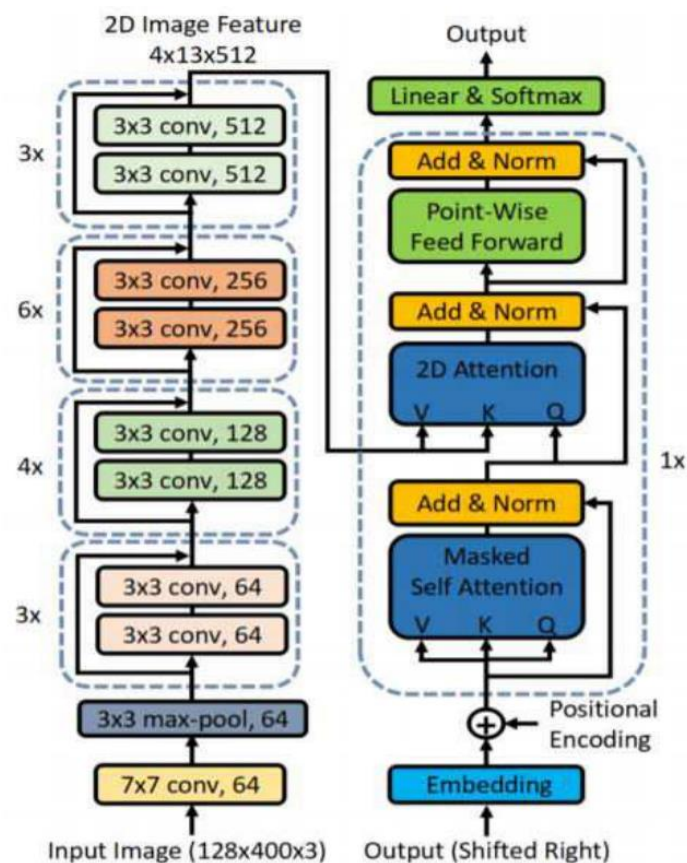


Рисунок 23 – Вычислительный граф нейронной сети, использующей 2D-механизм внимания

Авторы работы [27] предлагают масштабировать входное изображение по ширине  $n$  раз и передавать все  $n$  изображений на вход сверточной сети, где все изображения используют общие веса (weight sharing). После этого извлеченные сверточной сетью признаки обрабатываются механизмом внимания. Схема данной архитектуры представлена на рисунке 24.

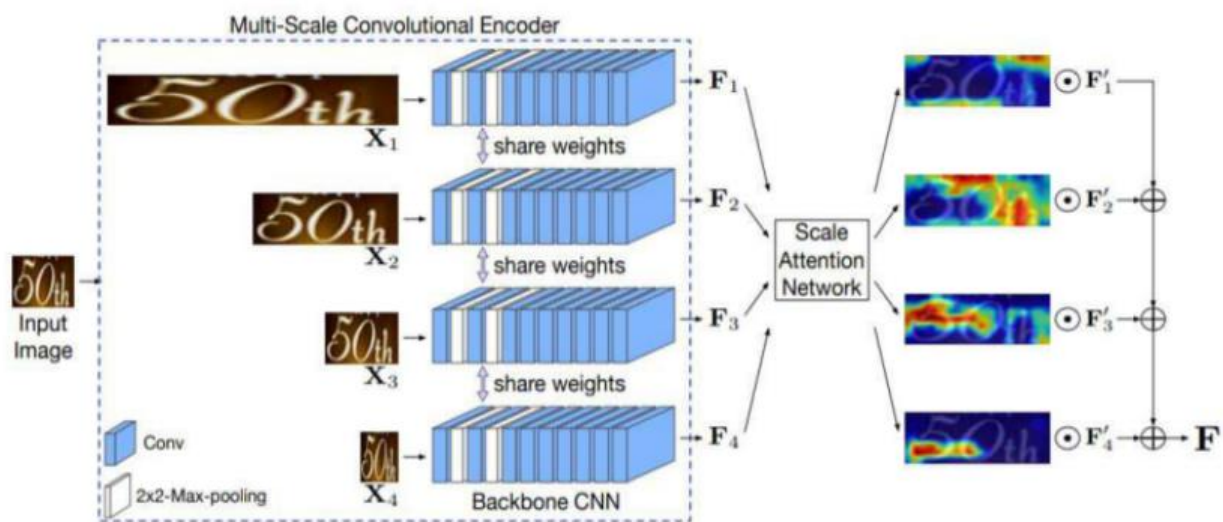


Рисунок 24 – Архитектура нейронной сети с различным масштабированием входного изображения по ширине

## 4 АКТУАЛЬНЫЕ НЕЙРОСЕТЕВЫЕ МЕТОДЫ ДЕТЕКЦИИ И РАСПОЗНАВАНИЯ ТЕКСТА

На протяжении многих лет для решения задачи распознавания текста на сложных графических сценах использовалось множество подходов. В данной главе мы сосредоточимся на современных методах, в частности тех, которые были использованы в экспериментах для создания собственной системы обнаружения и распознавания текста.

Большинство актуальных подходов к решению задачи STR основано на трансферном обучении (Transfer learning). Многие подходы появились в области классификации изображений и были обобщены до более конкретной, рассматриваемой в данной работе, задачи распознавания текста.

В машинном обучении трансферное обучение подразумевает адаптацию уже существующей модели, изначально разработанной для некоторой задачи, таким образом, чтобы она могла выполнять другую задачу при определенной тонкой настройке (fine-tuning), не начиная с нуля.

Это также является важной контекстной информацией и дает большее понимание рассматриваемых далее методов.

### 4.1 ResNet

Нейронные сети глубокого обучения состоят из большого количества последовательных слоев. Такая конструкция позволяет извлекать и анализировать большое количество признаков, тем самым повышая эффективность нейронных сетей в разы. Однако эти нейронные сети также имеют определенные проблемы, которые в некоторых случаях могут даже привести к расхождению решения. В частности, две проблемы могут возникнуть во время работы алгоритма обратного распространения ошибки (backpropagation), который отвечает за обучение параметров нейронной сети с использованием алгоритма градиентного спуска [28].

Первая проблема известна как исчезающие градиенты (vanishing gradients). Она возникает, когда значения градиента постепенно уменьшаются по мере работы алгоритма обратного распространения ошибки, что приводит к крайне малым значениям параметров в начальных слоях сети.

Вторая проблема – взрывающиеся градиенты (exploding gradients). Она возникает, напротив, когда значения градиентов постепенно увеличиваются во время работы алгоритма обратного распространения ошибки, что приводит к расхождению параметров в сторону слишком больших значений.

Архитектура остаточной нейронной сети (ResNet) изначально была задумана как метод решения проблемы исчезающих градиентов. Блок ResNet состоит из двух слоев, с весами и смещениями  $W_i$  и  $b_i$ , где  $i \in \{1,2\}$ , соответственно. Вычисляются следующие значения:

$$\begin{aligned}x' &= \text{ReLU}(W_1x + b_1); \\x'' &= \text{ReLU}(W_2x' + b_2 + x),\end{aligned}$$

где функция активации  $\text{ReLU}(x) = x^+ = \max(0, x)$ .

Рассматривая уравнения выше, мы видим, что включение входа  $x$  может предотвратить приближение выхода  $x''$  к нулю, даже когда веса  $W_i$  очень малы. На рисунке 25 представлена схема блока ResNet [29].

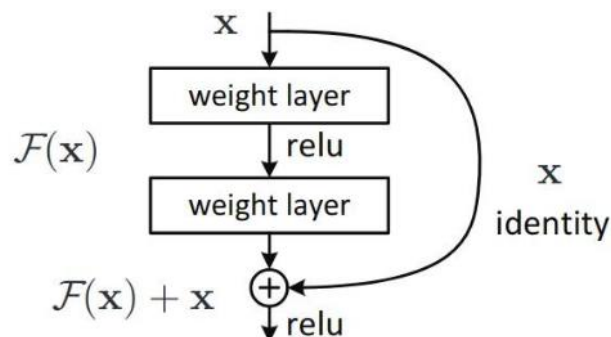


Рисунок 25 – Схема блока ResNet

Остаточные нейронные сети строятся путем последовательного складывания нескольких вышеупомянутых блоков воедино. Некоторые известные реализации остаточных сетей включают ResNet50, ResNet152 и ResNet200, где числовое значение указывает на количество блоков в нейронной сети. Впоследствии, по мере того, как ResNet показывали себя в решении разных проблем, предлагались и более сложные версии данной архитектуры с ещё большим количеством блоков.

## 4.2 MobileNet

MobileNet – интересный подход к построению легких сверточных нейронных сетей для решения задач классификации изображений. Хотя преобладающей тенденцией и является разработка всё более глубоких и сложных нейронных сетей для достижения лучшей эффективности в задачах типа ImageNet, существует также большой интерес к разработке алгоритмов, которые могут обеспечить хорошие результаты при высокой скорости работы и минимальных требованиях к аппаратному обеспечению. Этот тонкий баланс часто упускается из виду, но класс моделей MobileNet достиг впечатляющих результатов, значительно повысив как скорость, так и эффективность.

Основная концепция, лежащая в основе MobileNet, заключается в разделении традиционных слоев свертки на две независимые операции: свертку в глубину (depthwise convolution) и свертку по точкам с размерностью  $1 \times 1$  (поточечная свертка или pointwise convolution). В отличие от обычной свертки, которая фильтрует и комбинирует входную информацию за один шаг для создания нового набора выходов, свертка в глубину делит этот процесс на два этапа. На начальном этапе происходит фильтрация, а затем отдельный слой объединяет отфильтрованные результаты. В результате время вычислений и размер модели значительно сокращаются.

Рассмотрим компоненты MobileNet с точки зрения вычислительной сложности. Традиционный сверточный слой использует карту признаков (feature

map)  $F$  размера  $D_F \times D_F \times M$ , и выдает карту признаков  $G$  размера  $D_F \times D_F \times N$ , где  $D_F$  – пространственная ширина и высота квадратных входных и выходных карт признаков. Предполагается, что шаг  $s$  равен 1 и используется описанный ранее метод заполнения краёв нулевыми значениями (padding).

Таким образом, ядро свертки  $K$  имеет размер  $D_K \times D_K \times M \times N$ , где  $D_K$  – ширина и высота квадратного ядра. При этом вычислительные затраты на применение этой свертки составляют:

$$D_K \times D_K \times M \times N \times D_F \times D_F.$$

Свертка в глубину применяет один фильтр для каждого входного канала. Таким образом, для ядра  $K$  аналогично имеем размер  $D_K \times D_K \times M$ . Далее к  $m$ -му каналу  $F$  добавляется  $m$ -й фильтр  $K$ , в результате чего получается  $m$ -й канал выходной карты признаков  $G$  и вычислительные затраты равны

$$D_K \times D_K \times M \times D_F \times D_F.$$

Обратим внимание на то, что в этом выражении нет переменной  $N$ , как в предыдущем, из-за чего вычислений становится значительно меньше. Однако вместе с этим исчезает и возможность создавать новые признаки – теперь у нас лишь фильтруются признаки для  $F$ . Эту проблему решает поточечная свертка.

Поточечная свертка – это, по сути, обычная свертка с размером ядра  $1 \times 1$ . Следовательно, вычислительные затраты такой свертки составляют  $M \times N \times D_F \times D_F$ . Используя такие “единичные” свертки,  $M$  каналов каждого пикселя во входной карте признаков преобразуются в новый набор из  $N$  каналов.

Как итог, общая стоимость свертки в глубину с последующей поточечной сверткой может быть рассчитана как

$$D_K \times D_K \times M \times D_F \times D_F + M \times N \times D_F \times D_F,$$

что дает нам улучшение в плане вычислительной стоимости по сравнению с традиционными сверточными слоями [6]:

$$\frac{D_K \times D_K \times M \times D_F \times D_F + M \times N \times D_F \times D_F}{D_K \times D_K \times M \times N \times D_F \times D_F} = \frac{1}{N} + \frac{1}{D_K^2}.$$

### 4.3 Модели детекции текста

В последующих разделах представлено несколько известных алгоритмов детекции и распознавания текста на сложных графических сценах. Хотя некоторые алгоритмы и продемонстрировали более хорошую точность на эталонном датасете ICDAR15 [30], они достигли этого в основном за счет включения в них значительного числа параметров и дополнительных процедур. Поэтому в данной работе были выбраны модели, которые концептуально проще, но достигают высокой точности.

#### 4.3.1 EAST

Появление EAST (An Efficient and Accurate Scene Text Detector) [5] ознаменовало собой значительный прорыв в области распознавания текста на изображениях. Ключевой характеристикой модели является простота, при которой, однако, достигалась state-of-the-art точность. Это привело к улучшению результатов F-меры на наборе данных ICDAR15, а также продемонстрировало значительно более высокую скорость работы.

Работа модели состоит из двух этапов. На начальном этапе используется полностью сверточная нейронная сеть, которая, в свою очередь, использует трансферное обучение от другой сверточной нейронной сети, изначально разработанной для классификации изображений. Она извлекает признаки и

применяет дополнительные свертки для создания карт геометрии (geometry map) и карт оценок (score maps).

На втором этапе происходит постобработка, в ходе которой карты и карты геометрии и оценок преобразуются в повернутые прямоугольники. Затем для объединения перекрывающихся прямоугольников применяется техника, называемая “не-максимальным подавлением” (Non-Maximum Suppression, NMS) [31]. Принцип работы EAST показан на рисунке 26.

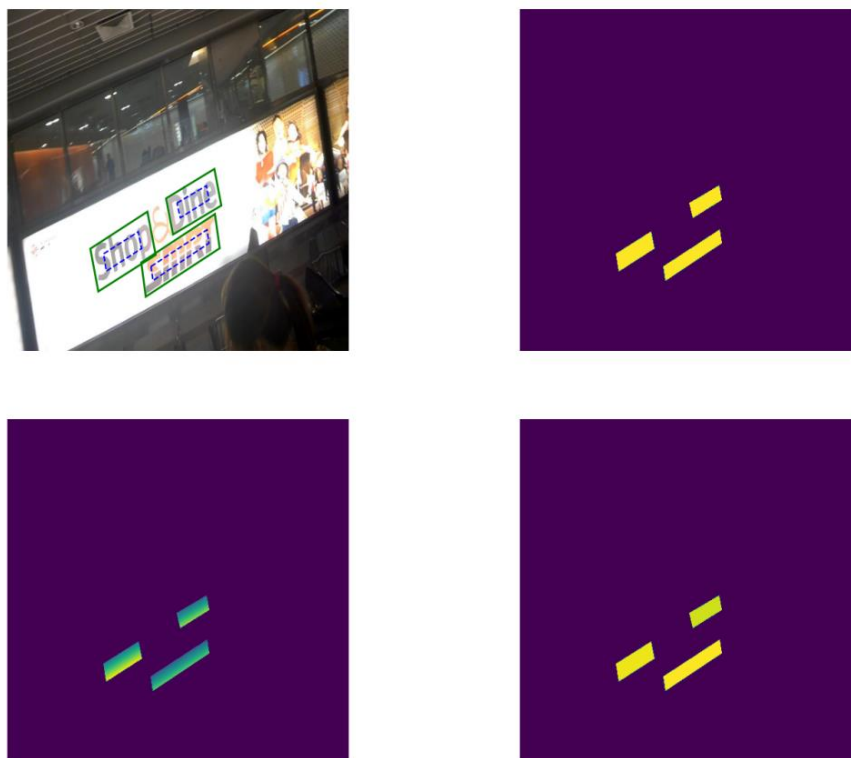


Рисунок 26 – Визуализация различных карт признаков, которые модель EAST обучается предсказывать (по часовой стрелке): исходные области и их сжатые версии, карта оценок (классификация принадлежности к выделенной области каждого пикселя), угол поворота, расстояние каждого пикселя до верхней границы области

Подход EAST представляет текстовые регионы путем создания шести карт для каждого изображения:



- 1) карта оценок. В исходном наборе данных прямоугольники (повернутые) рисуются над текстом, а затем сжимаются. Пикселям внутри сжатого полигона присваивается оценка 1, в то время как пиксели вне полигона получают оценку 0. Эту задачу можно понимать как задачу классификации по пикселям, где каждый пиксель помечается как находящийся внутри или вне прямоугольника;
- 2) карта расстояния до верхнего края. Эта карта показывает расстояние от каждого пикселя до верхней границы прямоугольника, к которому он принадлежит (если он находится внутри текстовой области);
- 3) карта расстояния до нижнего края. Подобно предыдущей карте, эта карта представляет собой расстояние от каждого пикселя до нижнего края соответствующего прямоугольника (если он находится внутри текстовой области);
- 4) карта расстояний до правого края. Означает расстояние от каждого пикселя до правого края охватывающего его прямоугольника (если он находится в пределах текстовой области);
- 5) карта расстояний до левого края. Эта карта отражает расстояние от каждого пикселя до левого края связанного с ним прямоугольника (если он находится внутри текстовой области);
- 6) карта угла поворота. Указывает угол поворота каждого пикселя в прямоугольнике.

В качестве альтернативы карту геометрии можно представить как тензор с пятью каналами. Первые четыре канала кодируют расстояния до краев прямоугольника, а пятый представляет угол поворота.

Теперь опишем архитектуру модели. Использование полной сверточной нейронной сети предполагает извлечение признаков из различных слоев нейронной сети PVANet [32]. Такой подход применяется из-за различий в характеристиках, связанных с обнаружением больших и маленьких слов. В частности, характеристики нейронной сети на поздних стадиях эффективны для

обнаружения больших слов, в то время как низкоуровневая информация с ранних стадий больше подходит для небольших областей.

Таким образом, мы извлекаем выходы четырех различных слоев сети, обозначенных как  $f_i$ , где  $i = 1, 2, 3, 4$ . Затем вычисляются слои:

$$g_i = \begin{cases} \text{unpool}(h_i), & \text{если } i \leq 3 \\ \text{conv}_{3 \times 3}(h_i), & \text{если } i = 4 \end{cases}$$

$$h_i = \begin{cases} f_i, & \text{если } i = 1 \\ \text{conv}_{3 \times 3}(\text{conv}_{1 \times 1}([g_{i-1}; f_i])), & \text{если } i \neq 1 \end{cases}$$

где  $\text{unpool}$  – билинейный апсемплинг (upsampling), удваивающая ширину и высоту изображения, а «;» – конкатенация каналов.

На рисунке 27 представлено визуальное представление описанного процесса. Начиная с извлечения признаков  $f_i$  из базовой сети, мы начинаем с апсемплинга признака, а затем объединяем его с другим признаком и применяем две сверточные операции. Первая свертка – это поточечная свертка, направленная на сокращение числа каналов. После сокращения числа каналов применяется свертка  $3 \times 3$ . Эта последовательность операций повторяется для последующих характеристик основной нейронной сети.

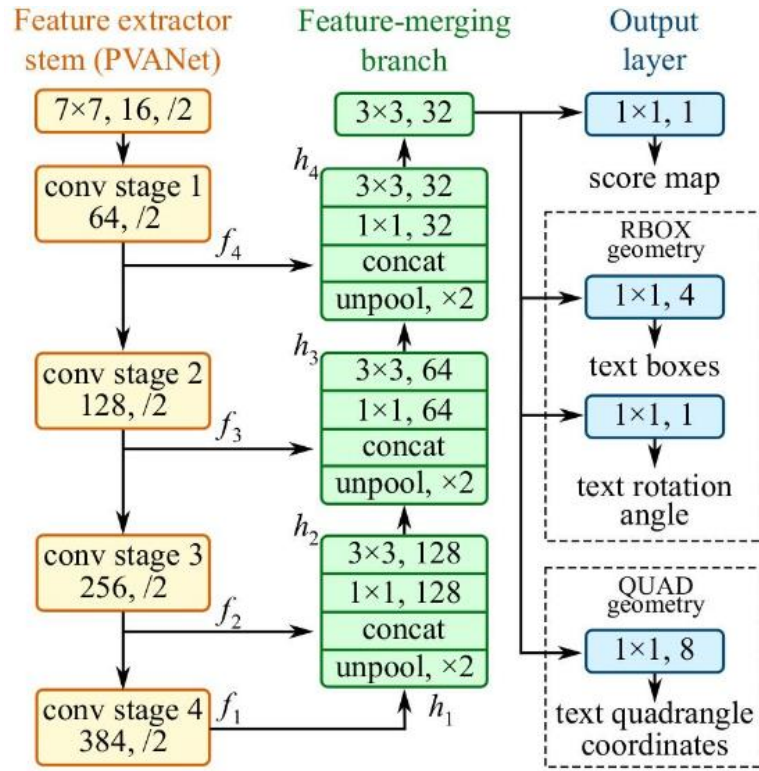


Рисунок 27 – Схема архитектуры EAST

Наконец, последний признак  $g_4$  позволяет извлечь карту оценок, карту геометрии и карту углов. Для этого на  $g_4$  выполняются три последовательные свертки с соответствующими выходными каналами, установленными на 1, 4 и 1, соответственно. В этих последних свертках используется сигмоидная функция активации. Таким образом, размеры результирующих карт составляют  $(h/4, w/4, 1)$ ,  $(h/4, w/4, 4)$  и  $(h/4, w/4, 1)$  соответственно, где  $h$  и  $w$  представляют собой высоту и ширину карт.

Чтобы облегчить обучение сверточных ядер, определяется функция потерь, вычисляемая в два этапа:

$$L = L_s + \lambda_g L_g,$$

где  $L_s$  – потери карты оценок (score loss),  $L_g$  – потери карты геометрии,  $\lambda_g$  – коэффициент балансировки, который в проведенных в настоящей работе экспериментах был установлен на  $\lambda_g = 1$ .

Score loss задается сбалансированной по классам кросс-энтропией [33]:

$$L_s = -\beta Y^* \log \hat{Y} - (1 - \beta)(1 - Y^*) \log(1 - \hat{Y}),$$

где  $\hat{Y}$  – предсказание карты оценок,  $Y^*$  – целевая переменная,  $\beta$  – фактор баланса между положительными и отрицательными объектами [34]:

$$\beta = 1 - \frac{\sum_{y^* \in Y^*} y^*}{Y^*}.$$

Геометрические потери (geometry loss) определяются на основе отношения пересечения к объединению (Intersection over Union, IoU) и косинуса предсказанного и истинного углов. Метрика IoU в русскоязычной литературе более известна как коэффициент Жаккара (Jaccard index).

При сравнении предсказанного прямоугольника  $\hat{R}$  и истинного прямоугольника  $R^*$ , окружающего пиксель, более высокое отношение пересечения к объединению указывает на большее сходство между двумя прямоугольниками. Аналогично, для предсказанного угла  $\hat{\theta}$  и истинного угла  $\theta^*$  более близкое значение косинуса  $\hat{\theta} - \theta^*$  к 1 указывает на большее сходство между углами.

Коэффициент IoU варьируется от 0 до 1, где значение 0 означает пустое пересечение между прямоугольниками, а значение 1 означает, что пересечение равно объединению, то есть прямоугольники идентичны.

Чтобы включить эту информацию в функцию потерь, мы можем взять отрицательный логарифм отношения IoU. Это преобразование приводит к тому, что значение потерь равно 0, когда отношение IoU равно 1, и приближается к бесконечности, когда отношение IoU приближается к 0 (см. рисунок 28) [35]. Следовательно, геометрические потери можно преобразовать

$$L_{AABB} = -\log IoU(\hat{R}, R^*) = -\log \frac{|\hat{R} \cap R^*|}{|\hat{R} \cup R^*|}$$

и, приняв во внимание, что  $L_\theta(\hat{\theta}, \theta^*) = 1 - \cos(\hat{\theta}, \theta^*)$ , получить суммарные геометрические потери:

$$L_g = L_{AABB} + 10 \cdot L_\theta.$$

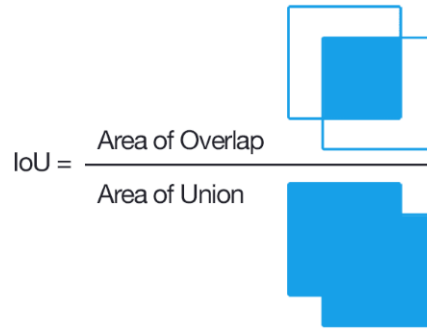


Рисунок 28 – Графическое представление Intersection over Union

Благодаря минимизации общей потери  $L = L_s + \lambda_g L_g$ , нейронная сеть проходит процесс обучения, направленный на создание карт геометрии и оценок, которые приближаются к истинным значениям. Эта оптимизация достигается с помощью алгоритма градиентного спуска или его модификаций, таких как Adam [36]. Путем итерационного обновления параметров нейронной сети, на основе этих алгоритмов оптимизации модель становится способной генерировать карты, которые эффективно идентифицируют и очерчивают текстовые регионы.

Как было замечено ранее, сверточная нейронная сеть генерирует набор из трех тензоров или карт. Однако конкретное количество текстовых полей, а также их положение и ориентацию еще необходимо определить.

Чтобы получить желаемую информацию о повернутых прямоугольниках, требуется дополнительный шаг, в котором будут найдены координаты  $(x, y)$  всех четырех углов прямоугольника.

В первую очередь, для каждого пикселя соответствующее значение в карте оценок сравнивается с пороговым значением, обозначаемым как  $\alpha$ . Если значение  $y^*$  превышает этот порог, окружающая область может быть выведена из карты геометрии с учетом угла и расстояний до верхнего, нижнего, левого и правого краев. Однако, такой подход приводит к большому количеству пересекающихся прямоугольников, что приводит к некой захламленности результатов, содержащих избыточные данные.

Для решения проблемы используется техника NMS, которая позволяет отбросить пересекающиеся прямоугольники и выбрать тот, который имеет благоприятное соотношение пересечения и объединения (IoU) по сравнению с другими прямоугольниками, окружающими конкретный экземпляр текста, и при этом имеет высокую оценку.

Более того, существует модифицированная версия NMS под названием *locality-aware NMS* [5], которая способна работать за линейное время. Стоит отметить, что эти алгоритмы используют жадную стратегию, которая не всегда дает оптимальный результат. Приоритет отдается скорости и удовлетворительным результатам.

#### 4.3.2 CRAFT

CRAFT (Character Region Awareness For Text detection) [37] – модель, также использующая в своей основе сверточную нейронную сеть, которая создает карты. Теперь их две: оценка региона (region score) и оценка сходства (affinity score).

Карта оценки региона используется для оценивания центральных областей отдельных символов в тексте, а карта оценки сходства оценивает среднюю точку между двумя последовательными символами. В результате данный подход кардинально отличается от метода EAST, поскольку он сосредоточен на детекции текста на уровне символов.

После обнаружения символов и нахождения их сходства с помощью двух вышеописанных карт, можно восстановить слова из символов. На рисунке 29 показана визуализация данного процесса.



Рисунок 29 – Оценки регионов и выведенные с помощью модели CRAFT  
регионы текста

Коротко опишем архитектуру модели CRAFT, схема которой изображена на рисунке 30, а также её преимущества и недостатки.

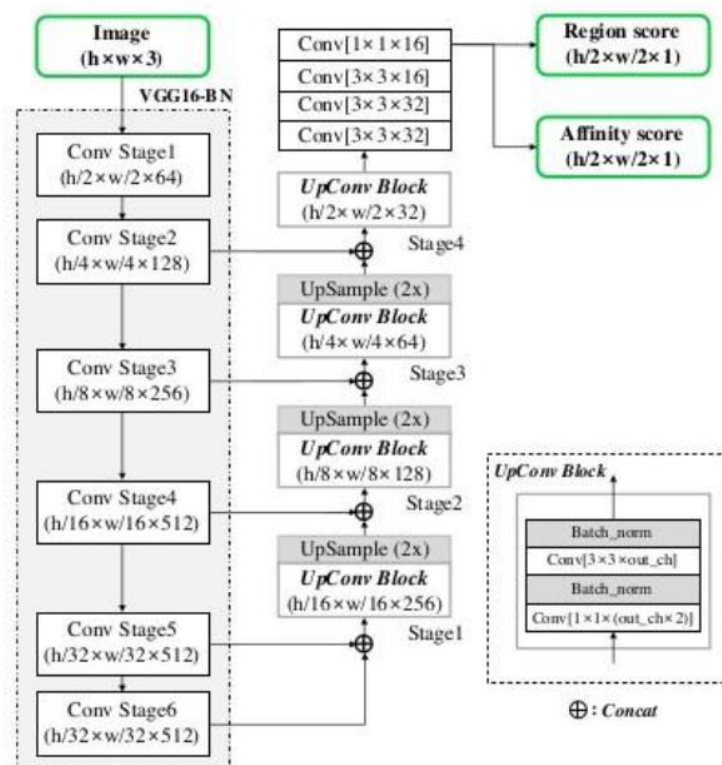


Рисунок 30 – Схема архитектуры CRAFT

Базовая архитектура CRAFT имеет некоторое сходство с EAST [5]. Она использует нейронную сеть VGG-16, обученную на наборе данных ImageNet [38], для классификации изображений, в частности, для извлечения общих признаков. Эти признаки извлекаются из различных слоев сети, причем каждый слой имеет размеры, вдвое меньшие, чем предыдущий слой. Выходы этих слоев впоследствии конкатенируются и апсемплируются, следуя подходу, аналогичному EAST.

После этих операций к полученным характеристикам применяется последовательность сверток. Этот процесс в конечном итоге дает карты region score и affinity score, или, что эквивалентно, тензор с двумя каналами, высотой  $h/2$  и шириной  $w/2$ , где  $h$  и  $w$  – соответственно исходные высота и ширина входного изображения.

Подход на уровне символов в CRAFT дает определенные преимущества (простота и способность распознавать сложные по форме тексты), но также представляет некоторые проблемы, наиболее серьезной из которых является



необходимость в аннотациях на уровне символов при обучении. Хотя модель CRAFT и превосходит EAST по метрике F-score (см. таблицу 1), она уступает ей в скорости работы, требуя заметно большее количество памяти и вычислительной мощности.

Таблица 1 – Сравнение моделей CRAFT и EAST на различных метриках

| Метод | Precision | Recall | F-score |
|-------|-----------|--------|---------|
| EAST  | 0.847     | 0.773  | 0.808   |
| CRAFT | 0.897     | 0.842  | 0.869   |

Из этого следует, что, с нашей точки зрения, CRAFT менее привлекателен по сравнению с EAST, поскольку основное внимание в данной работе уделяется использованию облегченных моделей.

#### 4.4 Модели распознавания текста

В прошлом алгоритмы распознавания текста опирались на созданные вручную признаки, такие как дескрипторы гистограммы ориентированных градиентов (histogram of oriented gradients descriptors), связанные компоненты (connected components) и преобразование ширины штриха (stroke width transform) [3]. Однако с появлением глубоких нейронных сетей, в частности, сверточных рекуррентных нейронных сетей (CRNN), эти алгоритмы были превзойдены. Алгоритмы глубокого обучения автоматически изучают параметры признаков, устраняя необходимость в “ручных” признаках. Кроме того, эти модели демонстрируют лучшую обобщенность на другие задачи.

Далее будет подробно рассмотрена модель CRNN, которая представляет из себя особый тип сверточной нейронной сети, объединяющей сильные стороны сверточных и рекуррентных слоев.

#### 4.4.1 CRNN

Для решения задачи классификации локализованного текста на сложной графической сцене авторами CRNN предлагается использовать комбинацию сверточной нейронной сети (CNN), рекуррентной нейронной сети (RNN) и специализированной функции ошибки Connectionist Temporal Classification (CTC-loss) [39]. Подход с применением CTC произвел революцию в обучении рекуррентных нейронных сетей при работе с несегментированными последовательностями. В контексте нашей конкретной задачи распознавания текста, когда нам представлено изображение, у нас нет знаний о сегментации символов в слове. Метод CTC предлагает решение, позволяя предсказывать слова без необходимости явной сегментации входного изображения. Перед описанием этого метода следует рассмотреть принцип работы CRNN.

Для начала входные изображения проходят через сверточные слои, которые извлекают из них карты признаков. На рисунке 31 изображено, как входное изображение подается на вход слоям сверточной нейронной сети для последующего извлечения признаков. [7]

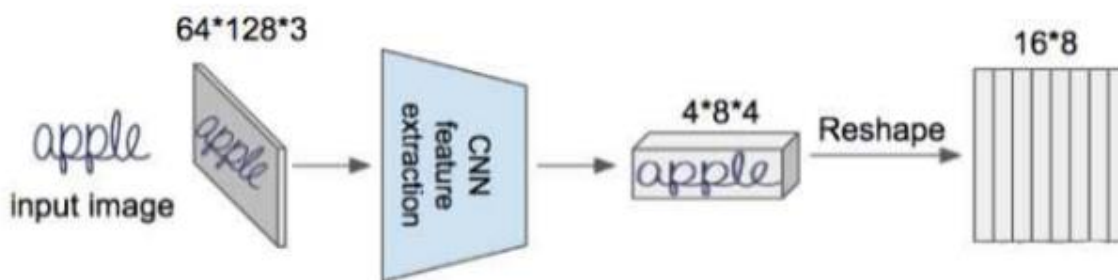


Рисунок 31 – Схема процесса извлечения признаков при помощи CRNN

Затем эти карты признаков преобразуются в последовательность признаков. Эту последовательность можно представить как столбцы или группы из  $k$  последовательных столбцов карты признаков. Такое расположение называется рецептивным полем (см. рисунок 32).

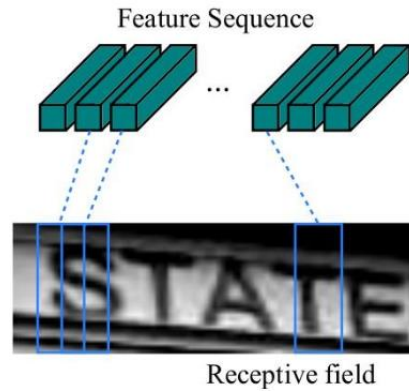


Рисунок 32 – Иллюстрация рецептивных полей над изображением (в реальном алгоритме это делается не на самом изображении, а на столбцах карты признаков, выводимой сверточными слоями)

На выходе получается трехмерный массив, который необходимо преобразовать к двумерному, размерность которого должна быть равна максимальному количеству символов во входном изображении. Затем каждый столбец полученной матрицы передается соответствующему LSTM-блоку для обработки извлеченных признаков, как показано на рисунке 33.

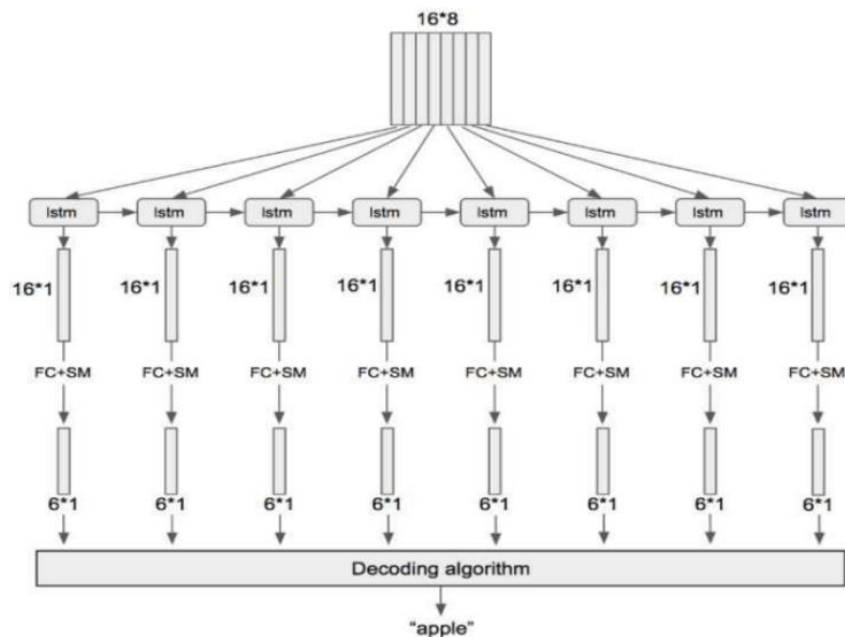


Рисунок 33 – Схема процесса обработки извлеченных признаков рекуррентным слоем

Использование рекуррентного слоя необходимо для представления входных признаков в виде последовательности, чтобы распознавание символов происходило в том порядке, в котором они представлены на исходном изображении.

Вспомним, что блок LSTM состоит из ячейки памяти и трех мультипликативных гейтов: входного, выходного и гейта забывания. Блок обладает способностью сохранять ранее встречаемый контекст, а гейты регулируют продолжительность его хранения в памяти. Эта особенность позволяет архитектурам, использующим LSTM, улавливать временные зависимости и делает их хорошо подходящими для работы с последовательностями, полученными, в частности, из изображений.

Кроме того, в контексте распознавания символов и изучения определенного рецептивного поля, нам интересна не только информация, предшествующая этому рецептивному полю, но также и последующий контекст. В результате появляется концепция двунаправленного (bi-directional) слоя LSTM, позволяющая учитывать движение как вперед (forward), так и назад (backward).

Итак, после обработки рекуррентным слоем каждый вектор передается полносвязному слою и функции активации softmax. Размерность полносвязного слоя определяется длиной массива всех возможных распознаваемых символов, включая добавление blank-маркера.

Blank-маркер используется алгоритмом CTC-loss для обозначения отсутствия символа в тексте. Он добавляется, так как большинство последовательностей символов, подаваемых на вход, меньше максимально возможной длины. Функция softmax преобразует входной вектор в вектор распределения вероятностей для всех возможных символов, включая маркер отсутствия символа (blank), как показано в [22]. Рисунок 34 демонстрирует пример работы декодирующего алгоритма.

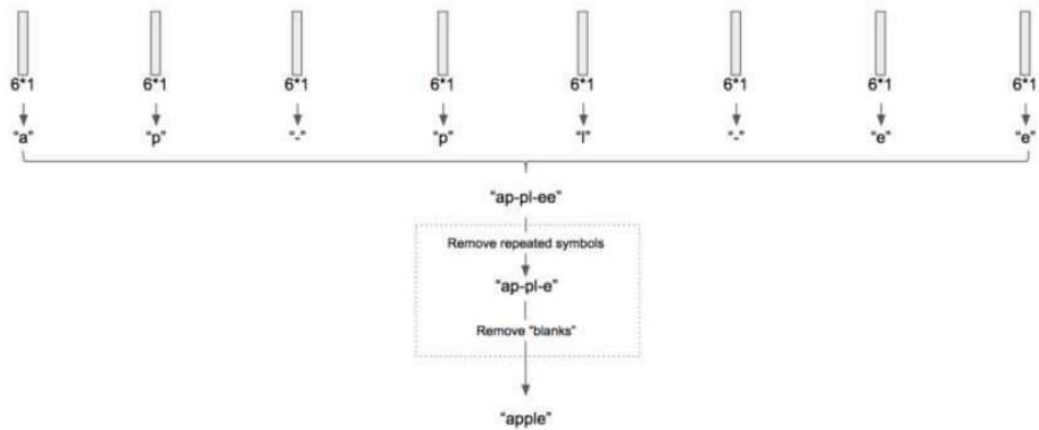


Рисунок 34 – Схема работы декодирующего алгоритма на конкретном примере

Рисунок 35, в свою очередь, демонстрирует схему самого процесса декодирования.

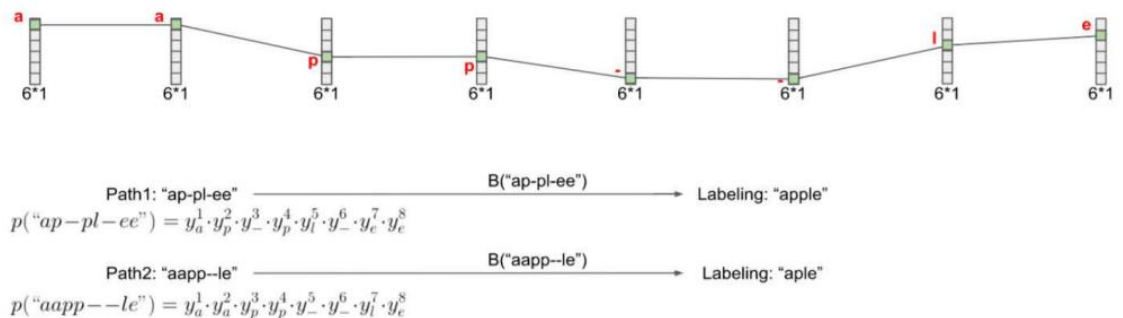


Рисунок 35 – Процесс работы декодирующего алгоритма

Результатом является матрица, в которой каждый столбец содержит вероятности, распределенные функцией активации softmax. Число столбцов соответствует максимально возможной длине слова, а число строк – длине "алфавита"  $L$ , увеличенному на 1 для учета blank-маркера. Данная матрица передается в декодирующий алгоритм, который в первую очередь удаляет все повторяющиеся символы и затем blank-маркеры.

Получаем итоговый алфавит  $L'$  – множество всех возможных символов, которые мы стремимся распознать, дополнительно включающее также blank-

маркер. И таким образом, вероятность пути (последовательности символов)  $Y$  – это произведение вероятностей всех активированных ячеек каждого из столбцов матрицы:

$$p(Y) = \prod_{t=1}^T y_{Y_t}^t, \forall Y \in L'.$$

Для каждого слова существует несколько возможных путей, и для расчета вероятности этого слова необходимо просуммировать вероятности по всем этим путям [39].

Функция потерь напоминает бинарную кросс-энтропию, но вместо вероятности класса используется вероятность слова:

$$CTC = -\ln(p(\text{"apple"})).$$

Для сценария, рассмотренного в данном примере (где алфавит состоит из 6 символов, а максимальное число символов равно 8), имеется  $6^8=1679616$  возможных путей. Однако, в реальной жизни длина алфавита и максимальное число символов зачастую значительно превышают эти значения. В связи с этим, для нахождения вероятности слова гораздо более эффективным методом является использование динамического программирования.

В этом случае создается новая матрица, которая подобна матрице, полученной в результате применения softmax-слоя (см. рисунок 36).

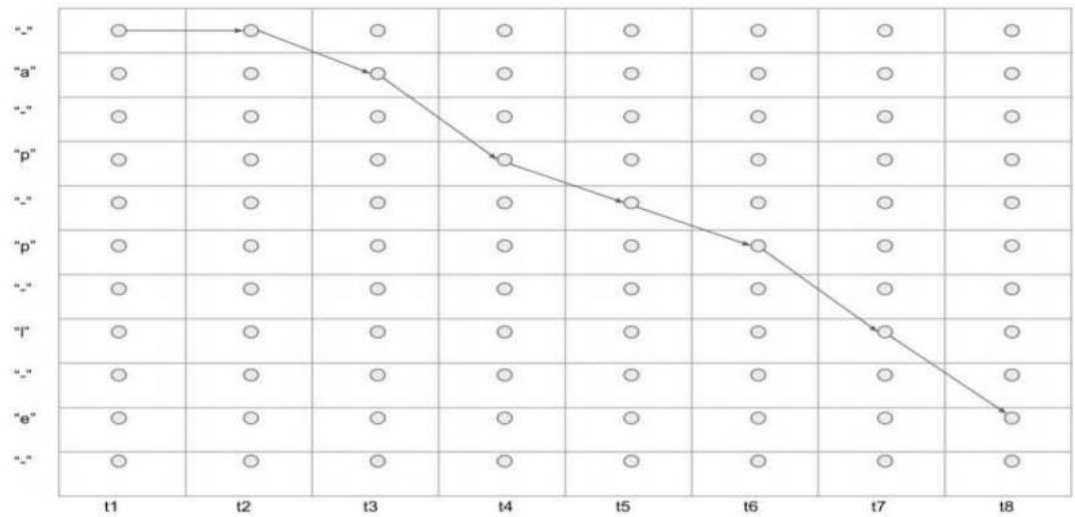


Рисунок 36 – Визуализация матрицы для построения множества всех возможных путей

Используя данную матрицу, можно построить множество всех возможных путей для заданного слова и вычислить коэффициент  $\alpha$  (см. рисунок 37). Расчет коэффициента  $\alpha$  выполняется по формуле:

$$\alpha_t(s) = y_{\text{seq}(s)}^t (\alpha_{t-1}(s) + \alpha_{t-1}(s-1)).$$

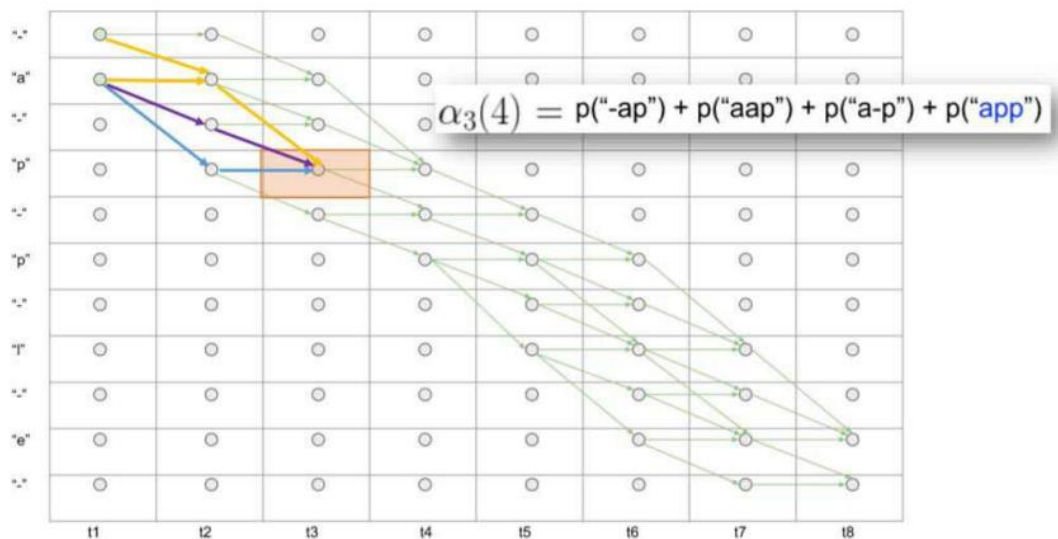


Рисунок 37 – Визуализация возможных путей

Для определения вероятности каждого элемента в матрице, находящегося на пересечении s-й строки и t-го столбца, вычисляется сумма всех возможных путей, которые ведут к этому элементу (см. рисунок 38). Затем результат умножается на вероятность данного элемента. Эта процедура повторяется для каждого столбца, включая последний, значение которого суммируется и передается в функцию ошибки. Таким образом, данный метод позволяет эффективно определить вероятность каждого слова, учитывая все возможные пути.

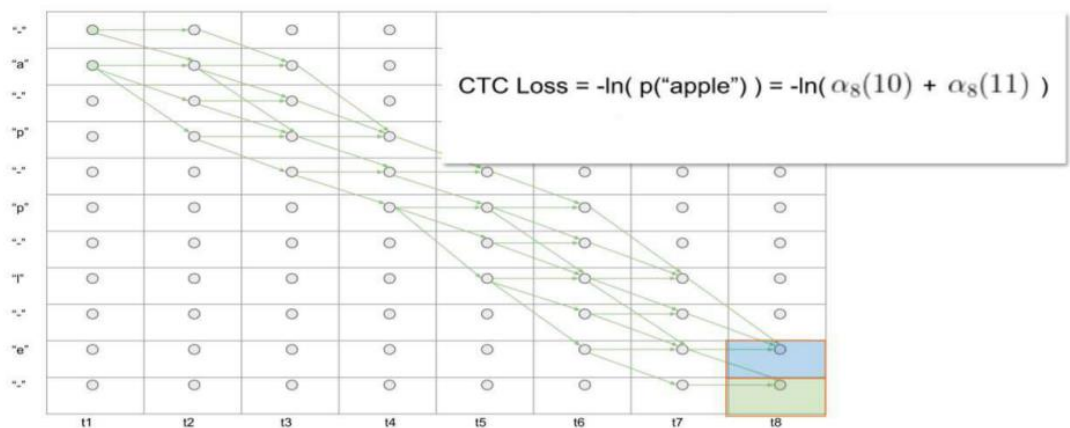


Рисунок 38 – Расчет суммы путей

Коэффициент  $\alpha$  вводится для прямого распространения ошибки, для обратного же используется коэффициент  $\beta$ , определяемый формулой:

$$\beta_t(s) = y_{seq(s)}^t (\beta_{t+1}(s) + \beta_{t+1}(s + 1)).$$

Коэффициент  $\beta$  рассчитывается для всех путей, выходящих из каждого элемента матрицы. Поделим произведение  $\alpha_s^t$  и  $\beta_s^t$  на вероятность элемента  $y_s^t$  и просуммируем вдоль t-го столбца матрицы. Получим формулу:

$$p(\text{"apple"}) = \sum_{s=1}^{seq(s)} \frac{\alpha_s^t \beta_s^t}{y_s^t}.$$



Чтобы нейронную сеть можно было обучить при помощи алгоритма градиентного спуска, необходимо продифференцировать функцию ошибки по всем выходам  $y_s^t$ :

$$\frac{d(-\ln(p(\text{apple})))}{dy_k^t} = -\frac{1}{p(\text{"apple"})} \times \frac{d(p(\text{"apple"}))}{dy_k^t}.$$

Рассматриваются только те пути, которые идут через символ  $k$  в момент времени  $t$  (см. рис 40):

$$\frac{d(p(\text{"apple"}))}{dy_k^t} = -\frac{1}{y_k^{t^2}} \sum_{seq(s)=k} \alpha_t(s) \beta_t(s).$$

Архитектура CRNN является полностью дифференцируемой системой, построенной в форме единого вычислительного графа (см. рисунок 39). Для улучшения точности распознавания, распознанное слово часто проверяется по заранее заданному словарю. В одном из исследований в данной работе для этого вычисляется расстояние Левенштейна между распознанным словом и каждым словом из словаря, после чего выбирается пара слов с наименьшим расстоянием.

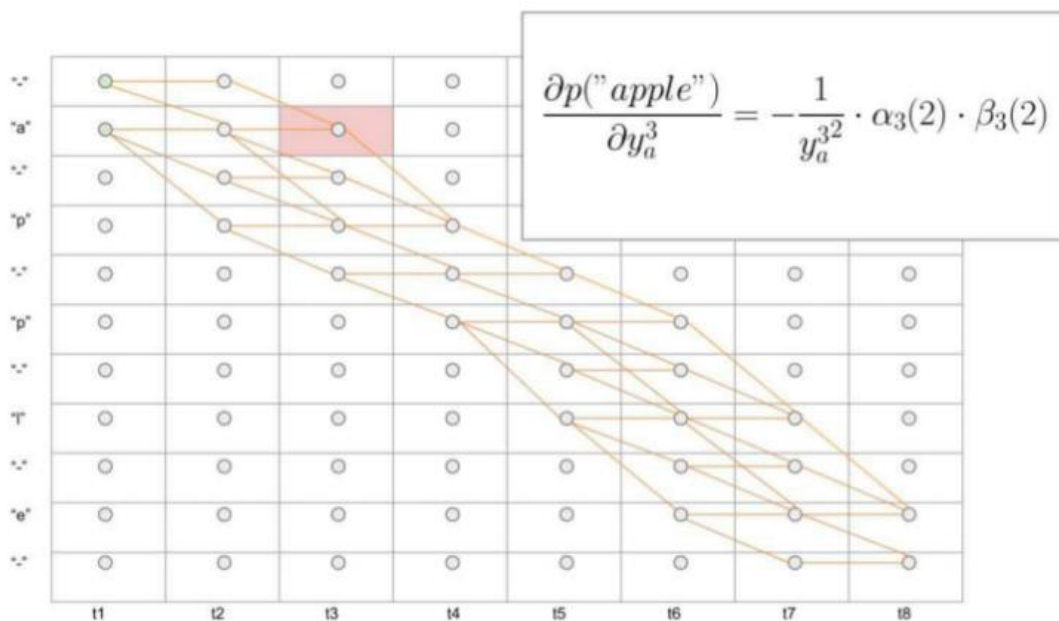


Рисунок 39 – Визуализация дифференцирования по символу на примере символа «а» в момент времени  $t = 3$

## 4.5 Сквозные модели

Хотя большинство алгоритмов распознавания текста обычно включают в себя двухэтапный процесс, состоящий из алгоритма детекции и алгоритма распознавания, существуют альтернативные подходы, направленные на выполнение всего процесса в рамках одной нейронной сети, используя сквозную (end-to-end) структуру. Одним из заметных достижений в этом направлении является модель FOTS.

### 4.5.1 FOTS

Архитектура FOTS (Fast Oriented Text Spotting with a Unified Network) – сквозная нейронная сеть для обнаружения и распознавания текста, в которой используется общий набор сверток при генерации признаков для компонентов детекции и распознавания.

Следуя методологии, описанной в статье, компонент детекции выделяет регион с текстом в прямоугольник с нужной областью (bounding box), который

затем передается в промежуточный слой под названием RoI Rotate. Этот слой переводит признаки на области детекции в прямоугольные карты, которые, в свою очередь, передаются в модель CRNN для распознавания, использующую CTC-потери в сочетании с потерями EAST для оптимизации поиска. На рисунке 40 представлено визуальное представление этого процесса.

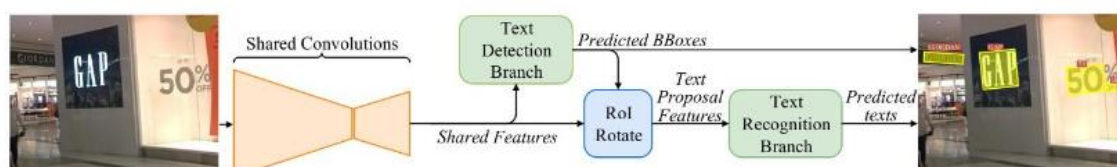


Рисунок 40 – Архитектура FOTS

Этот сквозной подход имеет более высокую скорость выполнения благодаря устранению накладных расходов за выполнение общих сверток. Без общих сверток пришлось бы применять гораздо большее их количество к интересующим областям. Таким образом, FOTS достигла поразительной скорости, став при этом state-of-the-art моделью на момент выхода работы, в которой была предложена.

Подобно алгоритму EAST, FOTS использует широко распространенный алгоритм классификации изображений ResNet50. Он извлекает как высокоуровневые, так и низкоуровневые признаки из карт признаков этой архитектуры. Затем эти карты признаков подвергаются последовательному апсемплингу, и к ним применяются свертки в соответствии с подходом, описанным в разделе 4.3.1. Схема этого процесса визуализирована на рисунке 41.

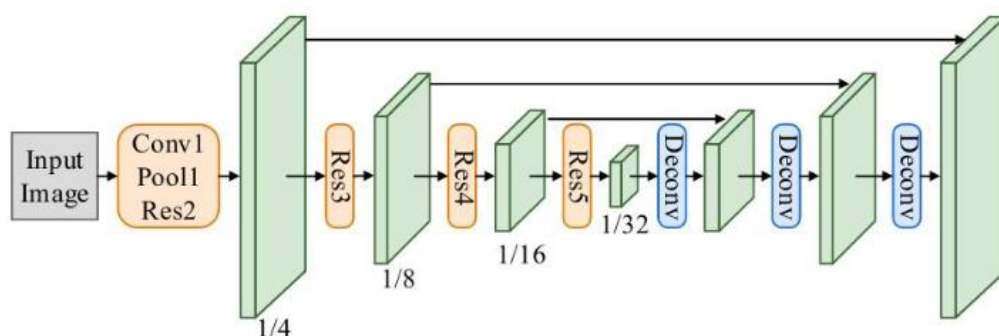


Рисунок 41 – Общие свертки FOTS (Conv1-Res5 – слои из ResNet50, а Deconv – свертка и билинейный апсемплинг)

Детекция в FOTS проходит похожим образом, что и в алгоритме EAST. В частности, к карте признаков  $g_4$  применяются три свертки для вычисления score map, geometry map и angle map. Затем эти карты используются для предсказания ограничительных рамок (bounding boxes) и проходят через NMS. Это было подробно рассмотрено в разделе 4.3.1. В этом разделе, для удобства, будем обозначать полученную ранее функцию потерь как  $L_{detect}$ .

RoIRotate (Region of Interest Rotate) использует аффинные преобразования для перехода от  $B$  текстовых полей к множеству  $B$  карт признаков, выровненных по осям. Затем эти карты признаков могут быть переданы в CRNN для распознавания. Это означает, что CRNN начнет свою обработку не с исходного изображения, а с признаков глубокого уровня.

Метод состоит из двух основных этапов. Сначала вычисляется аффинное преобразование для выравнивания повернутого прямоугольника в горизонтальный прямоугольник с центром в начале координат. Затем эти преобразования применяются к общим признакам  $g_4$ .

Пусть  $(x, y)$  – координаты точки на картах признаков, а  $(t, b, l, r)$  – расстояния от текста до верхней, нижней, левой и правой сторон текста соответственно. Пусть  $\theta$  представляет собой ориентацию выхода из компонента детекции. Определяем высоту и ширину прямоугольника как  $h_t$  и  $w_t$ , соответственно, а также матрицу, которая принимает единый прямоугольник с центром в начале координат  $(0, 0)$  и возвращает повернутый прямоугольник:

$$t_x = l \cos \theta - t \sin \theta - x;$$

$$t_y = t \cos \theta - l \sin \theta - y;$$

$$s = \frac{h_t}{t + b};$$

$$w_t = s (l + r);$$

$$\begin{aligned} M &= \begin{pmatrix} \cos \theta & -\sin \theta & 0 \\ \sin \theta & \cos \theta & 0 \\ 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} s & 0 & 0 \\ 0 & s & 0 \\ 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} 1 & 0 & t_x \\ 0 & 1 & t_y \\ 0 & 0 & 1 \end{pmatrix} = \\ &= s \begin{pmatrix} \cos \theta & -\sin \theta & t_x \cos \theta - t_y \sin \theta \\ \sin \theta & \cos \theta & t_x \sin \theta + t_y \cos \theta \\ 0 & 0 & \frac{1}{s} \end{pmatrix}. \end{aligned}$$

Тогда для обратного хода мы можем взять RoI с

$$\begin{pmatrix} x_i^s \\ y_i^s \\ 1 \end{pmatrix} = M^{-1} \begin{pmatrix} x_i^t \\ y_i^t \\ 1 \end{pmatrix}$$

для всех  $i \in [1, \dots, h_t]$ ,  $j \in [1, \dots, w_t]$ , и всех каналов.

Рисунок 42 наглядно демонстрирует этот процесс. Важно отметить, что в реальном алгоритме вращение применяется к картам признаков, расположенным на более глубоких слоях сети, а не непосредственно к самому изображению.



Рисунок 42 – Пример, иллюстрирующий принцип работы RoIRotate непосредственно на изображении вместо карт признаков

Компонент распознавания производит свертки, передает результат в LSTM-блок, после чего вычисляются вероятности определенных символов с помощью полносвязного слоя. Для получения  $h(x)$  используется декодер CTC. Когда алгоритм применяется к  $N$  областям слов, получается следующая формула потерь при распознавании:

$$L_{recog} = -\frac{1}{N} \sum_{n=1}^N \log(p(y_n^*|x)).$$

Обратим внимание, что если истина предсказана с вероятностью 1, то потери равны нулю.

После этого потери объединяются для получения общих потерь сквозной модели:

$$L = L_{detect} + L_{recog}.$$

## 5 ИССЛЕДОВАНИЕ МОДЕЛИ CRNN

Целью данного эксперимента является оценка эффективности архитектуры CRNN+CTC-loss при распознавании текста на сложных графических сценах. Это исследование является не самостоятельным, а скорее вспомогательным, поскольку в ходе работы дало представление о представленной архитектуре для использования в дальнейшем.

### 5.1 Описание эксперимента

Разработанная нейронная сеть состоит из набора слоев, в соответствующей последовательности представленных в таблице 2. Результат передаётся алгоритму CTC-loss.

Таблица 2 – Слои исследуемой архитектуры нейронной сети CRNN

| №  | Тип слоя           | Ф-я активации | Ядро | Шаг | Кол-во каналов |
|----|--------------------|---------------|------|-----|----------------|
| 1  | Conv2D             | ReLU          | 3×3  | -   | 64             |
| 2  | MaxPooling2D       | -             | 2×2  | 2×2 | 64             |
| 3  | Conv2D             | ReLU          | 3×3  | -   | 128            |
| 4  | BatchNormalization | -             | -    | -   | -              |
| 5  | MaxPooling2D       | -             | 2×2  | 2×2 | 128            |
| 6  | Conv2D             | ReLU          | 3×3  | -   | 256            |
| 7  | BatchNormalization | -             | -    | -   | -              |
| 8  | Conv2D             | ReLU          | 3×3  | -   | 256            |
| 9  | MaxPooling2D       | -             | 2×2  | 1×2 | 256            |
| 10 | Conv2D             | ReLU          | 3×3  | -   | 512            |
| 11 | BatchNormalization | -             | -    | -   | -              |
| 12 | Conv2D             | ReLU          | 3×3  | -   | 512            |
| 13 | MaxPooling2D       | -             | 2×2  | 1×2 | 512            |

|    |               |         |     |   |     |
|----|---------------|---------|-----|---|-----|
| 14 | Conv2D        | ReLU    | 3×3 | - | 512 |
| 15 | Bidirectional | -       | -   | - | -   |
| 16 | Bidirectional | -       | -   | - | -   |
| 17 | Dense         | Softmax | -   | - | -   |

В данном исследовании был выбран алгоритм оптимизации Adam. Количество эпох обучения и параметры оптимизации указаны в таблице 3.

Таблица 3 – Гиперпараметры исследуемой архитектуры CRNN

|                                          |       |
|------------------------------------------|-------|
| Размер батча                             | 64    |
| Количество эпох                          | 2     |
| Коэффициент скорости обучения $\alpha_0$ | 0.001 |

Алфавит содержит 43 символа, а максимально возможная длина слова равна 16 символам. Веса нейронной сети до обучения инициализировались нормально распределенными значениями со стандартным отклонением 0.1 и математическим ожиданием, равным 0. Для крайних пикселей в сверточных слоях было использовано автоматическое заполнение значений нулями, чтобы получить на выходе слоя ту же размерность, что и на входе.

Для проведения эксперимента была разработана программа на языке программирования Python с использованием фреймворка TensorFlow 2 и API Keras.

## 5.2 Методика оценивания

Как было упомянуто, для оценки данной модели используется метрика расстояния Левенштейна. Она измеряет сходство между двумя строками, представленными в виде последовательности символов.



Принцип работы метрики Левенштейна заключается в измерении минимального количества операций редактирования (вставки, удаления или замены символов), которые необходимо выполнить для превращения одной строки в другую.

Для определения этого расстояния используется таблица размером  $(m + 1)$  на  $(n + 1)$ , где  $m$  и  $n$  - это длины двух строк. Значения в ячейках этой таблицы соответствуют расстоянию редактирования между подстроками первоначальной строки и редактируемой строки. Нулевое значение означает, что строки полностью идентичны, в то время как большие значения указывают на большое количество операций редактирования, которые необходимы для преобразования первоначальной строки в редактируемую.

Программно метрика вычисляется при помощи динамического программирования путем заполнения таблицы для каждой частной подстроки первоначальной строки с очередной частной подстрокой потенциально более короткой редактируемой строки. Каждая ячейка заполняется значением, которое представляет определенный тип операции (вставка, удаление или замена), которую необходимо выполнить для получения соответствующего элемента редактируемой строки.

После того, как таблица рассчитана (включая последний элемент), расстояние Левенштейна между двумя строками определяется значением, находящимся в последней ячейке. Это значение представляет собой минимальное количество операций редактирования, необходимых для преобразования первоначальной строки в редактируемую.

Визуализация процесса построения таблицы и вычисления расстояния Левенштейна были показаны на рисунках 36-39.

### 5.3 Описание данных

Модель будем обучать на данных из набора Synth 90k [41], а тестировать – на наборах данных IIT 5k и SVT.

Набор данных Synth 90k состоит из изображений в оттенках серого, имеющих различную ширину, но фиксированную высоту (31 пиксель). На каждом изображении представлено одно случайное слово английского языка. Тренировочная выборка содержит более, чем 7 миллионов изображений.

Набор данных ИИТ 5k состоит из 5000 фотографий различных витрин магазинов, рекламных объявлений и уличных вывесок, на которых текст был заранее локализован. Набор SVT содержит приблизительно 1000 изображений и составлен аналогичным образом, однако также содержит, помимо локализованных областей, нелокализованные сцены.

Программа приводит каждое входное изображение к размеру  $31 \times 100$  и подает их на входной слой нейронной сети вместе с истинным текстом, представленном на соответствующем изображении.

## 5.4 Результаты эксперимента

В таблице 4 представлены показатели метрики Левенштейна для тестов на двух наборах данных с использованием большого словаря (1000 слов), малого словаря (100 слов) и без использования словаря.

Таблица 4 – Результаты тестирования модели CRNN+CTC-loss на метрике расстояния Левенштейна

| Метод              | ИИТ 5k | SVT  |
|--------------------|--------|------|
| С большим словарём | 92.7   | -    |
| С малым словарём   | 96.2   | 95.4 |
| Без словаря        | 73.9   | 75.1 |

## 6 ОБЪЕДИНЕНИЕ МОДЕЛЕЙ ДЕТЕКЦИИ И РАСПОЗНАВАНИЯ

Данный эксперимент заключался в объединении алгоритма детекции текста на сложной графической сцене с алгоритмом распознавания текста для создания удобного подключаемого модуля Python. Этот модуль был разработан для получения изображений на вход и предоставления не только положения обнаруженной области с текстом, но и самого предсказанного текста. Это послужило эталоном для последующих, более продвинутых начинаний. В отличие от последующих разработок, этот первоначальный эксперимент не предполагал обучения нейронных сетей.

### 6.1 EAST

Для детекции была использована реализация EAST, которая в итоге была изменена следующим образом: [42]

- 1) вместо использования PVANet была выбрана архитектура ResNet50 в качестве основы для извлечения признаков;
- 2) сбалансированная кросс-энтропия была заменена на Dice loss [43]. Это изменение было направлено на улучшение показателя IoU;
- 3) вместо поэтапного снижения скорости обучения было использовано линейное снижение скорости обучения.

Важно отметить, что были использованы предварительно обученные веса, предоставленные авторами.

Вводя изображения в алгоритм, мы получаем список координат в заданном формате:

$$((x1,y1),(x2,y2),(x3,y3),(x3,y3),(x4,y4))$$

Эти координаты описывают четыре угла повернутого прямоугольника, в который заключен экземпляр текста. Когда модель получает их, она извлекает горизонтальные изображения, включающие только текст. Изображения выделенных областей далее будут передаваться в компонент распознавания.

## 6.2 CRNN

Для этапа распознавания текста была использована архитектура сверточной рекуррентной нейронной сети (CRNN), аналогичная той, которая была реализована в главе 5. Используемая реализация и предварительно обученные веса были получены из источника [44].

Архитектура CRNN принимает на вход изображение, содержащее горизонтально выровненный текст, и генерирует соответствующий текст на выходе.

## 6.3 Результаты

В представленной модели объединяется оба вышеописанных компонента для достижения результатов детекции и распознавания текста, предлагается удобный для пользователя модуль на языке Python. В результате мы можем получить аннотированное изображение, включающее местоположение текста и его транскрипцию. Структура модели и процесс распознавания текста на тестовом изображении представлены на рисунке 43.

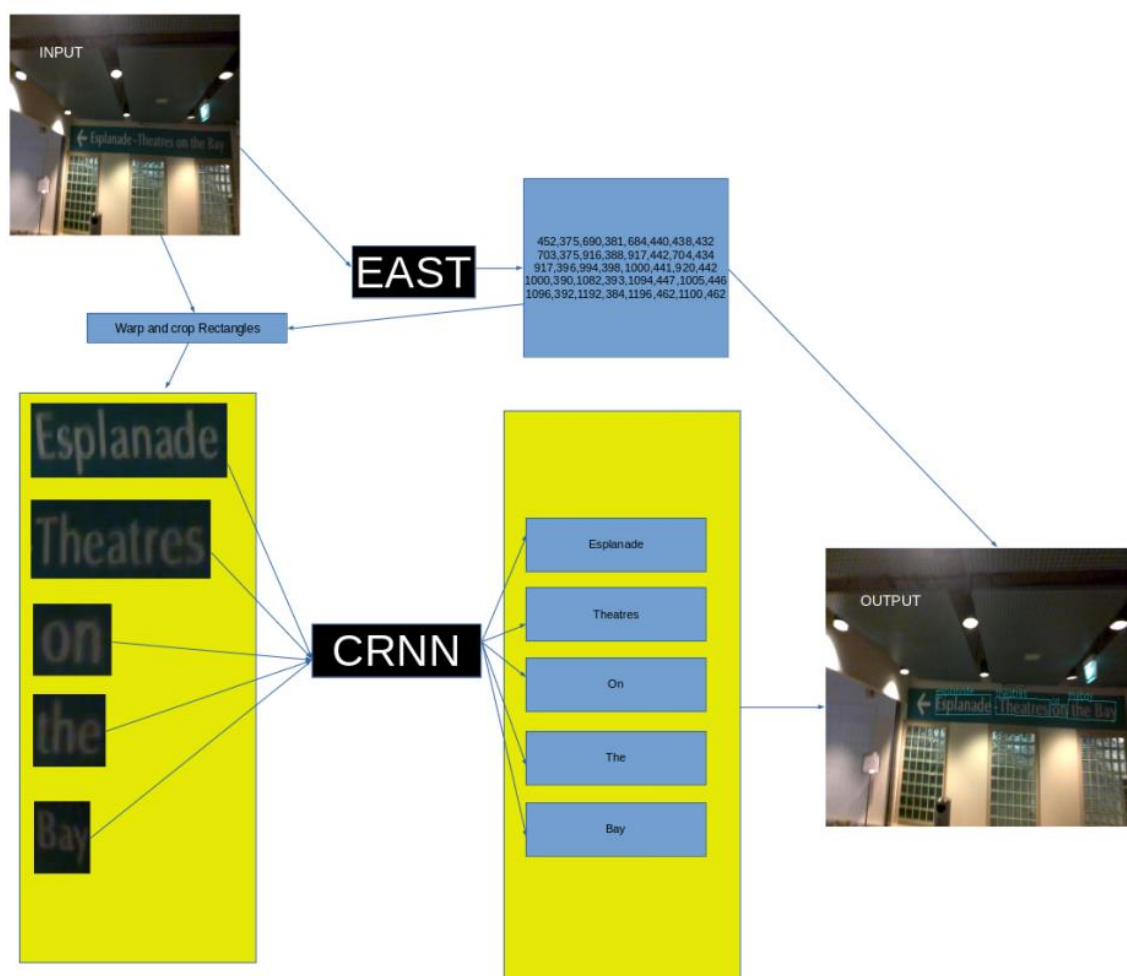


Рисунок 43 – Схематичное представление объединённой модели и процесса распознавания текста

На начальном этапе исследования основной задачей было глубокое понимание основных концепций, лежащих в основе EAST и CRNN, поскольку с ними будут проводиться дальнейшие, более сложные эксперименты. Были получены эталонные результаты детекции и распознавания, которые послужат ориентиром в экспериментах с модификациями EAST и FOTS, а также при исследовании оригинальной архитектуры CRNN.

## 7 МОДИФИЦИРОВАНИЕ АРХИТЕКТУР

В данной работе также представлены попытки разработать собственные архитектуры нейронных сетей, основанных на изученных ранее. Основная идея заключалась в улучшении моделей таким образом, чтобы они стали более легковесными – быстрее обучались и работали.

Успешным оказался эксперимент с разработкой легковесной версии EAST. При разработке легковесной модификации FOTS возникли трудности, в первую очередь с нехваткой вычислительных ресурсов, однако она может быть обучена и протестирована в дальнейшем.

### 7.1 Легковесная модель детекции

Модифицированная версия EAST была получена благодаря использованию трансферного обучения и некоторым другим изменениям. После многочисленных экспериментов в качестве базовой (то есть лежащей в основе и перенесенной при помощи трансферного обучения) нейронной сети была выбрана MobileNet от Keras.

Как уже было сказано, MobileNet отличается значительно меньшим количеством параметров по сравнению с многими другими моделями, что и придает разработанной модификации легковесный характер. В данной модели извлекается выход поточечных сверток из различных слоев MobileNet. В частности, основа состоит из 13 блоков, состоящих из разделяемых сверток. Каждый блок состоит из свертки в глубину, за которой следует поточечная свертка. Выходы поточечных сверток извлекаются в блоках 2, 5, 11 и 13 обозначаются как  $f_1$ ,  $f_2$ ,  $f_3$  и  $f_4$ , соответственно.

Для определения  $g_i$  и  $h_i$  используются две операции Keras – Conv2D и UpSampling2D. Из  $g_4$  извлекаются следующие компоненты:

- 1) score map, полученная путем применения свертки  $1 \times 1$  с одним фильтром и использования сигмоидной функции активации из Keras.

Эта функция активации гарантирует, что значения находятся в диапазоне от 0 до 1;

- 2) geometry map, созданная путем применения свертки  $1 \times 1$  с 4 фильтрами, представляющими расстояния каждого пикселя до верхней, правой, нижней и левой границ текстовой области. Здесь также применяется сигмоидная функция активации, а затем значения масштабируются по размеру изображения в пределах от 0 до 1;
- 3) angle map, полученная путем применения свертки  $1 \times 1$  с одним фильтром и использованием сигмоидной функции активации. Затем выход этой свертки преобразуется в угловое представление в соответствии с правилом  $y' = (y - 1/2) \times \pi/2$ , в результате чего значения в диапазоне от 0 до 1 переходят в значения в диапазоне от  $-\pi/4$  до  $\pi/4$ .

Для повышения производительности и предотвращения переобучения в модифицированной модели также введены ядерные регуляризаторы (kernel regularizers), которые накладывают дополнительный штраф на функцию потерь. Этот штраф определяется путем взятия квадрата от вектора параметров или весов. Если у нас есть слои  $W_1, \dots, W_N$ , где параметрами  $W_1$  являются  $\alpha_1^1, \dots, \alpha_1^{p_1}$ , то потери, которые мы оптимизируем, описываются функцией:

$$L' = L + \lambda \sum_{i=1}^N \sum_{j=1}^{p_i} (\alpha_i^j)^2,$$

где  $\lambda$  – коэффициент регуляризации. Данный способ называется L2-регуляризацией.

В данной работе был применен коэффициент регуляризации  $\lambda = 0,001$  ко всем сверткам.

## 7.2 Легковесная сквозная модель

Теперь воспользуемся разработанным выше (при описании легковесной версии EAST) компонентом детекции и добавим к нему компонент распознавания.

В рамках данного эксперимента была создана реализация RoIRotate. Для этого была использована нейронная сеть Spatial Transformer Network [45], которая позволила реализовать аффинное преобразование интересующих регионов таким образом, чтобы все операции были дифференцируемы. Как известно, это является необходимым условием для корректной работы оптимизатора (градиентного спуска).

После преобразования батча из  $N$  изображений в  $N'$  областей интереса, их общие свертки (с 32 каналами) трансформируются с помощью RoIRotate и передаются на вход CRNN.

После получения выходного сигнала от полносвязного слоя применяется СТС-декодирование, аналогично эксперименту, описанному в разделе 5.1. Стоит отметить, что потери при детекции идентичны потерям, используемым в легковесной EAST. В процессе обучения был использован тот же набор данных, что и для обучения легковесной EAST.

Разработанная нейронная сеть состоит из набора слоев, в соответствующей последовательности представленных в таблице 5.

Таблица 5 – Слои исследуемой архитектуры легковесной нейронной сети

| № | Тип слоя           | Ф-я активации | Ядро | Шаг | Кол-во каналов |
|---|--------------------|---------------|------|-----|----------------|
| 1 | Conv2D             | ReLU          | 3×3  | -   | 64             |
| 2 | Conv2D             | ReLU          | 3×3  | -   | 128            |
| 3 | BatchNormalization | -             | -    | -   | -              |
| 4 | MaxPooling2D       | -             | 2×2  | 2×2 | -              |
| 5 | Conv2D             | ReLU          | 3×3  | -   | 256            |



Окончание таблицы 5

|    |                    |         |     |   |     |
|----|--------------------|---------|-----|---|-----|
| 6  | Conv2D             | ReLU    | 3×3 | - | 256 |
| 7  | BatchNormalization | -       | -   | - | -   |
| 8  | Dense              | ReLU    | -   | - | -   |
| 9  | Bidirectional      | -       | -   | - | -   |
| 10 | Dropout            | -       | -   | - | -   |
| 11 | Dense              | Softmax | -   | - | -   |

## 8 ИССЛЕДОВАНИЕ МОДИФИЦИРОВАННЫХ АРХИТЕКТУР

В рамках данного эксперимента оценивается эффективность модифицированных архитектур.

### 8.1 Описание эксперимента

В этом эксперименте также используется оптимизатор Adam, параметр скорости обучения (learning rate) которого установлен на  $\alpha_0 = 0.0001$ , однако теперь скорость обучения изменяется в зависимости от эпохи при помощи функции планировщика (scheduler). Планировщик создает экспоненциальное снижение, заставляя скорость обучения постепенно затухать. Для каждого шага  $s$  вычисляется скорость обучения  $\alpha_s = \alpha_0 \cdot \lambda^s$ , где коэффициент затухания  $\lambda = 0.94$ .

Скорость обучения влияет на то, насколько большой шаг делается в нисходящем направлении при градиентном спуске. Чем выше скорость обучения, тем быстрее уменьшаются потери (loss), но тем больше риск попасть в локальные минимумы вместо того, чтобы найти глобальный.

Кроме того, каждые пять эпох будем запускать алгоритм на тестовом наборе, включая часть NMS, для которой, следует обратить внимание, расчет потерь не нужен.

### 8.2 Методика оценивания

В эксперименте с CRNN+CTC-loss для оценивания модели вычислялась метрика расстояния Левенштейна, применяемая к моделям распознавания, использующим словари. Поскольку модели, представленные в данном эксперименте, имеют совершенно иную логику, необходимо ввести новые метрики:

- precision – доля объектов, относящихся к положительному классу, отнесённых моделью к положительному классу. Данная метрика

показывает, насколько точно модель способна обнаруживать необходимый (положительный) класс;

- recall – доля объектов из всех объектов положительного класса, отнесённых моделью к положительному классу. Данная метрика показывает, насколько хорошо модель способна отличать один класс от другого;
- F-score – метрика, которая является компромиссом между Precision и Recall. Позволяет варьировать коэффициент баланса  $\beta$  от 0 до 2 между двумя метриками. F-score также называется F1 в случае, если  $\beta = 1$ . [46]

Метрики Precision и Recall находятся в балансе: при росте одной из метрик уменьшается вторая. В данном эксперименте мы ориентируемся, преимущественно, на метрику F-score, поскольку она является в некотором смысле универсальной.

На рисунке 44 изображен график, показывающий изменение значений вышеописанных метрик в зависимости от порогового значения классификации. Визуальное представление этого порогового значения – прямая, разделяющая классы в некотором пространстве объектов (ответов).

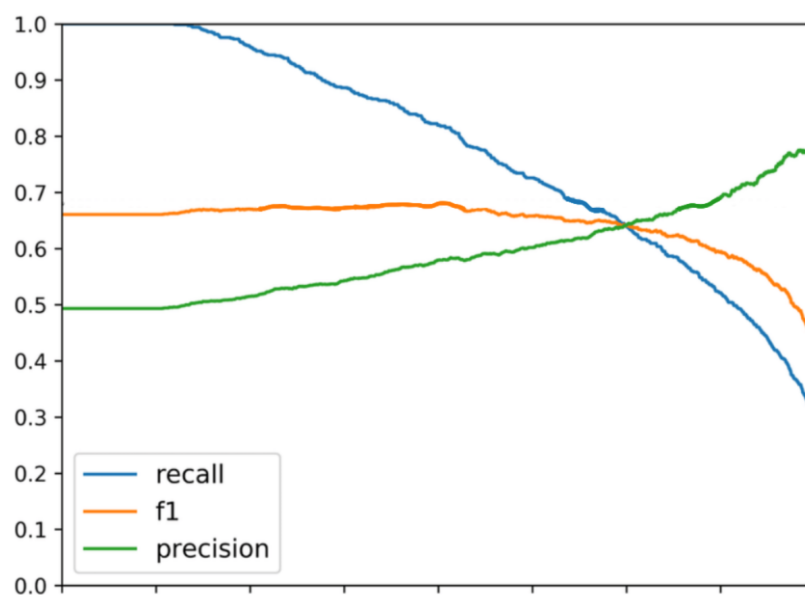


Рисунок 44 – Визуальное сравнение метрик Precision, Recall и F1

Следует напомнить, что модель детекции предсказывает координаты области с текстом. Следовательно, будет ошибочно полагать, что расположение верно классифицированной области должно идеально совпадать с расположением истинной области.

Решение этой проблемы является крайне простым: введём некоторое пороговое значение, которое должна превысить метрика IoU для того, чтобы модель отнесла предсказанную область к необходимому классу. Таким образом, для каждой из областей с текстом каждого экземпляра изображения (заметим, что изображение может содержать несколько областей с текстом) будем вычислять соотношение IoU, проверять условие (сравнивать с пороговым значением) и на основе этого вычислять итоговое значение метрики качества модели.

Что касается сквозной модели, в которой, помимо детекции, также выполняется и распознавание, то здесь добавляется дополнительное к вышеописанному условие. После сравнения IoU с пороговым значением, в случае положительного результата (верной детекции), выделенный текст посимвольно сравнивается с ответом. В случае, если все символы совпадают, предсказание считается истинноположительным (true positive).

### 8.3 Описание данных

В данном эксперименте использовались наборы данных ICDAR15 [30] и ICDAR13. Итоговый набор тренировочных данных включает 230 изображений из ICDAR13 и 1000 изображений из ICDAR15. На рисунке 45 показаны примеры изображений из двух исходных наборов данных.



Рисунок 45 – Примеры изображений из наборов данных ICDAR13 (слева) и ICDAR15 (справа)

Все изображения имеют формат JPEG и сопровождаются соответствующим TXT-файлом, содержащим данные о положении текста на изображении. Эти данные также были нормализованы и приведены к единому формату. Формат файла включает строки, определяющие координаты текстовых областей:

$$((x1,y1),(x2,y2),(x3,y3),(x3,y3),(x4,y4))$$

Для слов, которые считаются слишком трудными для распознавания, используется пустое слово «####»:

210,340,275,330,276,344,210,354,BEWARE  
 234,352,256,349,257,363,235,367,###  
 223,369,273,363,273,376,224,382,TEPS

Для валидации было использовано 500 валидационных изображений из набора данных ICDAR15.

Учитывая ограниченность данных, становится крайне важным предотвратить переобучение. Для решения этой проблемы будем использовать методы аугментации, позволяющие внести большее разнообразие в набор данных без необходимости использования дополнительных изображений.

Методы аугментации данных включают в себя применение к исходным изображениям случайных поворотов, отзеркаливания по вертикали и горизонтали, изменения цветовой гаммы, создания шума, обрезания и масштабирования (см. рисунок 46). Важно уточнить, что эти преобразования, в случае, если при аугментации изображения изменяются координаты областей с текстом, также применяются для файлов с ответами, а сам процесс аугментации проводится исключительно над тренировочным набором данных.



Рисунок 46 – Пример изображения и нескольких его аугментированных вариантов

Таким образом, аугментация позволяет создать несколько вариантов одного изображения, тем самым увеличивая размер исходного набора данных и снижая риск переобучения. Реализованный в данной работе подход к аугментации вдохновлен подходом, представленным в реализации EAST [45].

## 8.4 Результаты

Далее представлены результаты экспериментов, проведенных над эталонной (объединённой) моделью, а также над легковесными модификациями архитектур.

### 8.4.1 Объединение моделей

Как упоминалось ранее, в данной работе были объединены две заранее обученные модели. Результаты распознавания итоговой модели будем считать неким эталоном, к которому необходимо прийти, используя более легковесные модели. В таблице 6 показаны значения различных метрик для разработанной объединённой сквозной модели на обоих этапах работы.

Таблица 6 – Сравнение эффективности работы разработанной модели на этапах детекции и распознавания

| Результат     | Precision | Recall | F-score |
|---------------|-----------|--------|---------|
| Детекция      | 0.845     | 0.763  | 0.801   |
| Распознавание | 0.443     | 0.385  | 0.411   |

### 8.4.2 Легковесный EAST

Вначале был проведен эксперимент с отключенными переменными предобученной модели. Этот подход, запрещающий алгоритму, обновляющему значения при прямом и обратном распространении ошибки изменять значения переменных, называется заморозкой параметров базовой модели и описан в официальной документации Keras. [47]

В процессе обучения участвовали только дополнительные слои, которые были добавлены поверх базовой модели MobileNet. После 90 эпох базовая модель была разморожена, а затем проведен процесс тонкой настройки модели (fine tuning), то есть обучение всех параметров на низкой скорости обучения (learning rate). В таблице 7 представлены гиперпараметры, которые были заданы для этих двух этапов обучения.

Таблица 7 – Гиперпараметры модифицированной архитектуры EAST с замороженной базовой моделью

| Этап обучения                            | Замороженная модель | Тонкая настройка |
|------------------------------------------|---------------------|------------------|
| Размер батча                             | 8                   | 4                |
| Количество эпох                          | 90                  | 120              |
| Коэффициент скорости обучения $\alpha_0$ | 0.0001              | 0.00009          |
| Коэффициент затухания $\lambda$          | 0.94                | 0.94             |

Результат оказался неудовлетворительным. Значение метрики F-score не достигало 30%, а тонкая настройка параметров повысила точность лишь незначительно. Очевидно, что продолжать обучение, увеличив количество эпох, не имеет смысла.

Поскольку точность с замороженной базовой моделью оказалась слишком низкой, эксперимент был повторно проведен без какого-либо замораживания. Начиная с первой эпохи, модель обучалась на всех параметрах.

В этот раз в процессе обучения был замечен практически непрерывный рост значения метрики F-score в зависимости от эпохи. На рисунке 47 показаны графики точности для обоих подходов.



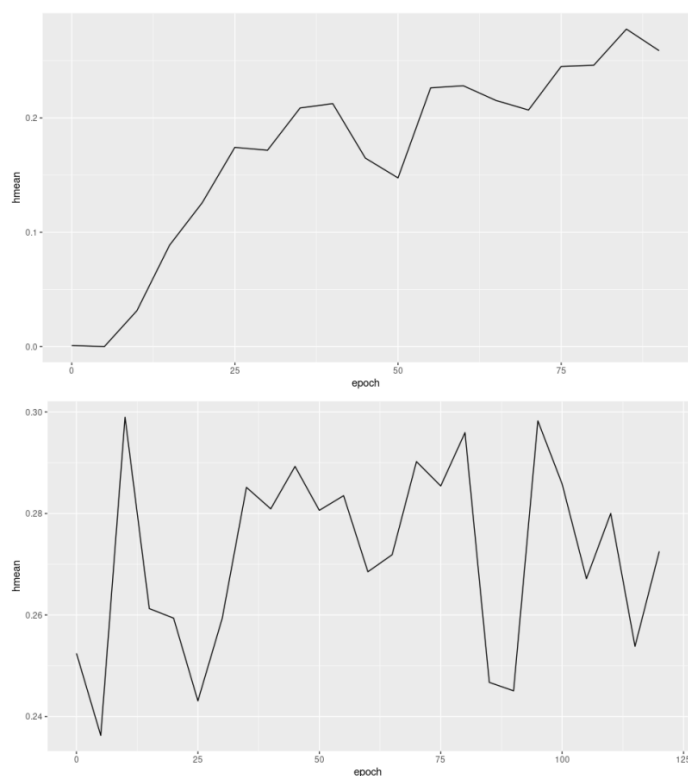


Рисунок 47 – Сравнение графиков точности моделей с замороженными базовыми параметрами и с размороженными базовыми параметрами в зависимости от эпохи обучения

Принимая во внимание характер изменения значения F-score, модель была повторно обучена на большем количестве эпох. В таблице 8 представлены гиперпараметры, которые были заданы для данной модели.

Таблица 8 – Гиперпараметры модифицированной архитектуры EAST с размороженной базовой моделью

|                                          |        |
|------------------------------------------|--------|
| Размер батча                             | 4      |
| Количество эпох                          | 400    |
| Коэффициент скорости обучения $\alpha_0$ | 0.0001 |
| Коэффициент затухания $\lambda$          | 0.94   |

Полученная точность оказалась близка к той, которую показывала исходная модель EAST. Однако, модифицированная модель имеет гораздо меньшее

количество параметров, чем исходная. В таблице 9 представлены результаты для исходной и модифицированной моделей на различных метриках, а также количество параметров этих моделей.

Таблица 9 – Сравнение точности работы и количества параметров оригинальной и модифицированной архитектур

| Метод     | Precision | Recall | F-score | Кол-во параметров |
|-----------|-----------|--------|---------|-------------------|
| EAST      | 0.847     | 0.773  | 0.808   | 24.2 млн.         |
| Мод. EAST | 0.841     | 0.681  | 0.753   | 3.6 млн.          |

Для модифицированной и оригинальной моделей проводилось несколько тестов времени работы (inference) на различных размерах изображений из набора данных ICDAR15. Для каждого теста было взято 500 тестовых изображений. В таблице 10 представлены размеры изображений и среднее время обработки одного изображения, а на рисунке 48 – график зависимости среднего времени обработки изображения от ширины изображения для модифицированной и оригинальной моделей. Наблюдается значительное ускорение работы модифицированной модели по сравнению с оригинальной.

Таблица 10 – Сравнение скорости работы легковесной модели на различных размерах входного изображения

| Ширина,<br>пк. | Высота,<br>пк. | Время для<br>EAST, сек. | Время для мод.<br>EAST, сек. | Прирост<br>скорости, % |
|----------------|----------------|-------------------------|------------------------------|------------------------|
| 320            | 180            | 0.089                   | 0.052                        | 71                     |
| 640            | 360            | 0.199                   | 0.056                        | 255                    |
| 960            | 540            | 0.231                   | 0.084                        | 175                    |
| 1280           | 720            | 0.242                   | 0.115                        | 110                    |
| 1600           | 900            | 0.242                   | 0.127                        | 90                     |

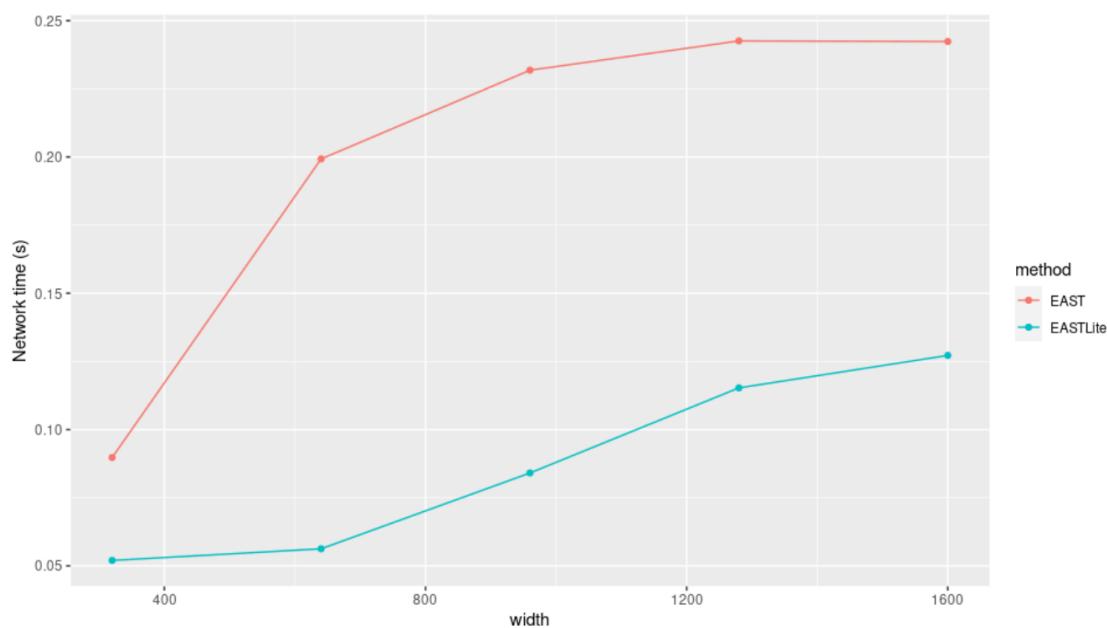


Рисунок 48 – График зависимости среднего времени обработки изображения от ширины изображения для модифицированной и оригинальной моделей

На рисунке 49 приведен пример работы модифицированной модели на фотографии, сделанной на улице в тёмное время суток.

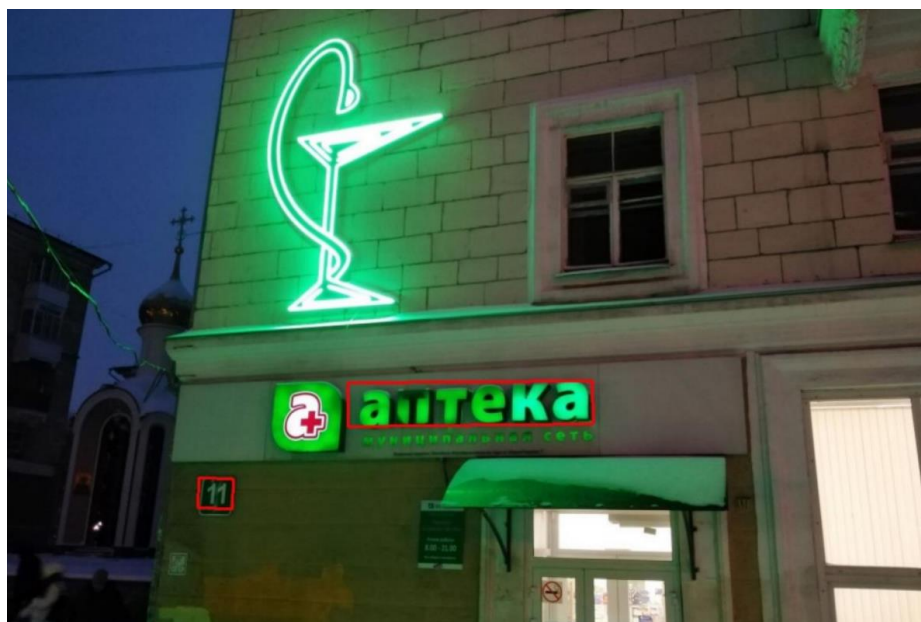


Рисунок 49 – Пример выделения на изображении областей с текстом в тёмное время суток

На рисунке 50 приведен пример работы модифицированной модели на фотографии, имеющей затемнения и текстовую область, занимающую большую часть изображения.

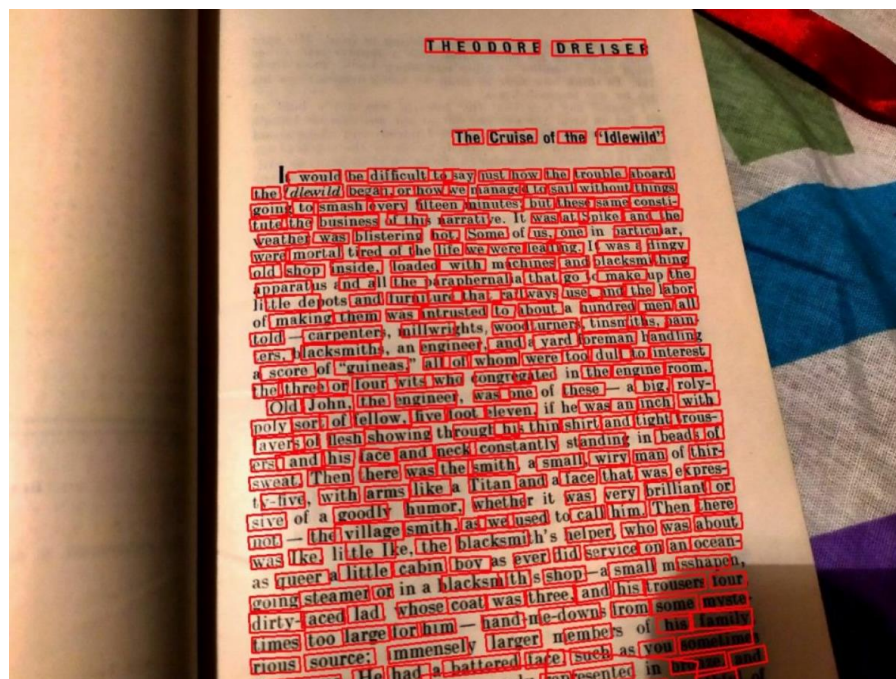


Рисунок 50 – Пример выделения на изображении большого количества областей с текстом с затемнениями

### 8.4.3 Легковесный FOTS

При реализации модифицированной легковесной архитектуры FOTS возникло большое количество проблем. Хотя созданная модель и может быть обучена, результаты обучения показали плохие результаты. Кроме того, из-за нехватки памяти в используемой аппаратной инфраструктуре пришлось работать с размером батча 1, так как в противном случае возникали бы ошибки характера "out of memory".

Можно сделать вывод, что модель требует доработки. Как минимум, имеет смысл завершить её полноценное обучение, используя более развитую аппаратную инфраструктуру, с большим количеством памяти и графических ядер. Результаты обучения представлены на рисунке 51.

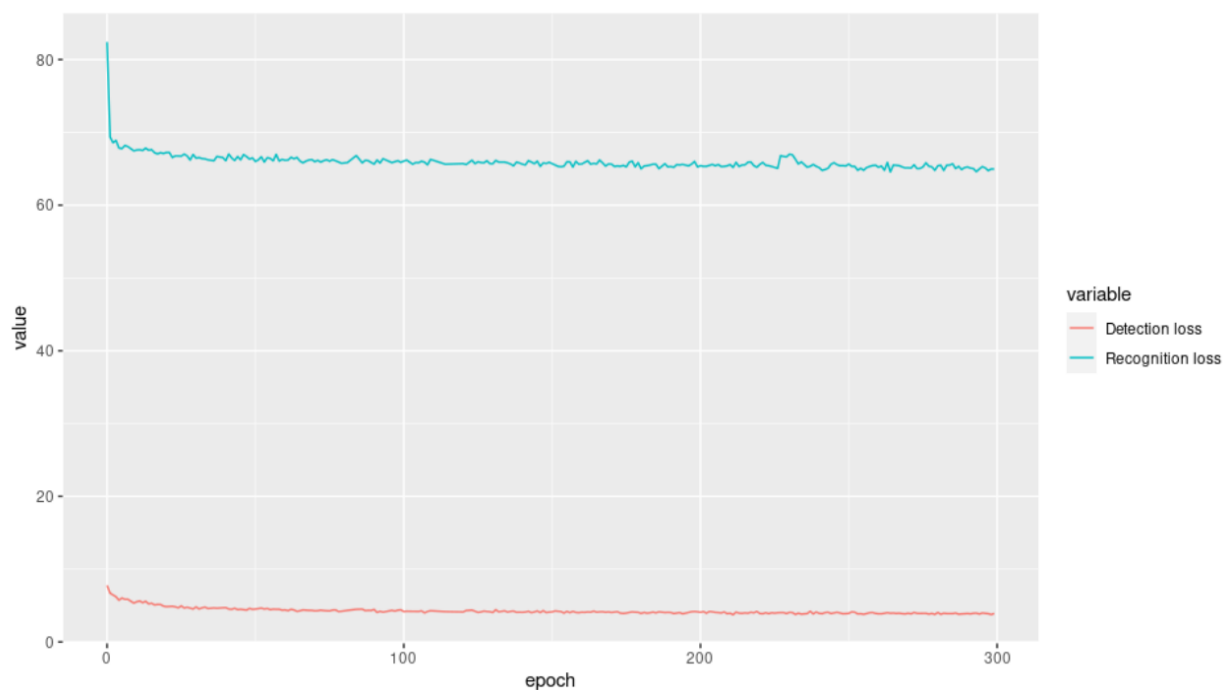


Рисунок 51 – Потери при детекции и распознавании в процессе обучения модифицированной модели FOTS

## ЗАКЛЮЧЕНИЕ

В данной работе был проведён детальный обзор и анализ некоторых существующих методов детекции и распознавания текста на изображениях сложных графических сцен. Выводы из этих исследований, а также многочисленные эксперименты, вдохновили на реализацию уникальных модифицированных архитектур нейронных сетей.

В рамках исследований были изучены такие модели, как EAST, FOTS, CRNN, MobileNet и CRAFT. Над архитектурой CRNN была проведена более детальное исследование, поскольку именно эта архитектура была избрана как кандидат на компонент распознавания в разработанных сквозных моделях.

Главным результатом работы можно считать создание новой, ресурсоэффективной модели детекции, способной выделять область с текстом в несколько раз быстрее, чем модели, показывающие сопоставимую точность.

Кроме того, была также предложена легковесная сквозная модель, основанная на вышеупомянутой модели детекции. С её обучением возникли трудности, связанные с нехваткой вычислительных ресурсов.

Модифицированные архитектуры были разработаны на языке Python с использованием фреймворка TensorFlow. Полученное программное обеспечение не имеет пользовательского интерфейса, а скорее является программным интерфейсом (API), который пользователь сможет с лёгкостью внедрять в собственные программные системы.

Также можно выделить несколько направлений будущей работы над данным проектом:

1. адекватное обучение легковесной сквозной модели с использованием подходящей аппаратной инфраструктуры, а также дальнейшее исследование данной модели;
2. переход от повернутых прямоугольников к более гибким геометрическим формам;

3. адаптирование моделей к работе с другими языками при помощи трансферного обучения;
4. работа над оптимизацией моделей, исследование различных модификаций алгоритма градиентного спуска в контексте данной задачи;
5. работа над внедрением моделей в мобильные устройства и тестирование в условиях, когда требуется быстроедействие;
6. проведение исследования существующих алгоритмов бинаризации изображений применительно к разработанной сквозной модели и попытка внедрения этапа бинаризации в эту модель с целью уменьшения затрат памяти при обучении.

## БИБЛИОГРАФИЧЕСКИЙ СПИСОК

1. Deep Learning Based OCR for Text in the Wild // nanonets.com : сайт. – URL: <https://nanonets.com/blog/deep-learning-ocr/> (дата обращения: 08.06.2023).
2. Rail OCR based on AI // www.supplai.nl : сайт. – URL: <https://www.supplai.nl/en/products-ai/rail-ocr-automation/> (дата обращения: 08.06.2023).
3. Text Recognition in the Wild: A Survey / X. Chen, L. Jin, Y. Zhu [и др.] // arXiv.org : электронный журнал. – URL: <https://arxiv.org/abs/2005.03492>. – Дата публикации: 03.12.2020.
4. Krizhevsky, A. ImageNet Classification with Deep Convolutional Neural Networks / A. Krizhevsky, I. Sutskever, G. E. Hinton // proceedings.neurips.cc : электронный журнал. – URL: [https://proceedings.neurips.cc/paper\\_files/paper/2012/file/c399862d3b9d6b76c8436e924a68c45b-Paper.pdf](https://proceedings.neurips.cc/paper_files/paper/2012/file/c399862d3b9d6b76c8436e924a68c45b-Paper.pdf). – Дата публикации: 2012.
5. EAST: An Efficient and Accurate Scene Text Detector / X. Zhou, C. Yao, H. Wen [и др.] // arXiv.org : электронный журнал. – URL: <https://arxiv.org/abs/1704.03155>. – Дата публикации: 10.07.2017.
6. MobileNets: Efficient Convolutional Neural Networks for Mobile Vision Applications / A. G. Howard, M. Zhu, B. Chen [и др.] // arXiv.org : электронный журнал. – URL: <https://arxiv.org/abs/1704.04861>. – Дата публикации: 17.04.2017.
7. An End-to-End Trainable Neural Network for Image-based Sequence Recognition and Its Application to Scene Text Recognition / B. Shi, X. Bai, C. Yao [и др.] // arXiv.org : электронный журнал. – URL: <https://arxiv.org/abs/1507.05717>. – Дата публикации: 21.07.2015.
8. FOTS: Fast Oriented Text Spotting with a Unified Network / X. Liu, D. Liang, S. Yan [и др.] // arXiv.org : электронный журнал. – URL: <https://arxiv.org/abs/1801.01671>. – Дата публикации: 15.06.2018.



9. Задача нахождения объектов на изображении // neerc.ifmo.ru : сайт. – URL:  
[https://neerc.ifmo.ru/wiki/index.php?title=Задача\\_нахождения\\_объектов\\_на\\_изображении](https://neerc.ifmo.ru/wiki/index.php?title=Задача_нахождения_объектов_на_изображении) (дата обращения: 08.06.2023).
10. Deep Residual Learning for Image Recognition / K. He, X. Zhang, S. Ren [и др.] // arXiv.org : электронный журнал. – URL:  
<https://arxiv.org/abs/1512.03385>. – Дата публикации: 10.12.2015.
11. Ioffe, S. Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift / S. Ioffe, C. Szegedy // arXiv.org : электронный журнал. – URL: <https://arxiv.org/abs/1502.03167>. – Дата публикации: 02.03.2015.
12. Kunishige, F. Scenery Character Detection with Environmental Context / F. Kunishige, F. Yaokai, S. Uchida // www.iapr-tc11.org : электронный журнал. – URL: <http://www.iapr-tc11.org/archive/icdar2011/fileup/PDF/4520b049.pdf>. – Дата публикации: 2011.
13. Epshtein, B. Detecting Text in Natural Scenes with Stroke Width Transform / B. Epshtein, E. Ofek, Y. Wexler // ResearchGate : электронный журнал. – URL:  
[https://www.researchgate.net/publication/224164328\\_Detecting\\_Text\\_in\\_Natural\\_Scenes\\_with\\_Stroke\\_Width\\_Transform](https://www.researchgate.net/publication/224164328_Detecting_Text_in_Natural_Scenes_with_Stroke_Width_Transform). – Дата публикации: 2010.
14. Du, Y. Dot Text Detection Based on FAST Points / Y. Du, H. Ai, S. Lao // dblp.org : электронный журнал. – URL:  
<https://dblp.org/rec/conf/icdar/DuAL11.html>. – Дата публикации: 2011.
15. Text-attentional convolutional neural network for scene text detection / T. He, W. Huang, Y. Qiao, J. Yao // arXiv.org : электронный журнал. – URL: <https://arxiv.org/pdf/1510.03283.pdf>. – Дата публикации: 2016.
16. Mask TextSpotter: An End-to-End Trainable Neural Network for Spotting Text with Arbitrary Shapes / M. Liao, P. Lyu, M. He [и др.] // arXiv.org :

- электронный журнал. – URL: <https://arxiv.org/abs/1908.08207>. – Дата публикации: 22.08.2019.
- 17.PhotoOCR : Reading Text in Uncontrolled Conditions / A. Bissacco, M. Cummins, Y. Netzer [и др.] // ResearchGate : электронный журнал. – URL: [https://www.researchgate.net/publication/319770431\\_PhotoOCR\\_Reading\\_Text\\_in\\_Uncontrolled\\_Conditions](https://www.researchgate.net/publication/319770431_PhotoOCR_Reading_Text_in_Uncontrolled_Conditions). – Дата публикации: 2013.
- 18.Jaderberg, M. Deep Features for Text Spotting / M. Jaderberg, A. Vedaldi, A. Zisserman // ResearchGate : электронный журнал. – URL: [https://www.researchgate.net/publication/319770170\\_Deep\\_Features\\_for\\_Text\\_Spotting](https://www.researchgate.net/publication/319770170_Deep_Features_for_Text_Spotting). – Дата публикации: 2014.
- 19.Synthetic Data and Artificial Neural Networks for Natural Scene Text Recognition / M. Jaderberg, K. Simonyan, A. Vedaldi, A. Zisserman // arXiv.org : электронный журнал. – URL: <https://arxiv.org/abs/1406.2227>. – Дата публикации: 09.12.2014.
- 20.Reading Text in the Wild with Convolutional Neural Networks / M. Jaderberg, K. Simonyan, A. Vedaldi, A. Zisserman // arXiv.org : электронный журнал. – URL: <https://arxiv.org/abs/1412.1842>. – Дата публикации: 04.12.2014.
- 21.Multi-digit Number Recognition from Street View Imagery using Deep Convolutional Neural Networks / I. J. Goodfellow, B. Bulatov, J. Ibarz [и др.] // arXiv.org : электронный журнал. – URL: <https://arxiv.org/abs/1312.6082>. – Дата публикации: 14.04.2014.
- 22.Connectionist temporal classification: Labelling unsegmented sequence data with recurrent neural networks / A. Graves, S. Fernandez, F. Gomez, J. Schmidhuber // ResearchGate : электронный журнал. – URL: [https://www.researchgate.net/publication/221346365\\_Connectionist\\_temporal\\_classification\\_Labelling\\_unsegmented\\_sequence\\_data\\_with\\_recurrent\\_neural\\_networks](https://www.researchgate.net/publication/221346365_Connectionist_temporal_classification_Labelling_unsegmented_sequence_data_with_recurrent_neural_networks). – Дата публикации: 2006.

23. Ghosh, S. Visual Attention Models for Scene Text Recognition / S. Ghosh, E. Valveny, A. Bagdanov // ResearchGate : электронный журнал. – URL: [https://www.researchgate.net/publication/322780356\\_Visual\\_Attention\\_Models\\_for\\_Scene\\_Text\\_Recognition](https://www.researchgate.net/publication/322780356_Visual_Attention_Models_for_Scene_Text_Recognition). – Дата публикации: 2017.
24. Scene Text Recognition with Sliding Convolutional Character Models / F. Yin, Y. Wu, X. Zhang, C. Liu // arXiv.org : электронный журнал. – URL: <https://arxiv.org/abs/1709.01727>. – Дата публикации: 06.09.2017.
25. SCAN: Sliding Convolutional Attention Network for Scene Text Recognition / Y. Wu, F. Yin, X. Zhang [и др.] // arXiv.org : электронный журнал. – URL: <https://arxiv.org/abs/1806.00578>. – Дата публикации: 02.06.2018.
26. A Simple and Robust Convolutional-Attention Network for Irregular Text Recognition / P. Wang, L. Yang, H. Li [и др.] // www.semanticscholar.org : электронный журнал. – URL: <https://www.semanticscholar.org/paper/A-Simple-and-Robust-Convolutional-Attention-Network-Wang-Yang/5bf577d7f378138d37a165cf764cb1967392cb65>. – Дата публикации: 2019.
27. Liu, W. SAFE: Scale Aware Feature Encoder for Scene Text Recognition / W. Liu, C. Chen, K. Wong // arXiv.org : электронный журнал. – URL: <https://arxiv.org/abs/1901.05770>. – Дата публикации: 17.06.2019.
28. The Challenge of Vanishing/Exploding Gradients in Deep Neural Networks // www.analyticsvidhya.com : сайт. – URL: <https://www.analyticsvidhya.com/blog/2021/06/the-challenge-of-vanishing-exploding-gradients-in-deep-neural-networks/> (дата обращения: 08.06.2023).
29. Deep Residual Learning for Image Recognition / K. He, X. Zhang, S. Ren, J. Sun // arXiv.org : электронный журнал. – URL: <https://arxiv.org/abs/1512.03385>. – Дата публикации: 10.12.2015.

30. Overview - Incidental Scene Text // rrc.cvc.uab.es : сайт. – URL: <https://rrc.cvc.uab.es/?ch=4&com=introduction> (дата обращения: 08.06.2023).
31. Hosang, J. Learning non-maximum suppression / J. Hosang, R. Benenson, B. Schiele // arXiv.org : электронный журнал. – URL: <https://arxiv.org/abs/1705.02950>. – Дата публикации: 09.05.2017.
32. PVANET: Deep but Lightweight Neural Networks for Real-time Object Detection / S. Hong, B. Roh, Y. Cheon [и др.] // arXiv.org : электронный журнал. – URL: <https://arxiv.org/abs/1608.08021>. – Дата публикации: 30.09.2017.
33. Xie, S. Holistically-Nested Edge Detection / S. Xie, Z. Tu // arXiv.org : электронный журнал. – URL: <https://arxiv.org/abs/1504.06375>. – Дата публикации: 04.10.2015.
34. Scene Text Detection via Holistic, Multi-Channel Prediction / C. Yao, X. Bai, N. Sang [и др.] // arXiv.org : электронный журнал. – URL: <https://arxiv.org/abs/1606.09002>. – Дата публикации: 05.07.2016.
35. Intersection over Union (IoU) for object detection // pyimagesearch.com : сайт. – URL: <https://pyimagesearch.com/2016/11/07/intersection-over-union-iou-for-object-detection/> (дата обращения: 08.06.2023).
36. Kingma, D. P. Adam: A Method for Stochastic Optimization / D. P. Kingma, J. Ba // arXiv.org : электронный журнал. – URL: <https://arxiv.org/abs/1412.6980>. – Дата публикации: 30.01.2017.
37. Character Region Awareness for Text Detection / Y. Baek, B. Lee, D. Han [и др.] // arXiv.org : электронный журнал. – URL: <https://arxiv.org/abs/1904.01941>. – Дата публикации: 03.04.2019.
38. Simonyan, K. Very Deep Convolutional Networks for Large-Scale Image Recognition / K. Simonyan, A. Zisserman // arXiv.org : электронный журнал. – URL: <https://arxiv.org/abs/1409.1556>. – Дата публикации: 10.04.2015.

39. Shi, B. An End-to-End Trainable Neural Network for Image-Based Sequence Recognition and Its Application to Scene Text Recognition / B. Shi, X. Bai, C. Yao // ResearchGate : электронный журнал. – URL: [https://www.researchgate.net/publication/280330424\\_An\\_End-to-End\\_Trainable\\_Neural\\_Network\\_for\\_Image-Based\\_Sequence\\_Recognition\\_and\\_Its\\_Application\\_to\\_Scene\\_Text\\_Recognition](https://www.researchgate.net/publication/280330424_An_End-to-End_Trainable_Neural_Network_for_Image-Based_Sequence_Recognition_and_Its_Application_to_Scene_Text_Recognition). – Дата публикации: 2015.
40. Hochreiter, S. Long Short-term Memory / S. Hochreiter, J. Schmidhuber // ResearchGate : электронный журнал. – URL: [https://www.researchgate.net/publication/13853244\\_Long\\_Short-term\\_Memory](https://www.researchgate.net/publication/13853244_Long_Short-term_Memory). – Дата публикации: 1997.
41. Kunishige, Y. Scenery Character Detection with Environmental Context / Y. Kunishige, F. Yaokai, S. Uchida // ResearchGate : электронный журнал. – URL: [https://www.researchgate.net/publication/224265579\\_Scenery\\_Character\\_Detection\\_with\\_Environmental\\_Context](https://www.researchgate.net/publication/224265579_Scenery_Character_Detection_with_Environmental_Context). – Дата публикации: 2011.
42. Implementation of EAST scene text detector in Keras // GitHub : сайт. – URL: <https://github.com/janzd/EAST> (дата обращения: 08.06.2023).
43. Generalised Dice Overlap as a Deep Learning Loss Function for Highly Unbalanced Segmentations / C. H. Sudre, W. Li, T. Vercauteren [и др.] // ResearchGate : электронный журнал. – URL: [https://www.researchgate.net/publication/319633097\\_Generalised\\_Dice\\_Overlap\\_as\\_a\\_Deep\\_Learning\\_Loss\\_Function\\_for\\_Highly\\_Unbalanced\\_Segmentations](https://www.researchgate.net/publication/319633097_Generalised_Dice_Overlap_as_a_Deep_Learning_Loss_Function_for_Highly_Unbalanced_Segmentations). – Дата публикации: 2017.
44. Convolutional recurrent neural network for scene text recognition or OCR in Keras // GitHub : сайт. – URL: <https://github.com/janzd/CRNN> (дата обращения: 08.06.2023).

45. Spatial Transformer Networks / M. Jaderberg, K. Simonyan, A. Zisserman, K. Kavukcuoglu // arXiv.org : электронный журнал. – URL: <https://arxiv.org/abs/1506.02025>. – Дата публикации: 04.02.2016.
46. How to Calculate Precision, Recall, and F-Measure for Imbalanced Classification // machinelearningmastery.com : сайт. – URL: <https://machinelearningmastery.com/precision-recall-and-f-measure-for-imbalanced-classification/> (дата обращения: 08.06.2023).
47. Transfer learning & fine-tuning // keras.io : сайт. – URL: [https://keras.io/guides/transfer\\_learning/](https://keras.io/guides/transfer_learning/) (дата обращения: 08.06.2023).

## ПРИЛОЖЕНИЕ. ПРОГРАММНЫЙ КОД

В приложении приведены основные фрагменты кода, относящиеся к легковесной модели детекции.

model.py

```
def unpool(inputs):
    return UpSampling2D(
        size=(2, 2), data_format="channels_last", interpolation="bilinear"
    )(inputs)

def find_out_layers(m, base_model):
    transfer_layers = []
    if base_model == "resnet":
        transfer_layers = [
            "pool1_pool",
            "conv3_block4_out",
            "conv4_block6_out",
            "conv5_block3_out",
        ]
    elif base_model == "mobilenet":
        transfer_layers = [
            "conv_pw_2_relu",
            "conv_pw_5_relu",
            "conv_pw_11_relu",
            "conv_pw_13_relu",
        ]
    else:
        raise (INVALID_MODEL_MSG)
    return [l for l in m.layers if l.name in transfer_layers]

def dice_coefficient(y_true_score, y_pred_score, training_mask):
    eps = 1e-5
    intersection = tf.reduce_sum(y_true_score * y_pred_score * training_mask)
    union = tf.reduce_sum(y_true_score) + tf.reduce_sum(y_pred_score) + eps
    loss = 1.0 - (2 * intersection / union)
    tf.summary.scalar("classification_dice_loss", loss)
    return loss

def loss(y_true, y_pred):
    training_mask = y_true[:, :, :, -1:]
    y_true = y_true[:, :, :, :-1]
    classification_loss = 0.01 * dice_coefficient(
        y_true[:, :, :, -1:], y_pred[:, :, :, -1:], training_mask
    )

    d1_gt, d2_gt, d3_gt, d4_gt, theta_gt = tf.split(
        value=y_true[:, :, :, :-1], num_or_size_splits=5, axis=3
    )
    d1_pred, d2_pred, d3_pred, d4_pred, theta_pred = tf.split(
        value=y_pred[:, :, :, :-1], num_or_size_splits=5, axis=3
    )
    area_gt = (d1_gt + d3_gt) * (d2_gt + d4_gt)
```

```

area_pred = (d1_pred + d3_pred) * (d2_pred + d4_pred)
w_intersect = tf.minimum(d2_gt, d2_pred) + tf.minimum(d4_gt, d4_pred)
h_intersect = tf.minimum(d1_gt, d1_pred) + tf.minimum(d3_gt, d3_pred)
area_intersect = w_intersect * h_intersect
area_union = area_gt + area_pred - area_intersect
L_AABB = -tf.math.log((area_intersect + 1.0) / (area_union + 1.0))
L_theta = 1 - tf.math.cos(theta_pred - theta_gt)
L_g = L_AABB + 20 * L_theta

return 300 * (
    tf.reduce_mean(L_g * y_true[:, :, :, -1:] * training_mask) +
    classification_loss
)

def model(freeze=True, base_model=cfg.base_model):
    input = tf.keras.Input(shape=[None, None, 3], dtype=tf.float32)
    input = tf.keras.applications.resnet.preprocess_input(input)
    m = None
    if base_model == "resnet":
        m = tf.keras.applications.resnet50.ResNet50(
            input_tensor=input, include_top=False
        )
    elif base_model == "mobilenet":
        m = tf.keras.applications.MobileNet(input_tensor=input,
            include_top=False)
    else:
        raise (INVALID_MODEL_MSG)

    if freeze:
        for l in m.layers:
            l.trainable = False

    block_layer_indices = find_out_layers(m, base_model)
    block_layer_indices.reverse()
    f = [l.output for l in block_layer_indices]
    g = [None, None, None, None]
    h = [None, None, None, None]
    num_outputs = [None, 128, 64, 32]
    for i in range(4):
        if i == 0:
            h[i] = f[i]
        else:
            c1_1 = Conv2D(
                filters=num_outputs[i],
                kernel_size=1,
                activation="relu",
                padding="same",
                kernel_regularizer=tf.keras.regularizers.L2(0.001),
            )(tf.concat([g[i - 1], f[i]], axis=-1))
            h[i] = Conv2D(
                filters=num_outputs[i],
                kernel_size=3,
                activation="relu",
                padding="same",
                kernel_regularizer=tf.keras.regularizers.L2(0.001),
            )(c1_1)
        if i <= 2:
            g[i] = unpool(h[i])
        else:
            g[i] = Conv2D(
                filters=num_outputs[i],

```



```

        kernel_size=3,
        activation="relu",
        padding="same",
        kernel_regularizer=tf.keras.regularizers.L2(0.001),
    )(h[i])

F_score = Conv2D(
    filters=1,
    kernel_size=1,
    activation=tf.nn.sigmoid,
    padding="same",
    kernel_regularizer=tf.keras.regularizers.L2(0.001),
)(g[3])
geo_map = (
    Conv2D(
        filters=4,
        kernel_size=1,
        activation=tf.nn.sigmoid,
        padding="same",
        kernel_regularizer=tf.keras.regularizers.L2(0.001),
    )(g[3])
    * TEXT_SCALE
)
angle_map = (
    (
        Conv2D(
            filters=1,
            kernel_size=1,
            activation=tf.nn.sigmoid,
            padding="same",
            kernel_regularizer=tf.keras.regularizers.L2(0.001),
        )(g[3])
        - 0.5
    )
    * np.pi
    / 2
)
F_geometry = tf.concat([geo_map, angle_map], axis=-1)

output = tf.concat([F_geometry, F_score], axis=-1)

model = tf.keras.Model(inputs=input, outputs=[output], name="east_tf_keras")

return model

if __name__ == "__main__":
    model()

```

inference.py

```

SCORE_MAP_THRESHOLD = 0.8
BOX_THRESHOLD = 0.1
NMS_THRESHOLD = 0.2

def get_images(validation_dataset):
    files = []
    exts = ["jpg", "png", "jpeg", "JPG"]
    for parent, _, filenames in os.walk(
        cfg.validation_data_path if validation_dataset else cfg.data_path
    ):

```

```

):
    for filename in filenames:
        for ext in exts:
            if filename.endswith(ext):
                files.append(os.path.join(parent, filename))
                break
    print(f"Find {len(files)} images")
    return files

def resize_image(im, max_side_len=2400):
    h, w, _ = im.shape

    resize_w = w
    resize_h = h

    if max(resize_h, resize_w) > max_side_len:
        ratio = (
            float(max_side_len) / resize_h
            if resize_h > resize_w
            else float(max_side_len) / resize_w
        )
    else:
        ratio = 1.0
    resize_h = int(resize_h * ratio)
    resize_w = int(resize_w * ratio)

    resize_h = resize_h if resize_h % 32 == 0 else (resize_h // 32 - 1) * 32
    resize_w = resize_w if resize_w % 32 == 0 else (resize_w // 32 - 1) * 32
    resize_h = max(32, resize_h)
    resize_w = max(32, resize_w)
    im = cv2.resize(im, (int(resize_w), int(resize_h)))

    ratio_h = resize_h / float(h)
    ratio_w = resize_w / float(w)

    return im, (ratio_h, ratio_w)

def detect(score_map, geo_map, timer):
    if len(score_map.shape) == 4:
        score_map = score_map[0, :, :, 0]
        geo_map = geo_map[
            0,
            :,
            :,
        ]
    xy_text = np.argwhere(score_map > SCORE_MAP_THRESHOLD)
    xy_text = xy_text[np.argsort(xy_text[:, 0])]
    start = time.time()
    text_box_restored = utils.restore_rectangle_rbox(
        xy_text[:, :-1] * 4, geo_map[xy_text[:, 0], xy_text[:, 1], :]
    )
    boxes = np.zeros((text_box_restored.shape[0], 9), dtype=np.float32)
    boxes[:, :8] = text_box_restored.reshape((-1, 8))
    boxes[:, 8] = score_map[xy_text[:, 0], xy_text[:, 1]]
    timer["restore"] = time.time() - start
    start = time.time()
    boxes = lanms.merge_quadrangle_n9(boxes.astype("float32"), NMS_THRESHOLD)
    timer["nms"] = time.time() - start

    if boxes.shape[0] == 0:

```

```

        return None, timer

    for i, box in enumerate(boxes):
        mask = np.zeros_like(score_map, dtype=np.uint8)
        cv2.fillPoly(mask, box[:8].reshape((-1, 4, 2)).astype(np.int32) // 4, 1)
        boxes[i, 8] = cv2.mean(score_map, mask)[0]
    boxes = boxes[boxes[:, 8] > BOX_THRESHOLD]

    return boxes, timer

def get_run_name():
    _date_and_time = str(datetime.now().isoformat()).split("T")
    return _date_and_time[0].replace("-", "") + _date_and_time[1].replace(":",
"" )[:6]

def write_result(boxes, im_fn, run_name, timers):
    if not os.path.exists("output"):
        os.mkdir("output")
    if not os.path.exists(f"output/{run_name}"):
        os.mkdir(f"output/{run_name}")
    file_name = f"res_{os.path.basename(im_fn).split('.')[0]}.txt"
    with open(os.path.join(f"output/{run_name}", file_name), "w") as f:
        line_txts = []
        for box in boxes:
            line_txts.append(",".join([str(c) for coord in box for c in coord]))
        f.write("\n".join(line_txts))
    if cfg.write_timer:
        with open(os.path.join(f"output/{run_name}", f"timer_{run_name}.json"),
"w") as f:
            json.dump(timers, f)

def infer(m, visualize_inferred_map=False, validation_dataset=False):
    run_name = get_run_name()
    imgs = get_images(validation_dataset)
    timers = []
    for im_fn in imgs:
        print(f"Inferring {im_fn}")
        im = cv2.imread(im_fn)[: , : , ::-1]
        boxes, score, geo, timer = infer_im(m, im)
        timers.append(timer)
        if visualize_inferred_map:
            utils.visualize_inferred(im, score[0, :, :, 0], geo[0, ...])
        if cfg.visualize:
            utils.visualize_boxes(im[: , : , ::-1], boxes)
        if not cfg.dont_write:
            write_result(boxes, im_fn, run_name, timers)
    return f"output/{run_name}"

def infer_im(m, im):
    im_resized, (ratio_h, ratio_w) = resize_image(im)

    timer = {}
    t0 = time.time()
    output = m(tf.convert_to_tensor(im_resized[np.newaxis, :, :, :]))
    timer["network"] = time.time() - t0
    geo, score = tf.split(output, [5, 1], -1)
    boxes, timer = detect(np.array(score), np.array(geo), timer)
    if boxes is not None:

```

```

        boxes = boxes[:, :8].reshape(-1, 4, 2)
        boxes[:, :, 0] /= ratio_w
        boxes[:, :, 1] /= ratio_h
        boxes = boxes.astype(np.int32)
    else:
        boxes = []
    return boxes, score, geo, timer

if __name__ == "__main__":
    m = model()
    m.load_weights(cfg.checkpoint_path).expect_partial()
    infer(m)

train.py

MAX_STEPS = 100000
DECAY_RATE = 0.94

def loss_fun(y_true, y_pred):
    return m.loss(y_true, y_pred)

def train():
    training_name = re.sub("[\-\s\:]'", "", str(datetime.utcnow()))[:14]
    model = m.model(freeze=not cfg.unfreeze)
    if cfg.load:
        model.load_weights(cfg.checkpoint_path).expect_partial()
    lr_schedule = tf.keras.optimizers.schedules.ExponentialDecay(
        cfg.learning_rate, decay_steps=MAX_STEPS, decay_rate=DECAY_RATE
    )
    model.compile(
        optimizer=tf.keras.optimizers.Adam(learning_rate=lr_schedule),
        loss=loss_fun,
        metrics=[loss_fun],
    )

    model_checkpoint_callback = tf.keras.callbacks.ModelCheckpoint(
        filepath="checkpoint/ckpt",
        save_weights_only=True,
        monitor="loss",
        mode="min",
        save_best_only=True,
    )
    history = model.fit(
        IcdarTrainingSequence(),
        validation_data=IcdarValidationSequence(),
        use_multiprocessing=False,
        epochs=cfg.epochs,
        callbacks=[model_checkpoint_callback,
        IcdarEvaluationCallback(training_name)],
    )
    with open(os.path.join(f"training_results_{training_name}", "history",
    "wb")) as f:
        pickle.dump(history.history, f)

if __name__ == "__main__":
    train()

```

```

def polygon_area(poly):
    edge = [
        (poly[1][0] - poly[0][0]) * (poly[1][1] + poly[0][1]),
        (poly[2][0] - poly[1][0]) * (poly[2][1] + poly[1][1]),
        (poly[3][0] - poly[2][0]) * (poly[3][1] + poly[2][1]),
        (poly[0][0] - poly[3][0]) * (poly[0][1] + poly[3][1]),
    ]
    return np.sum(edge) / 2.0

def check_and_validate_polys(polys, tags, xxx_todo_changeme):
    (h, w) = xxx_todo_changeme
    if polys.shape[0] == 0:
        return polys
    polys[:, :, 0] = np.clip(polys[:, :, 0], 0, w - 1)
    polys[:, :, 1] = np.clip(polys[:, :, 1], 0, h - 1)

    validated_polys = []
    validated_tags = []
    for poly, tag in zip(polys, tags):
        p_area = polygon_area(poly)
        if abs(p_area) < 1:
            print("invalid poly")
            continue
        if p_area > 0:
            print("poly in wrong direction")
            poly = poly[(0, 3, 2, 1), :]
        validated_polys.append(poly)
        validated_tags.append(tag)
    return np.array(validated_polys), np.array(validated_tags)

def crop_area(im, polys, tags, crop_background=False, max_tries=50):
    h, w, _ = im.shape
    pad_h = h // 10
    pad_w = w // 10
    h_array = np.zeros((h + pad_h * 2), dtype=np.int32)
    w_array = np.zeros((w + pad_w * 2), dtype=np.int32)
    for poly in polys:
        poly = np.round(poly, decimals=0).astype(np.int32)
        minx = np.min(poly[:, 0])
        maxx = np.max(poly[:, 0])
        w_array[minx + pad_w : maxx + pad_w] = 1
        miny = np.min(poly[:, 1])
        maxy = np.max(poly[:, 1])
        h_array[miny + pad_h : maxy + pad_h] = 1
    h_axis = np.where(h_array == 0)[0]
    w_axis = np.where(w_array == 0)[0]
    if len(h_axis) == 0 or len(w_axis) == 0:
        return im, polys, tags
    for i in range(max_tries):
        xx = np.random.choice(w_axis, size=2)
        xmin = np.min(xx) - pad_w
        xmax = np.max(xx) - pad_w
        xmin = np.clip(xmin, 0, w - 1)
        xmax = np.clip(xmax, 0, w - 1)
        yy = np.random.choice(h_axis, size=2)
        ymin = np.min(yy) - pad_h
        ymax = np.max(yy) - pad_h

```

```

ymin = np.clip(ymin, 0, h - 1)
ymax = np.clip(ymax, 0, h - 1)
if (
    xmax - xmin < cfg.min_crop_side_ratio * w
    or ymax - ymin < cfg.min_crop_side_ratio * h
):
    continue
if polys.shape[0] != 0:
    poly_axis_in_area = (
        (polys[:, :, 0] >= xmin)
        & (polys[:, :, 0] <= xmax)
        & (polys[:, :, 1] >= ymin)
        & (polys[:, :, 1] <= ymax)
    )
    selected_polys = np.where(np.sum(poly_axis_in_area, axis=1) == 4)[0]
else:
    selected_polys = []
if len(selected_polys) == 0:
    if crop_background:
        return (
            im[ymin : ymax + 1, xmin : xmax + 1, :],
            polys[selected_polys],
            tags[selected_polys],
        )
    else:
        continue
im = im[ymin : ymax + 1, xmin : xmax + 1, :]
polys = polys[selected_polys]
tags = tags[selected_polys]
polys[:, :, 0] -= xmin
polys[:, :, 1] -= ymin
return im, polys, tags

return im, polys, tags

def shrink_poly(poly, r):
    R = 0.3
    if np.linalg.norm(poly[0] - poly[1]) + np.linalg.norm(
        poly[2] - poly[3]
    ) > np.linalg.norm(poly[0] - poly[3]) + np.linalg.norm(poly[1] - poly[2]):
        ## p0, p1
        theta = np.arctan2((poly[1][1] - poly[0][1]), (poly[1][0] - poly[0][0]))
        poly[0][0] += R * r[0] * np.cos(theta)
        poly[0][1] += R * r[0] * np.sin(theta)
        poly[1][0] -= R * r[1] * np.cos(theta)
        poly[1][1] -= R * r[1] * np.sin(theta)
        ## p2, p3
        theta = np.arctan2((poly[2][1] - poly[3][1]), (poly[2][0] - poly[3][0]))
        poly[3][0] += R * r[3] * np.cos(theta)
        poly[3][1] += R * r[3] * np.sin(theta)
        poly[2][0] -= R * r[2] * np.cos(theta)
        poly[2][1] -= R * r[2] * np.sin(theta)
        ## p0, p3
        theta = np.arctan2((poly[3][0] - poly[0][0]), (poly[3][1] - poly[0][1]))
        poly[0][0] += R * r[0] * np.sin(theta)
        poly[0][1] += R * r[0] * np.cos(theta)
        poly[3][0] -= R * r[3] * np.sin(theta)
        poly[3][1] -= R * r[3] * np.cos(theta)
        ## p1, p2
        theta = np.arctan2((poly[2][0] - poly[1][0]), (poly[2][1] - poly[1][1]))
        poly[1][0] += R * r[1] * np.sin(theta)

```

```

poly[1][1] += R * r[1] * np.cos(theta)
poly[2][0] -= R * r[2] * np.sin(theta)
poly[2][1] -= R * r[2] * np.cos(theta)
else:
    ## p0, p3
    theta = np.arctan2((poly[3][0] - poly[0][0]), (poly[3][1] - poly[0][1]))
    poly[0][0] += R * r[0] * np.sin(theta)
    poly[0][1] += R * r[0] * np.cos(theta)
    poly[3][0] -= R * r[3] * np.sin(theta)
    poly[3][1] -= R * r[3] * np.cos(theta)
    ## p1, p2
    theta = np.arctan2((poly[2][0] - poly[1][0]), (poly[2][1] - poly[1][1]))
    poly[1][0] += R * r[1] * np.sin(theta)
    poly[1][1] += R * r[1] * np.cos(theta)
    poly[2][0] -= R * r[2] * np.sin(theta)
    poly[2][1] -= R * r[2] * np.cos(theta)
    ## p0, p1
    theta = np.arctan2((poly[1][1] - poly[0][1]), (poly[1][0] - poly[0][0]))
    poly[0][0] += R * r[0] * np.cos(theta)
    poly[0][1] += R * r[0] * np.sin(theta)
    poly[1][0] -= R * r[1] * np.cos(theta)
    poly[1][1] -= R * r[1] * np.sin(theta)
    ## p2, p3
    theta = np.arctan2((poly[2][1] - poly[3][1]), (poly[2][0] - poly[3][0]))
    poly[3][0] += R * r[3] * np.cos(theta)
    poly[3][1] += R * r[3] * np.sin(theta)
    poly[2][0] -= R * r[2] * np.cos(theta)
    poly[2][1] -= R * r[2] * np.sin(theta)
return poly

def point_dist_to_line(p1, p2, p3):
    x = np.linalg.norm(np.cross(p2 - p1, p1 - p3)) / np.linalg.norm(p2 - p1)
    return x

def fit_line(p1, p2):
    if p1[0] == p1[1]:
        return [1.0, 0.0, -p1[0]]
    else:
        [k, b] = np.polyfit(p1, p2, deg=1)
        return [k, -1.0, b]

def line_cross_point(line1, line2):
    if line1[0] != 0 and line1[0] == line2[0]:
        print("Cross point does not exist")
        return None
    if line1[0] == 0 and line2[0] == 0:
        print("Cross point does not exist")
        return None
    if line1[1] == 0:
        x = -line1[2]
        y = line2[0] * x + line2[2]
    elif line2[1] == 0:
        x = -line2[2]
        y = line1[0] * x + line1[2]
    else:
        k1, _, b1 = line1
        k2, _, b2 = line2
        x = -(b1 - b2) / (k1 - k2)
        y = k1 * x + b1

```

```

    return np.array([x, y], dtype=np.float32)

def line_vertical(line, point):
    if line[1] == 0:
        vertical = [0, -1, point[1]]
    else:
        if line[0] == 0:
            vertical = [1, 0, -point[0]]
        else:
            vertical = [-1.0 / line[0], -1, point[1] - (-1 / line[0] *
point[0])]
    return vertical

def rectangle_from_parallelogram(poly):
    p0, p1, p2, p3 = poly
    angle_p0 = np.arccos(
        np.dot(p1 - p0, p3 - p0) / (np.linalg.norm(p0 - p1) * np.linalg.norm(p3
- p0))
    )
    if angle_p0 < 0.5 * np.pi:
        if np.linalg.norm(p0 - p1) > np.linalg.norm(p0 - p3):
            # p0 and p2
            ## p0
            p2p3 = fit_line([p2[0], p3[0]], [p2[1], p3[1]])
            p2p3_vertical = line_vertical(p2p3, p0)

            new_p3 = line_cross_point(p2p3, p2p3_vertical)
            ## p2
            p0p1 = fit_line([p0[0], p1[0]], [p0[1], p1[1]])
            p0p1_vertical = line_vertical(p0p1, p2)

            new_p1 = line_cross_point(p0p1, p0p1_vertical)
            return np.array([p0, new_p1, p2, new_p3], dtype=np.float32)
        else:
            p1p2 = fit_line([p1[0], p2[0]], [p1[1], p2[1]])
            p1p2_vertical = line_vertical(p1p2, p0)

            new_p1 = line_cross_point(p1p2, p1p2_vertical)
            p0p3 = fit_line([p0[0], p3[0]], [p0[1], p3[1]])
            p0p3_vertical = line_vertical(p0p3, p2)

            new_p3 = line_cross_point(p0p3, p0p3_vertical)
            return np.array([p0, new_p1, p2, new_p3], dtype=np.float32)
    else:
        if np.linalg.norm(p0 - p1) > np.linalg.norm(p0 - p3):
            # p1 and p3
            ## p1
            p2p3 = fit_line([p2[0], p3[0]], [p2[1], p3[1]])
            p2p3_vertical = line_vertical(p2p3, p1)

            new_p2 = line_cross_point(p2p3, p2p3_vertical)
            ## p3
            p0p1 = fit_line([p0[0], p1[0]], [p0[1], p1[1]])
            p0p1_vertical = line_vertical(p0p1, p3)

            new_p0 = line_cross_point(p0p1, p0p1_vertical)
            return np.array([new_p0, p1, new_p2, p3], dtype=np.float32)
        else:
            p0p3 = fit_line([p0[0], p3[0]], [p0[1], p3[1]])
            p0p3_vertical = line_vertical(p0p3, p1)

```



```

        new_p0 = line_cross_point(p0p3, p0p3_vertical)
        plp2 = fit_line([p1[0], p2[0]], [p1[1], p2[1]])
        plp2_vertical = line_vertical(plp2, p3)

        new_p2 = line_cross_point(plp2, plp2_vertical)
        return np.array([new_p0, p1, new_p2, p3], dtype=np.float32)

def sort_rectangle(poly):
    p_lowest = np.argmax(poly[:, 1])
    if np.count_nonzero(poly[:, 1] == poly[p_lowest, 1]) == 2:
        p0_index = np.argmin(np.sum(poly, axis=1))
        p1_index = (p0_index + 1) % 4
        p2_index = (p0_index + 2) % 4
        p3_index = (p0_index + 3) % 4
        return poly[[p0_index, p1_index, p2_index, p3_index]], 0.0
    else:
        p_lowest_right = (p_lowest - 1) % 4
        p_lowest_left = (p_lowest + 1) % 4
        angle = np.arctan(
            -(poly[p_lowest][1] - poly[p_lowest_right][1])
            / (poly[p_lowest][0] - poly[p_lowest_right][0])
        )
        if angle <= 0:
            print(angle, poly[p_lowest], poly[p_lowest_right])
        if angle / np.pi * 180 > 45:
            p2_index = p_lowest
            p1_index = (p2_index - 1) % 4
            p0_index = (p2_index - 2) % 4
            p3_index = (p2_index + 1) % 4
            return poly[[p0_index, p1_index, p2_index, p3_index]], -(np.pi / 2 -
angle)
        else:
            p3_index = p_lowest
            p0_index = (p3_index + 1) % 4
            p1_index = (p3_index + 2) % 4
            p2_index = (p3_index + 3) % 4
            return poly[[p0_index, p1_index, p2_index, p3_index]], angle

def restore_rectangle_rbox(origin, geometry):
    d = geometry[:, :4]
    angle = geometry[:, 4]
    origin_0 = origin[angle >= 0]
    d_0 = d[angle >= 0]
    angle_0 = angle[angle >= 0]
    if origin_0.shape[0] > 0:
        p = np.array(
            [
                np.zeros(d_0.shape[0]),
                -d_0[:, 0] - d_0[:, 2],
                d_0[:, 1] + d_0[:, 3],
                -d_0[:, 0] - d_0[:, 2],
                d_0[:, 1] + d_0[:, 3],
                np.zeros(d_0.shape[0]),
                np.zeros(d_0.shape[0]),
                np.zeros(d_0.shape[0]),
                d_0[:, 3],
                -d_0[:, 2],
            ]
        )

```

```

p = p.transpose((1, 0)).reshape((-1, 5, 2)) # N*5*2

rotate_matrix_x = np.array([np.cos(angle_0),
np.sin(angle_0)]).transpose((1, 0))
rotate_matrix_x = (
    np.repeat(rotate_matrix_x, 5, axis=1).reshape(-1, 2,
5).transpose((0, 2, 1))
) # N*5*2

rotate_matrix_y = np.array([-np.sin(angle_0),
np.cos(angle_0)]).transpose(
    (1, 0)
)
rotate_matrix_y = (
    np.repeat(rotate_matrix_y, 5, axis=1).reshape(-1, 2,
5).transpose((0, 2, 1))
)

p_rotate_x = np.sum(rotate_matrix_x * p, axis=2)[:, :, np.newaxis] #
N*5*1
p_rotate_y = np.sum(rotate_matrix_y * p, axis=2)[:, :, np.newaxis] #
N*5*1

p_rotate = np.concatenate([p_rotate_x, p_rotate_y], axis=2) # N*5*2

p3_in_origin = origin_0 - p_rotate[:, 4, :]
new_p0 = p_rotate[:, 0, :] + p3_in_origin # N*2
new_p1 = p_rotate[:, 1, :] + p3_in_origin
new_p2 = p_rotate[:, 2, :] + p3_in_origin
new_p3 = p_rotate[:, 3, :] + p3_in_origin

new_p_0 = np.concatenate(
    [
        new_p0[:, np.newaxis, :],
        new_p1[:, np.newaxis, :],
        new_p2[:, np.newaxis, :],
        new_p3[:, np.newaxis, :],
    ],
    axis=1,
) # N*4*2
else:
    new_p_0 = np.zeros((0, 4, 2))
    origin_1 = origin[angle < 0]
    d_1 = d[angle < 0]
    angle_1 = angle[angle < 0]
    if origin_1.shape[0] > 0:
        p = np.array(
            [
                -d_1[:, 1] - d_1[:, 3],
                -d_1[:, 0] - d_1[:, 2],
                np.zeros(d_1.shape[0]),
                -d_1[:, 0] - d_1[:, 2],
                np.zeros(d_1.shape[0]),
                np.zeros(d_1.shape[0]),
                -d_1[:, 1] - d_1[:, 3],
                np.zeros(d_1.shape[0]),
                -d_1[:, 1],
                -d_1[:, 2],
            ]
        )
    p = p.transpose((1, 0)).reshape((-1, 5, 2)) # N*5*2

```

```

        rotate_matrix_x = np.array([np.cos(-angle_1), -np.sin(-
angle_1)]).transpose(
            (1, 0)
        )
        rotate_matrix_x = (
            np.repeat(rotate_matrix_x, 5, axis=1).reshape(-1, 2,
5).transpose((0, 2, 1))
        ) # N*5*2

        rotate_matrix_y = np.array([np.sin(-angle_1), np.cos(-
angle_1)]).transpose(
            (1, 0)
        )
        rotate_matrix_y = (
            np.repeat(rotate_matrix_y, 5, axis=1).reshape(-1, 2,
5).transpose((0, 2, 1))
        )

    p_rotate_x = np.sum(rotate_matrix_x * p, axis=2)[:, :, np.newaxis] #
N*5*1
    p_rotate_y = np.sum(rotate_matrix_y * p, axis=2)[:, :, np.newaxis] #
N*5*1

    p_rotate = np.concatenate([p_rotate_x, p_rotate_y], axis=2) # N*5*2

    p3_in_origin = origin_1 - p_rotate[:, 4, :]
    new_p0 = p_rotate[:, 0, :] + p3_in_origin # N*2
    new_p1 = p_rotate[:, 1, :] + p3_in_origin
    new_p2 = p_rotate[:, 2, :] + p3_in_origin
    new_p3 = p_rotate[:, 3, :] + p3_in_origin

    new_p_1 = np.concatenate(
        [
            new_p0[:, np.newaxis, :],
            new_p1[:, np.newaxis, :],
            new_p2[:, np.newaxis, :],
            new_p3[:, np.newaxis, :],
        ],
        axis=1,
    ) # N*4*2
else:
    new_p_1 = np.zeros((0, 4, 2))
return np.concatenate([new_p_0, new_p_1])

def generate_rbox(im_size, polys, tags):
    h, w = im_size
    poly_mask = np.zeros((h, w), dtype=np.uint8)
    score_map = np.zeros((h, w), dtype=np.uint8)
    geo_map = np.zeros((h, w, 5), dtype=np.float32)
    training_mask = np.ones((h, w), dtype=np.uint8)
    for poly_idx, poly_tag in enumerate(zip(polys, tags)):
        poly = poly_tag[0]
        tag = poly_tag[1]

        r = [None, None, None, None]
        for i in range(4):
            r[i] = min(
                np.linalg.norm(poly[i] - poly[(i + 1) % 4]),
                np.linalg.norm(poly[i] - poly[(i - 1) % 4]),
            )

```

```

shrunken_poly = shrink_poly(poly.copy(), r).astype(np.int32)[np.newaxis,
:, :]
cv2.fillPoly(score_map, shrunken_poly, 1)
cv2.fillPoly(poly_mask, shrunken_poly, poly_idx + 1)
poly_h = min(
    np.linalg.norm(poly[0] - poly[3]), np.linalg.norm(poly[1] - poly[2])
)
poly_w = min(
    np.linalg.norm(poly[0] - poly[1]), np.linalg.norm(poly[2] - poly[3])
)
if min(poly_h, poly_w) < cfg.min_text_size:
    cv2.fillPoly(training_mask, poly.astype(np.int32)[np.newaxis, :, :],
0)

if tag:
    cv2.fillPoly(training_mask, poly.astype(np.int32)[np.newaxis, :, :],
0)

xy_in_poly = np.argwhere(poly_mask == (poly_idx + 1))
box = cv2.minAreaRect(
    poly
)
(p0_rect, p1_rect, p2_rect, p3_rect), angle =
sort_rectangle(cv2.boxPoints(box))
for y, x in xy_in_poly:
    point = np.array([x, y], dtype=np.float32)
    geo_map[y, x, 0] = point_dist_to_line(p0_rect, p1_rect, point)
    geo_map[y, x, 1] = point_dist_to_line(p1_rect, p2_rect, point)
    geo_map[y, x, 2] = point_dist_to_line(p2_rect, p3_rect, point)
    geo_map[y, x, 3] = point_dist_to_line(p3_rect, p0_rect, point)
    geo_map[y, x, 4] = angle
return score_map, geo_map, training_mask

def infer_and_test(model):
    output_path = infer.infer(model, validation_dataset=True)
    zip_path = "tmp_results.zip"
    with ZipFile(zip_path, "w") as zipObj:
        for fn in os.listdir(output_path):
            if not ".txt" in fn:
                continue
            zipObj.write(os.path.join(output_path, fn), fn)

res_dic = script.main(zip_path)
return res_dic["method"]

```